A PRACTICAL GUIDE FOR NEW DEVELOPERS

PGE-PBIP Masterclass

How the GibbonGambits/PGE-PBIP-Template repository works, and how to build, check and share Power BI dashboards with VS Code, Git, GitHub and GitHub Copilot.

Facts checked against the repository and official documentation on 10 November 2026.

Contents



The guide at a glance. Start here is two pages. Parts 1 and 2 explain the pieces and how they connect; Part 3 walks through the common tasks; Part 4 is for looking things up.

Chapter	Page
Start here	3
1. Power BI projects: what the files are	5
2. Git and GitHub for this repo	10
3. VS Code and GitHub Copilot	16
4. How the parts work together	20
5. Rules for the AI: steering, specs and hooks	27
6. Set up your computer	31
7. Task: build a dashboard page	35
8. Task: check your work	39
9. Task: change the theme	45
10. Tests and GitHub automation	47
11. Custom visuals (the .pbiviz kit)	50
12. The Python email and alert guides	55
13. Which file do I touch?	57
14. Known gaps and how to fix them	61
Appendix A. File-by-file directory	64
Appendix B. Verified official references	85
Appendix C. Glossary	87

Start here

What this repository is

PGE-PBIP-Template is a starter kit for building Power BI dashboards with an AI coding assistant. You clone it, open it in VS Code, describe the page you want, and let GitHub Copilot (or Claude) build it, inside rules the team has written down.

The trick that makes this work: the dashboard is not stored as one binary `.pbix` file. It is stored as a **PBIP** (Power BI Project): a folder of small text files. Text files mean Git can show you every change, and an AI agent can read and edit them like any other code.

Around those files the repo adds three things: written rules the AI loads every time, checklists for quality, and PowerShell scripts that check the files and then screenshot what Power BI actually drew.

Think of it as a **restaurant franchise kit**:

In a franchise kit	In this repo
An empty, fitted kitchen	The blank report and data model (`Template.pbip`)
The house recipe book	The rule files the AI reads (`.github/copilot-instructions.md` and friends)
Recipe cards for specific dishes	Skills: step-by-step procedures in `.github/skills/`
The order ticket for tonight's table	A spec: your plan for one page, in `specs/`
Food-safety checklists	`docs/checklists/`
The inspector who tastes every plate	The verify loop: scripts that screenshot every page

The colour code you'll see everywhere

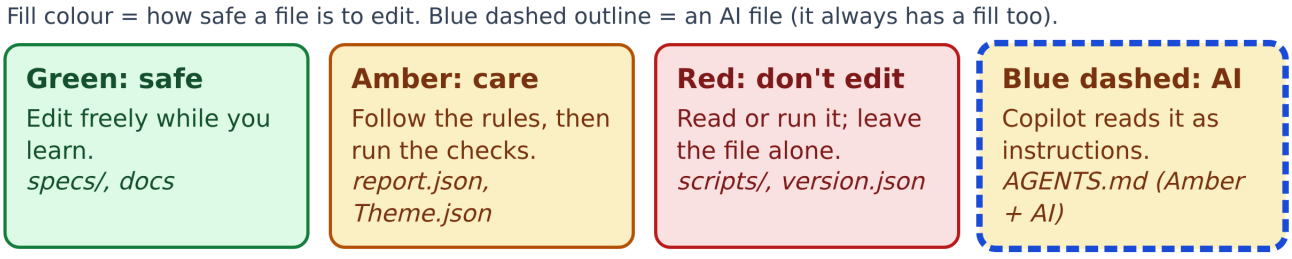


Figure 0.1 — The colour legend. Every diagram and table in this guide uses it.

1. **Green:** safe to edit while you learn.
2. **Amber:** edit with care, follow the rules, then run the checks.
3. **Red:** read it or run it, but don't change the file.
4. **Blue dashed outline:** an AI file that Copilot or Claude reads as instructions. It always has a fill colour too.

The whole system in one picture

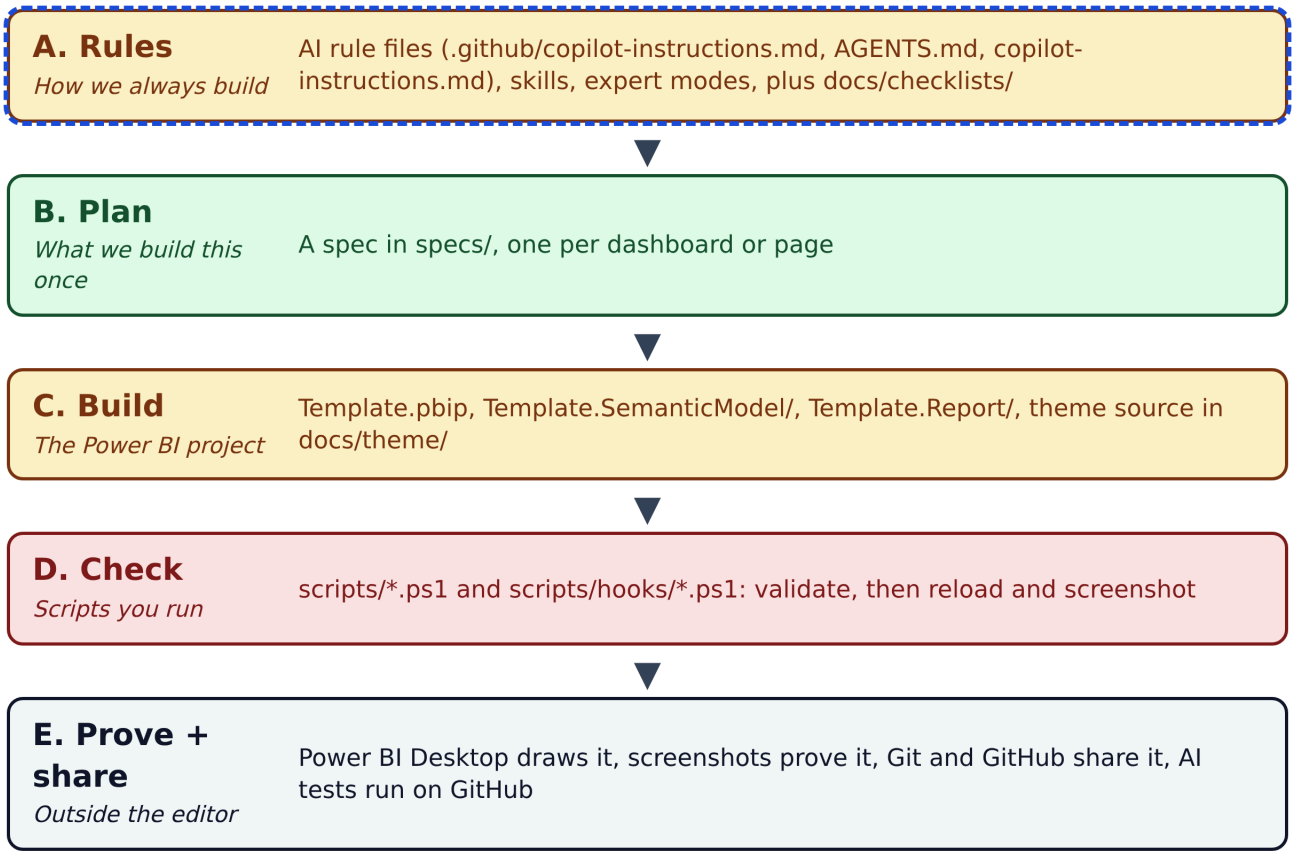


Figure 0.2 — The master map. Read it top to bottom: every folder in the repo belongs to one of these five stages.

- **A. Rules:** how the team always builds. Permanent, loaded by the AI every session.
- **B. Plan:** a spec for the one page you are building now.
- **C. Build:** the Power BI project files the AI (or you) edit.
- **D. Check:** scripts that validate the files, reload Power BI and screenshot each page.
- **E. Prove and share:** you review the screenshots, then Git and GitHub share the work.

Your everyday workflow

1. **Branch.** In VS Code, create a new Git branch for your change.
2. **Plan.** Copy `specs/TEMPLATE.md` to `specs/<page-name>.md` and fill it in.
3. **Build.** Ask Copilot: “Build the page in `specs/<page-name>.md`.” It reads the rules, then edits the project files.
4. **Check.** Run the three validators in `scripts/`, then the verify loop to screenshot every page.
5. **Review.** Look at the screenshots yourself. Valid files are not enough; the page must look right.
6. **Share.** Commit, push, and open a pull request. A reviewer approves and merges.

The rest of this guide explains each piece, then walks through the common tasks. The full file-by-file list is in Appendix A.

1. Power BI projects: what the files are

Power BI Desktop is Microsoft's free Windows app for building reports. A report is a set of pages full of **visuals** (charts, tables, cards). Behind the report sits a **semantic model**: the tables, the relationships between them, and the **measures** (named calculations such as "Total Sales", written in Power BI's formula language, **DAX**).

This chapter shows how this repo stores all of that as text.

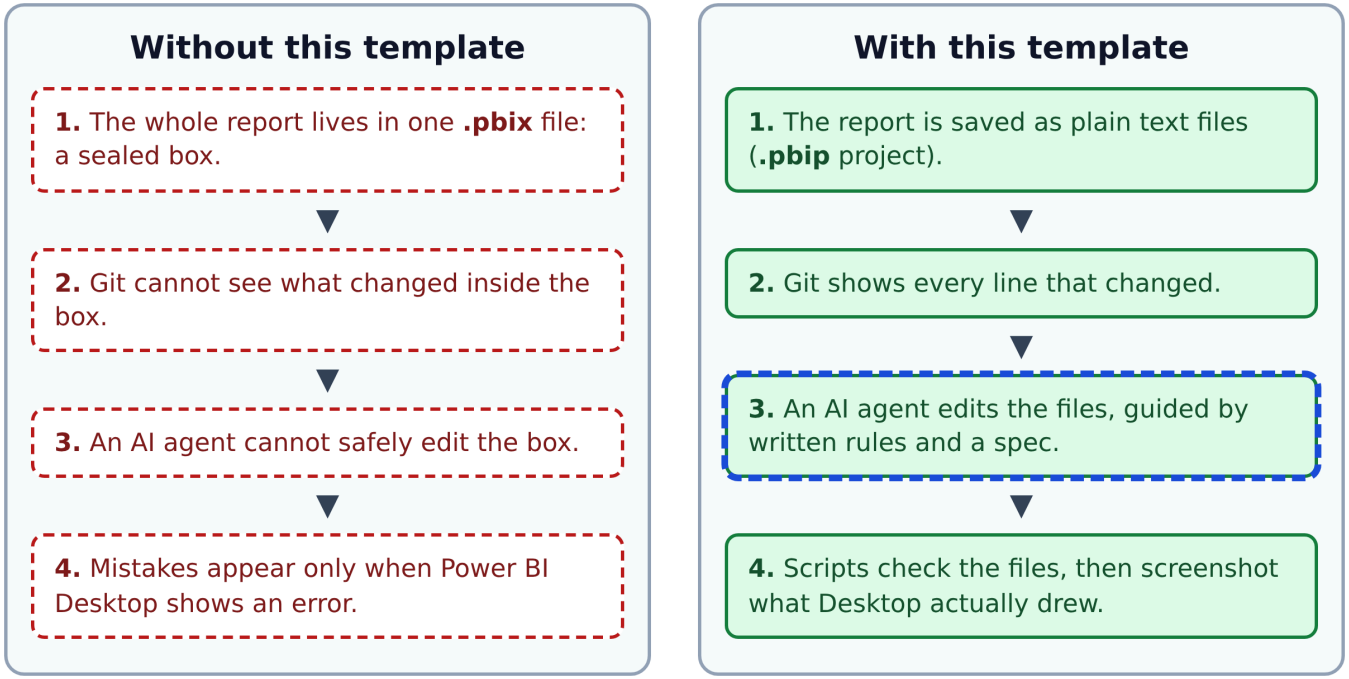


Figure 1.1 — Why the repo uses text files. Left: the classic single-file approach. Right: this template.

1. A `.pbix` file is one compressed bundle. You can't open it in an editor.
2. So Git can only say "the file changed", never what changed.
3. An AI agent can't safely edit it either.
4. With a project folder, scripts can check the files before Power BI ever opens them, and screenshots catch what still looks wrong.

.pbix vs .pbip

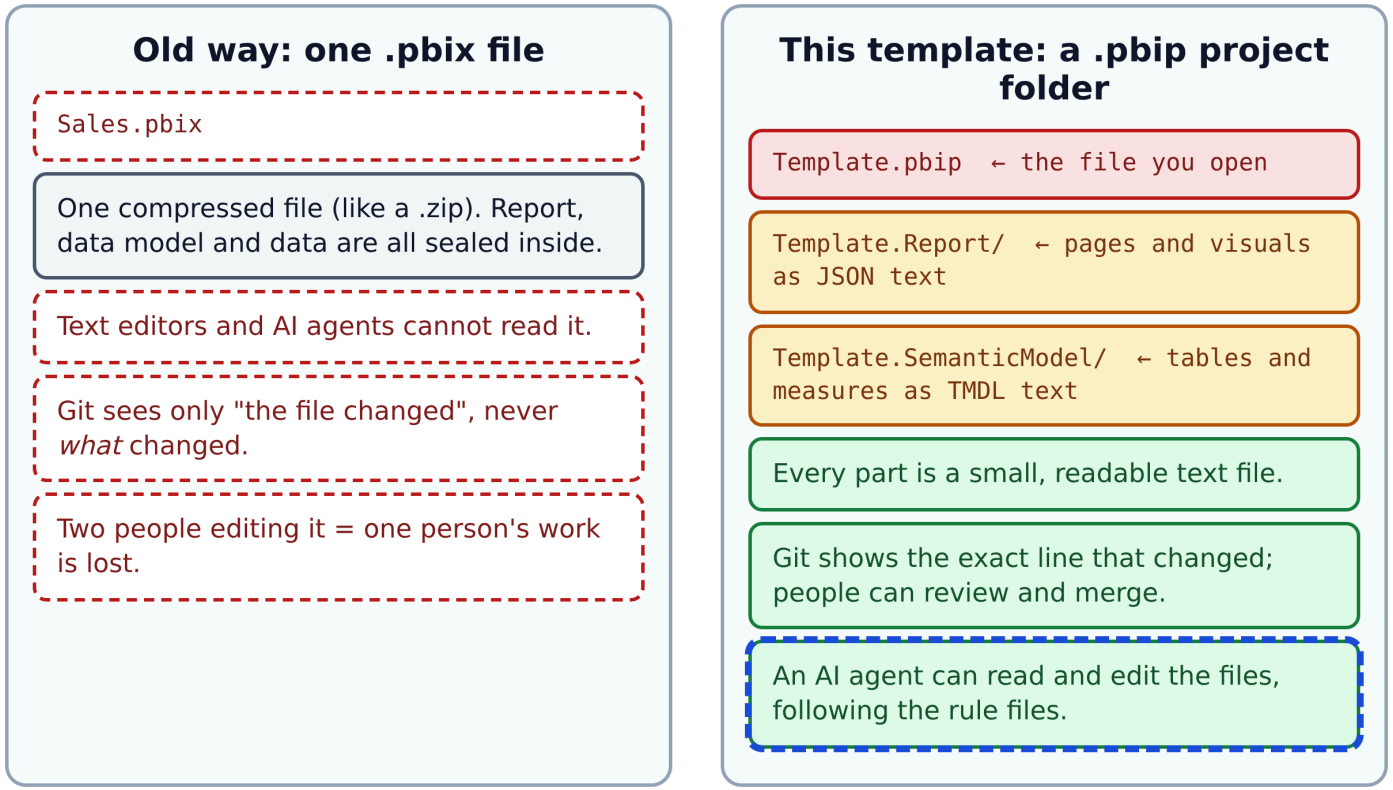


Figure 1.2 — One sealed file vs. a project folder.

1. `.pbix` seals report, model and data into one file.
2. `.pbip` (Power BI Project) saves the same thing as a folder. Microsoft's docs: the definitions are saved "as individual plain text files".
3. `Template.pbip` is the file you double-click. It just points to the report folder.
4. `Template.Report/` holds pages and visuals as JSON, in a layout called **PBIR** (Power BI enhanced report format), which Microsoft now uses by default.
5. `Template.SemanticModel/` holds the model as **TMDL** (Tabular Model Definition Language): indented text, a bit like YAML, with no braces.

What you get on day one

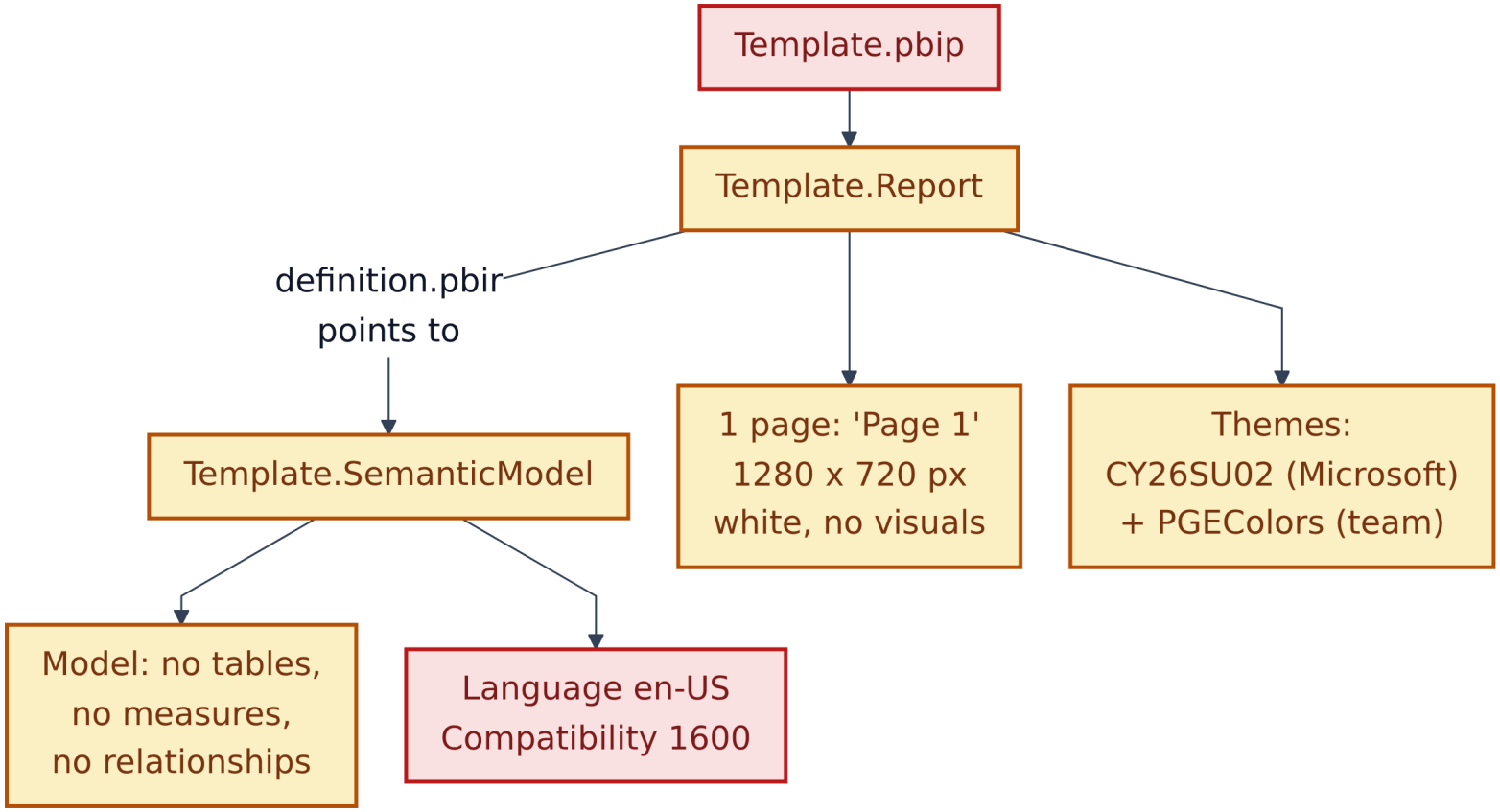


Figure 1.3 — The starting state. The template is deliberately almost empty.

1. `Template.pbip` opens the report.
2. The report has one blank page, “Page 1”, 1280 × 720 pixels.
3. Two themes are registered: Microsoft’s base theme `CY26SU02` and the team’s `PGEColors`.
4. `definition.pbir` points the report at the model.
5. The model has no tables, measures or relationships yet.
6. Language (en-US) and compatibility level (1600) are fixed settings. Leave them alone.

Inside the report folder

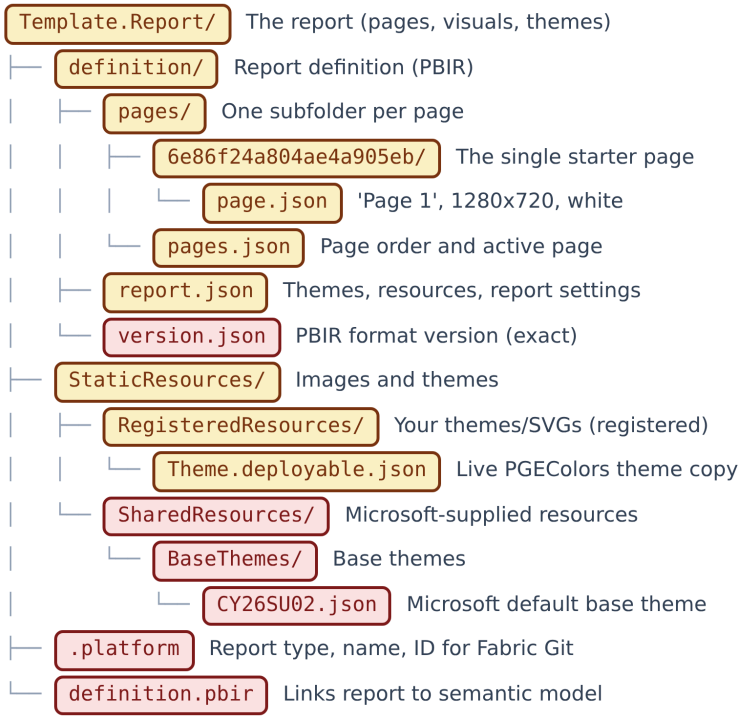


Figure 1.4 — `Template.Report/`. Each label shows its safety colour.

1. `definition/pages/` has one folder per page. The only page has a generated id, `6e86f24a804ae4a905eb`; its `page.json` names it “Page 1”.
2. `pages.json` sets the page order and which page opens first.
3. `report.json` registers the themes and resources. You’ll edit it when you add a theme or background image.
4. `version.json` must contain exact content, so it is Red.
5. `StaticResources/RegisteredResources/` is where your themes and SVG backgrounds go. Each must also be listed in `report.json`.
6. `.platform` is an ID file for **Microsoft Fabric** (Microsoft’s online analytics platform) when syncing with Git. Don’t touch it.

As you build, each visual gets its own `pages/<page-id>/visuals/<visual-id>/visual.json`.

Inside the model folder

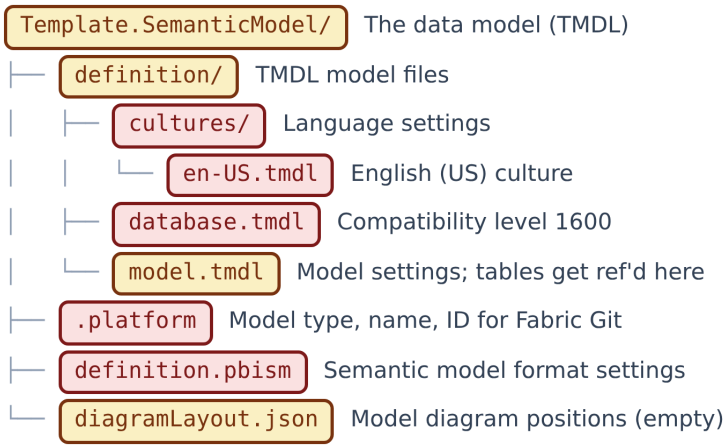


Figure 1.5 — `Template.SemanticModel/`.

1. `database.tmdl` sets the compatibility level. Red.
2. `model.tmdl` holds model-wide settings. Each new table gets a `ref table` line here. Amber.
3. `cultures/en-US.tmdl`, `definition.pbism` and `.platform` are fixed settings. Red.
4. `diagramLayout.json` remembers where tables sit in Power BI's diagram view. Empty for now.

New tables go in `definition/tables/<TableName>.tmdl`.

How the files find each other

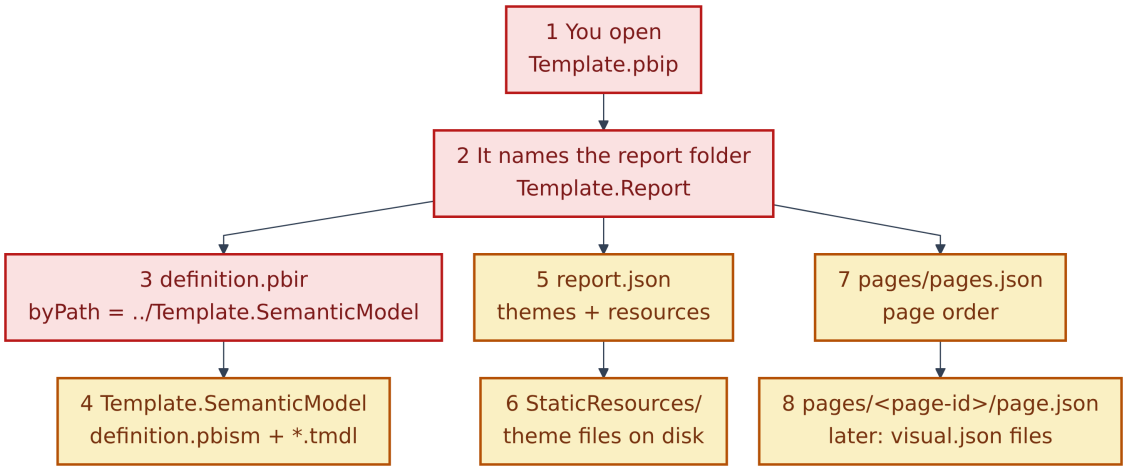


Figure 1.6 — The pointer chain. Each file tells Power BI where to look next.

1. You open `Template.pbip`.
2. It names the report folder, `Template.Report`.
3. `definition.pbir` points to `../Template.SemanticModel`.
4. Power BI loads the model from there.
5. `report.json` lists the themes and resources…
6. …which must exist on disk under `StaticResources/`.
7. `pages.json` gives the page order…
8. …and each `page.json` (and later `visual.json`) describes one page or visual.

If any name in this chain is wrong, the report won't open. The scripts in Chapter 8 check these links for you.

Model vs. report

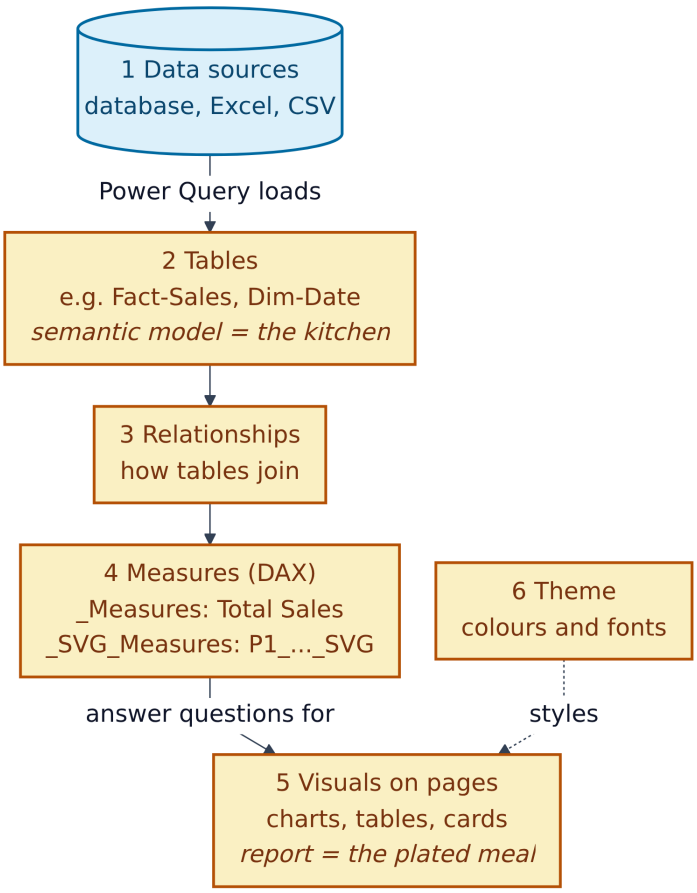


Figure 1.7 — From data to picture.

1. Data sources: a database, Excel, CSV.

2. Power Query (Power BI's data-loading tool) loads them into tables. The checklists prefer a **star schema**: *fact* tables of events (sales) surrounded by *dimension* tables of things (products, dates).

3. Relationships join the tables.

4. Measures turn rows into answers. Business measures live in a table called `_Measures`; measures that draw SVG picture cards live in `_SVG_Measures`.

5. Visuals on pages show the answers.

6. The theme styles every visual.

Three file rules to remember

- **Close Power BI Desktop before editing files by hand**, or it may save over your edits. The scripts will reload it for you.
- **Save `.tmdl` files as UTF-8 without a BOM** (a byte-order mark: an invisible marker at the start of a file). Power BI's TMDL reader rejects it. PowerShell's `Set-Content -Encoding UTF8` adds one, so the rulebook says to write files from Python instead.
- **No `//` comments in TMDL.** Put a `///` description line above each measure instead; it becomes the measure's description.

2. Git and GitHub for this repo

Git records the history of your files on your machine. **GitHub** hosts that history online and adds reviews and automation. GitHub's own docs put it this way: GitHub is "based on the open-source software, Git".



Figure 2.1 — The life of a change.

1. The original template on GitHub is called **upstream**.

2. A **fork** is your own copy on GitHub, made with the *Fork* button. It "lets you propose changes to a project without affecting the upstream repository". Your fork is usually called **origin**.

3. `git clone` downloads your fork, with full history.

4. A **branch** is a separate line of work, e.g. `feature/sales-page`.

5. You (or the agent) edit files.

6. A **commit** saves a snapshot with a message. Repeat 5–6 as often as you like.

7. `git push` uploads your commits to your fork.

8. A **pull request** (PR) asks the team to review and merge your branch.

9. After review, the branch is merged into `dev`, then `test`, then `main`.

Why fork instead of just clone?

Because this is a template: each team or dashboard gets its own copy to rename and customise, and nothing you try can break the original. You still get template updates by pulling from upstream. The recipe from `docs/REPOSITORY-GUIDE.md`:

```
git remote add upstream https://github.com/<upstream-owner>/<upstream-repo>.git
git fetch upstream
git checkout main
git merge upstream/main
```

The `<upstream-owner>/<upstream-repo>` part is a placeholder: for this template it is `GibbonGambits/PGE-PBIP-Template`. After merging, rerun the validators (Chapter 8).

Where a change lives

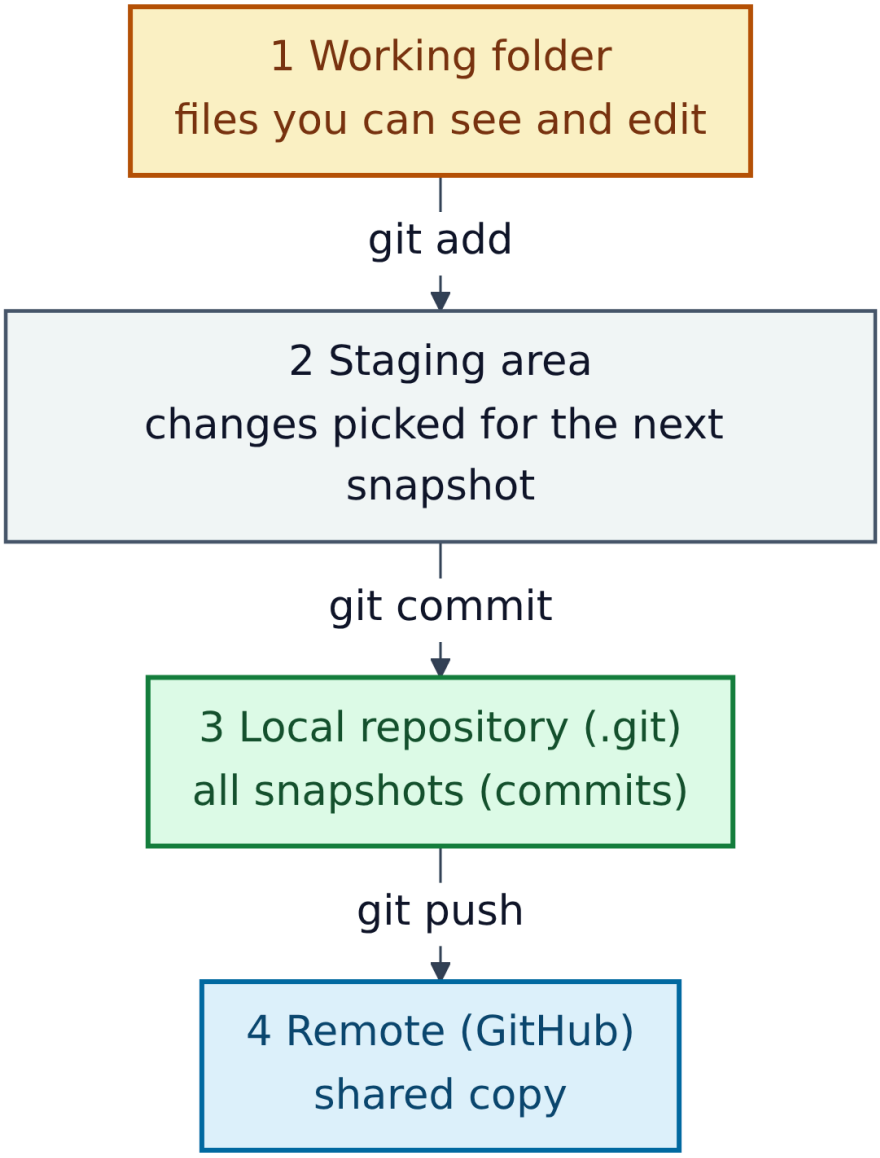


Figure 2.2 — The four places a change passes through.

1. Working folder: the files you edit.

2. Staging area: `git add` picks which changes go into the next commit.

3. Local repository: `git commit` stores the snapshot in the hidden `.git` folder.

4. Remote: `git push` sends commits to GitHub; `git pull` brings others' commits down.

VS Code's **Source Control** view (`Ctrl+Shift+G`) does all of this with buttons.

The branch path the repo recommends

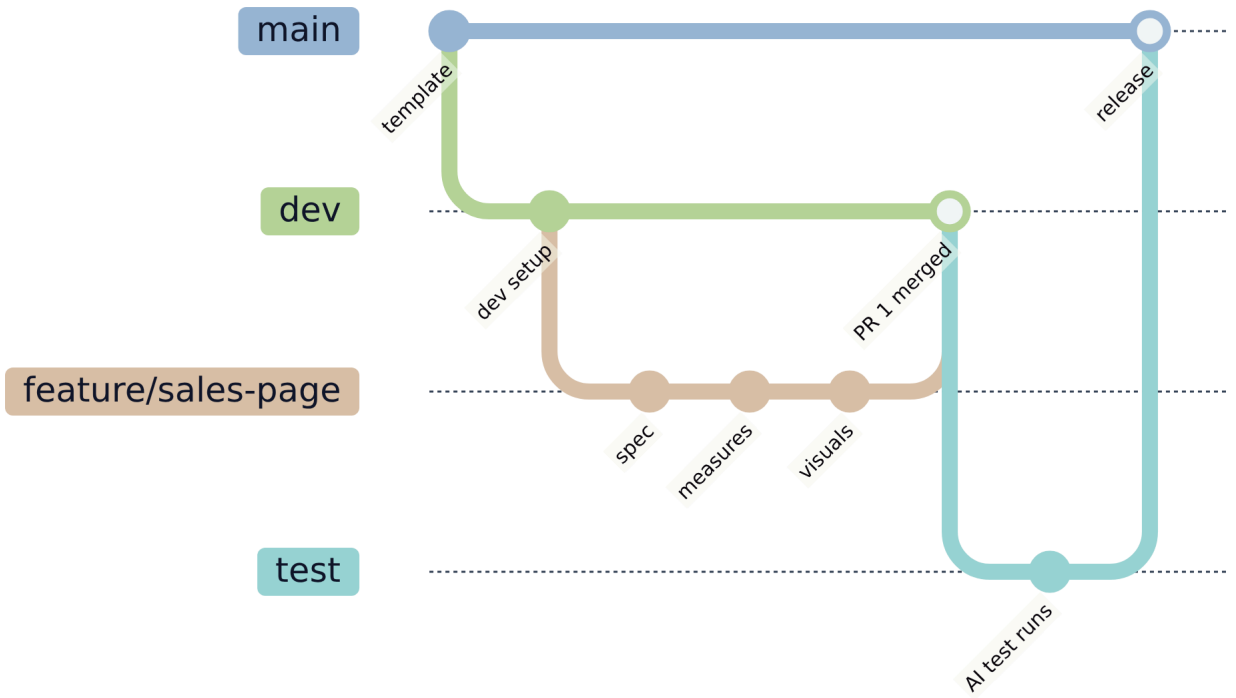


Figure 2.3 — Branches over time. Each line is a branch; each dot is a commit.

1. `main` holds what is released.
2. `dev` collects finished features.
3. `feature/sales-page` is your short-lived branch.
4. Your pull request into `dev` is merged.
5. `test` is the release candidate. Pushing here runs the AI test workflow (Chapter 10).
6. `test` is merged into `main` to release.

`docs/CICD-SETUP.md` also recommends protecting `dev`, `test` and `main`: require a pull request, passing checks and at least one reviewer.

Who does what when you save work

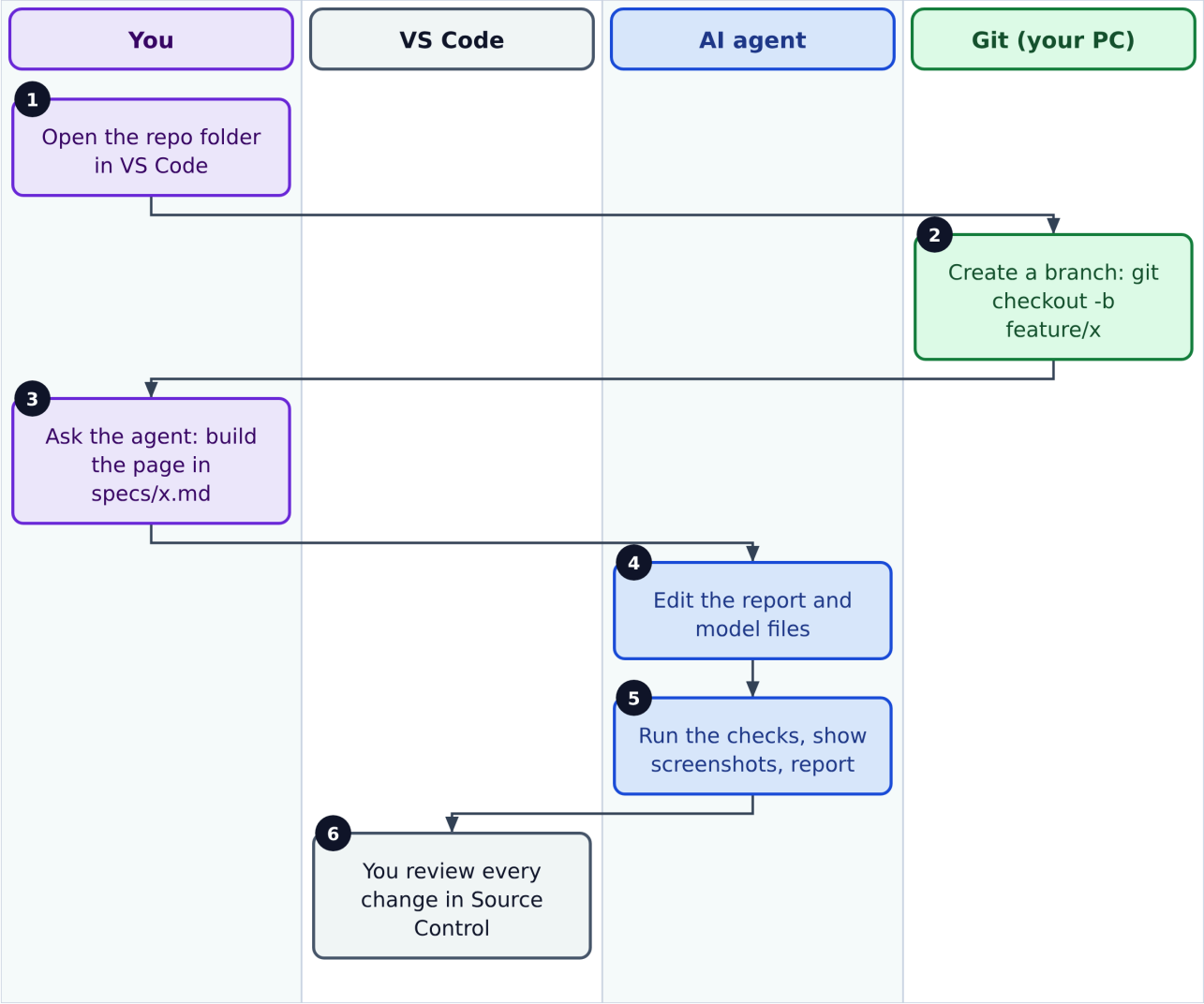


Figure 2.4 — Building the change. Each column is one person or tool; follow the numbers.

1. You open the repo folder in VS Code.
2. Git creates your branch.
3. You ask the agent to build what the spec describes.
4. The agent edits the report and model files.
5. The agent runs the checks and shows you screenshots.
6. You read every changed line in Source Control.

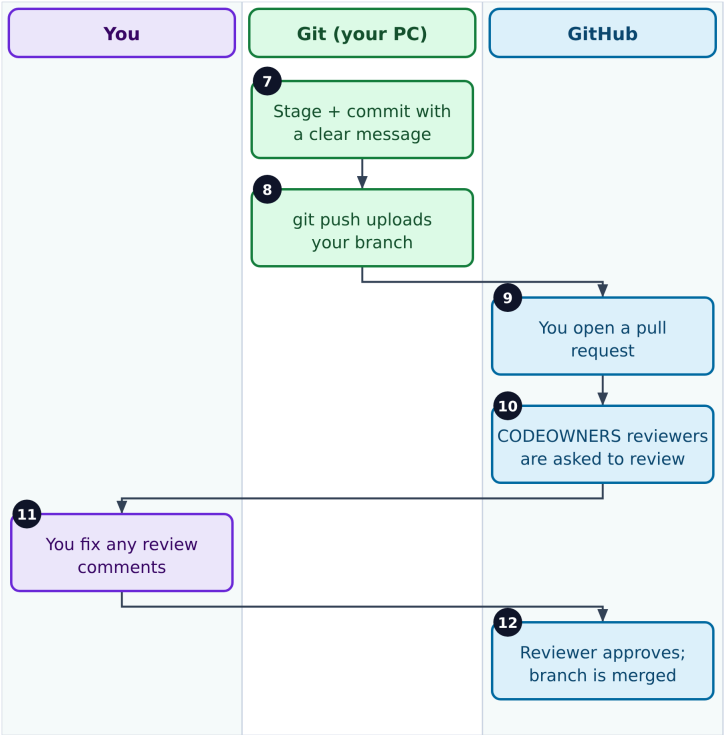


Figure 2.5 — Saving and sharing. Continues from step 6.

7. Git commits your changes with a clear message.
8. `git push` uploads the branch.
9. You open a pull request. It starts from `.github/PULL_REQUEST_TEMPLATE.md`: a summary and a three-item checklist.
10. GitHub asks the owners in `.github/CODEOWNERS` to review.
11. You fix any review comments by pushing more commits.
12. A reviewer approves and merges.

Git-related files in the repo

File	What it does
`.gitignore`	Keeps Power BI's local caches (`**/.pbi/…`), your `.env`, Python caches, `tests/screenshots/`, `.github/.pbip-state/` and `Proposals/` out of Git
`.github/CODEOWNERS`	Who reviews which folder. The `@your-org/...` handles are placeholders to replace
`.github/PULL_REQUEST_TEMPLATE.md`	The starting text of every pull request
`.github/ISSUE_TEMPLATE/`	Forms for bug reports and feature requests
`CONTRIBUTING.md`	Where new skills, specs and tests go

3. VS Code and GitHub Copilot

VS Code is Microsoft's free code editor. **GitHub Copilot** is GitHub's AI assistant inside it. In *agent* mode Copilot does more than chat: VS Code's docs say "an agent uses a model and tools to find relevant code, make changes, and run checks."

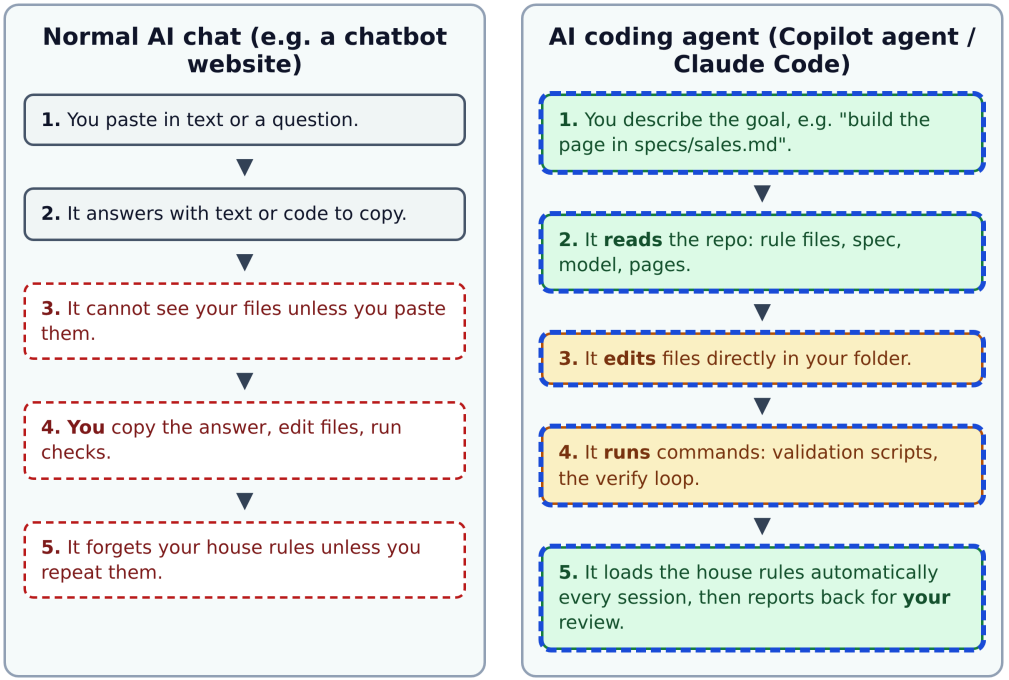


Figure 3.1 — A chatbot advises; an agent does the work.

1. With a chatbot you paste a question. With an agent you describe a goal, like "build the page in `specs/sales.md`".
2. The agent reads your repo: rules, spec, model, pages.
3. It edits files directly in your folder.
4. It runs commands, such as the validation scripts, and reads the output.
5. It loads the house rules automatically, then reports back for your review.

You stay responsible: read every change before you commit it.

The VS Code window

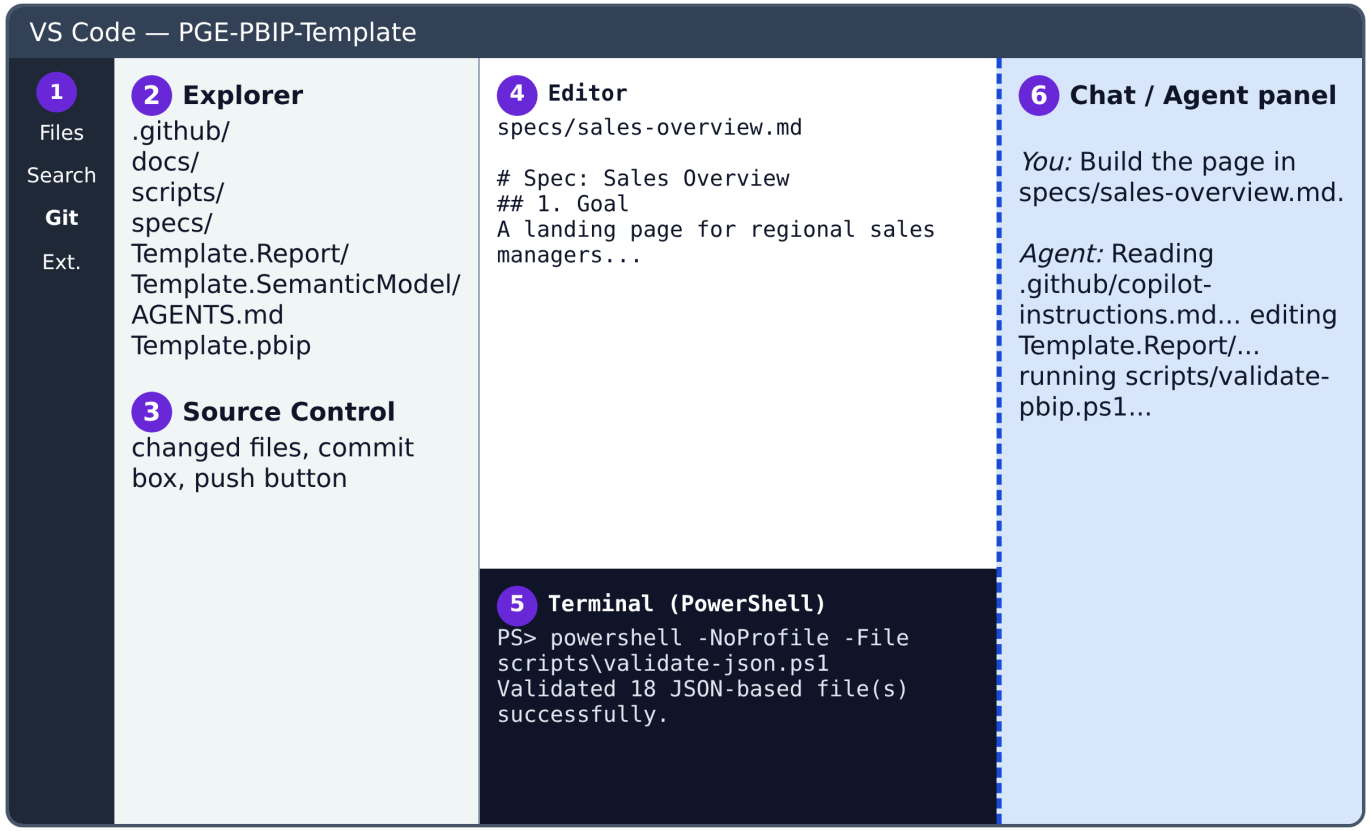


Figure 3.2 — The six areas you'll use.

1. **Activity bar:** switch between Files, Search, Git and Extensions.
2. **Explorer:** open the whole `PGE-PBIP-Template` folder, not single files.
3. **Source Control:** changed files, commit message box, push and pull. Git must be installed separately.
4. **Editor:** where you read and edit files.
5. **Terminal:** PowerShell inside VS Code, for running the repo's scripts.
6. **Chat / agent panel:** where you talk to Copilot and watch it work.

How Copilot learns this repo’s rules

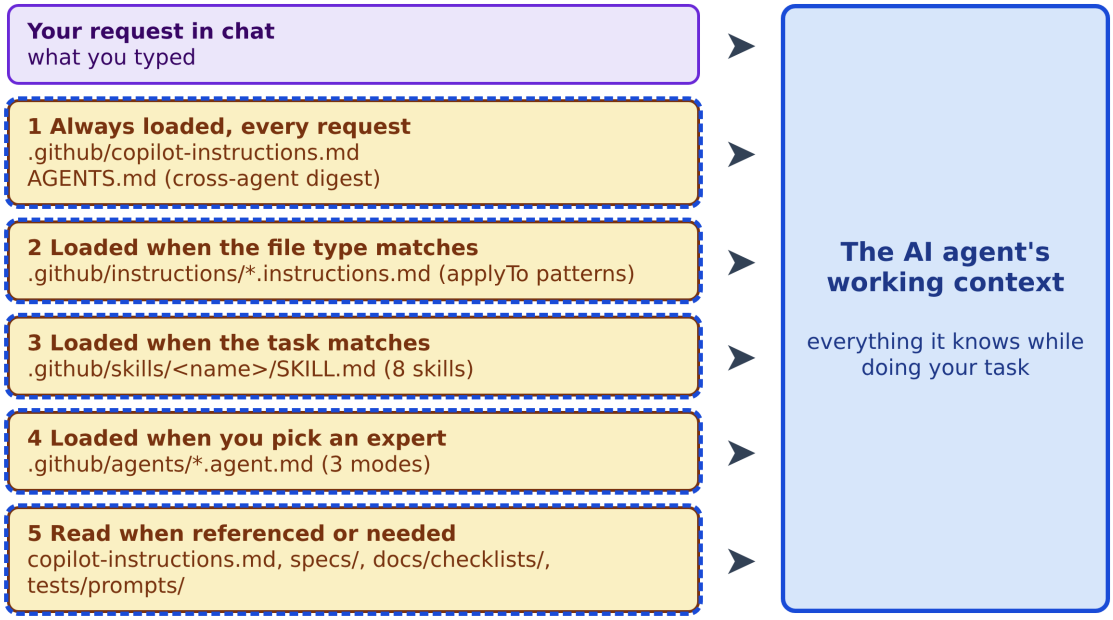


Figure 3.3 — Five ways instructions reach the agent. All are Amber + AI: changing them changes the agent’s behaviour for everyone.

1. **Always loaded: Copilot instructions** are a Markdown file of standing rules, here `.github/copilot-instructions.md`. GitHub’s docs: they “are automatically added to requests that you submit to Copilot”. Copilot also reads **`AGENTS.md`**, a cross-tool format; Claude Code reads it too when there is no `CLAUDE.md`.
2. **By file type:** files in `.github/instructions/` end in `.instructions.md` and have an `applyTo` pattern, e.g. `**/*.{ts,tsx,js,jsx,less,css}`. They load only for matching files.
3. **By task: skills** are folders with a `SKILL.md` procedure. The agent loads one when your task matches its `description`. This repo has eight.
4. **By choice: custom agents** (`.github/agents/*.agent.md`) are expert roles you pick: data modelling, DAX, and visualisation.
5. **When needed:** the deep rulebook `copilot-instructions.md` (727 lines), your spec, the checklists and the test prompts.

The agent’s working cycle

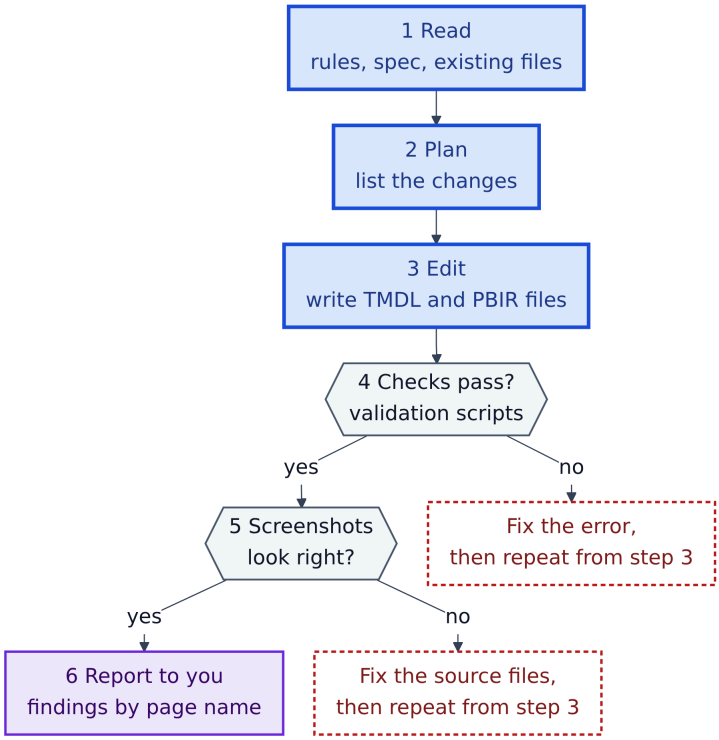


Figure 3.4 — What the agent does with your request.

1. Read the rules, the spec and the existing files.
2. Plan the changes.
3. Edit the TMDL and PBIR files.
4. Run the validation scripts; on failure, fix and repeat from step 3.
5. Screenshot every page; if something looks wrong, fix the source files (never the image) and repeat from step 3.
6. Report findings by page *display name* (“Sales Overview”, not a page id).

Copilot and Claude in this repo

	GitHub Copilot	Claude Code
Main rule file	`.github/copilot-instructions.md`	`AGENTS.md` (when there is no `CLAUDE.md`)
Skills	`.github/skills/*/SKILL.md`	Claude Code looks in `.claude/skills/`; these skills are written for Copilot
Expert modes	`.github/agents/*.agent.md`	Not used
Hooks	Run by hand (Chapter 5)	The hook scripts read the JSON that Claude Code hooks send and use exit code 2 to block, matching Claude Code’s hook design

The README also asks for a one-time step in **GitHub Copilot CLI** (Copilot in a terminal): adding Microsoft’s `skills-for-fabric` plugin collection. Chapter 6 has the commands.

Habits that keep you safe

- Write the spec first. A clear plan means fewer guesses.
- Work on a branch, never directly on `main`.
- Never paste secrets into a chat.
- “The JSON is valid” is not done. Ask for the screenshots.

4. How the parts work together

Now that you know the pieces, here is how one change flows through all of them.

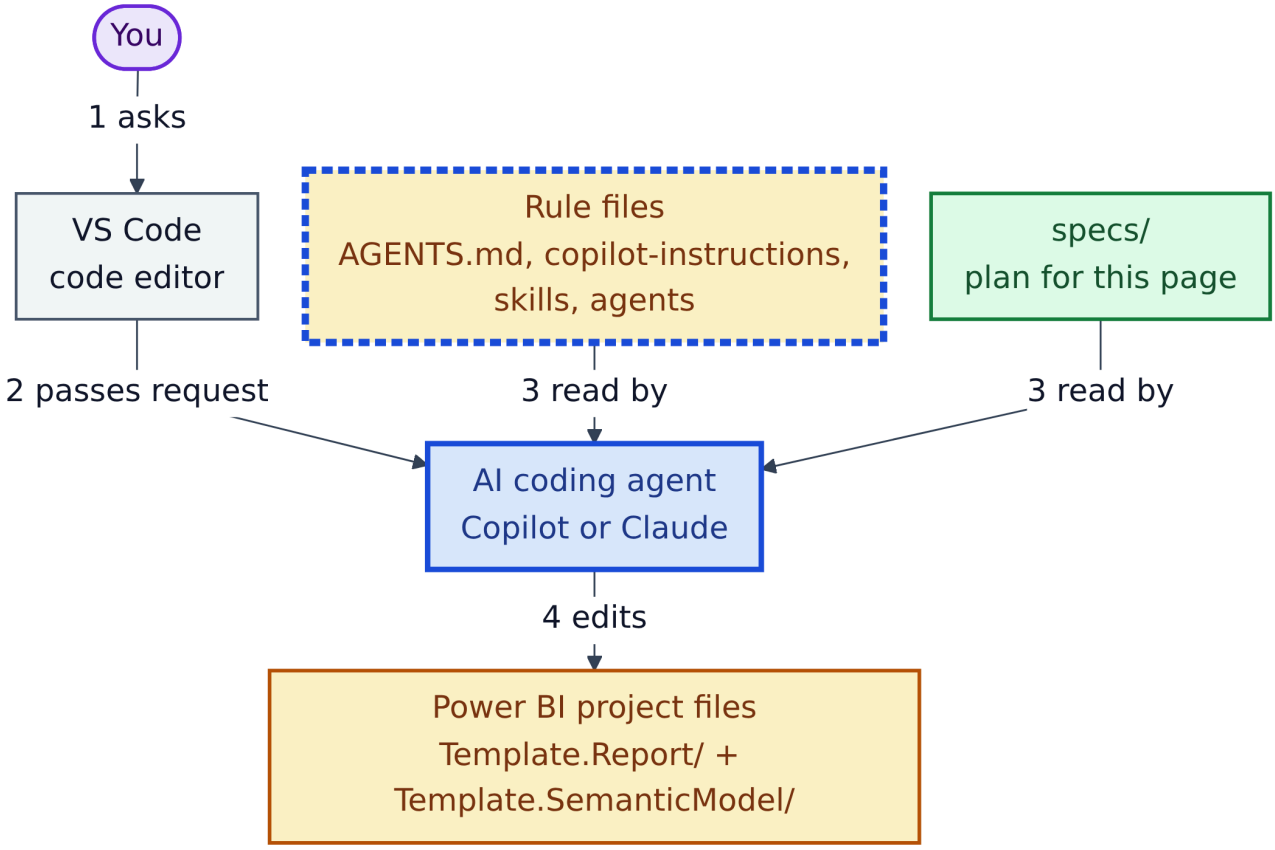


Figure 4.1 — Making a change.

1. You ask for a change in VS Code.
2. VS Code passes it to the AI agent.
3. The agent reads the rule files and your spec.
4. The agent edits the Power BI project files.

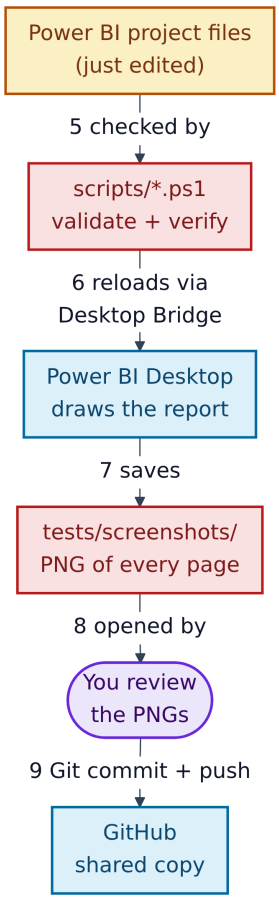


Figure 4.2 — Checking and sharing. Continues from Figure 4.1.

5. PowerShell scripts check the edited files.

6. The verify loop reloads Power BI Desktop through the **Desktop Bridge**: a local connection that lets programs control a running Power BI Desktop.

7. Power BI saves a screenshot of every page in `tests/screenshots/`.

8. You open and review the screenshots. If something is wrong, start again at step 1.

9. You commit and push to GitHub.

The order things run in

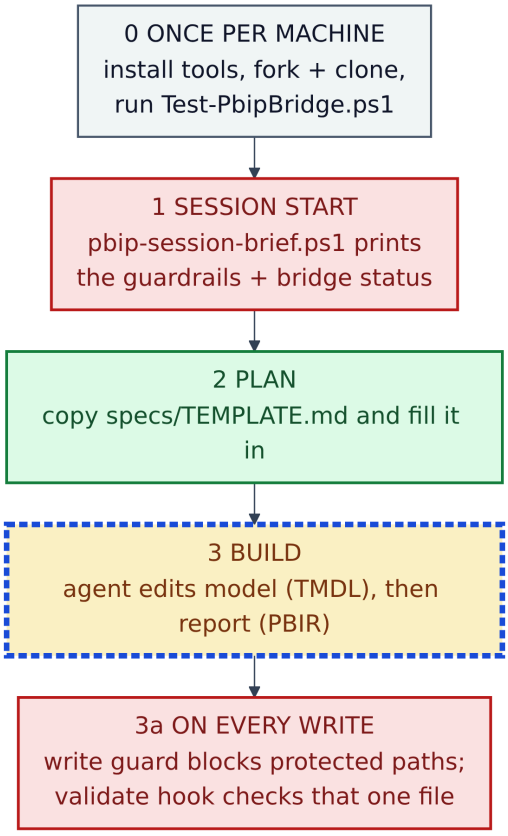


Figure 4.3 — Run order, part 1.

0. Once per machine: install tools, fork and clone, run the preflight `scripts\Test-PbipBridge.ps1` (Chapter 6).

1. Session start: `scripts\hooks\pbip-session-brief.ps1` prints the guardrails and whether the bridge is up.

2. Plan: copy and fill in a spec.

3. Build: the agent edits the model first, then the report.

3a, on every write: a write guard blocks protected folders, and a validator checks the file just written.

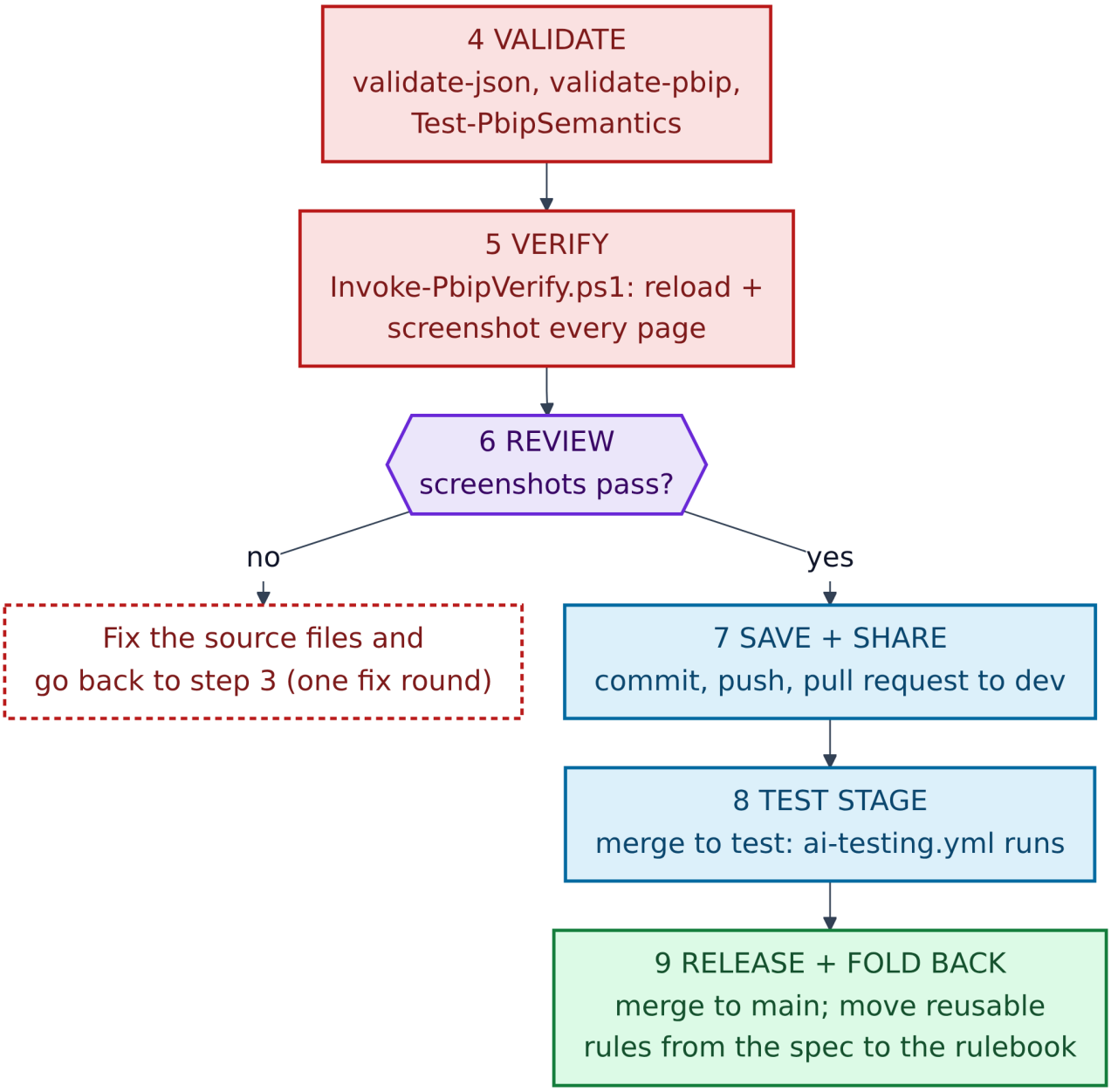


Figure 4.4 — Run order, part 2. Continues from step 3.

4. **Validate:** `validate-json.ps1`, `validate-pbip.ps1`, `Test-PbipSemantics.ps1`.
5. **Verify:** `Invoke-PbipVerify.ps1` reloads Power BI and screenshots every page.
6. **Review:** if screenshots fail, fix the source and go back to step 3. The repo allows **one fix round**: fix once, then confirm and report.
7. **Save and share:** commit, push, open a pull request into `dev`.
8. **Test stage:** merging to `test` runs `.github/workflows/ai-testing.yml`.
9. **Release and fold back:** merge to `main`; move any rule worth keeping from the spec into the rulebook.

Who does what, end to end

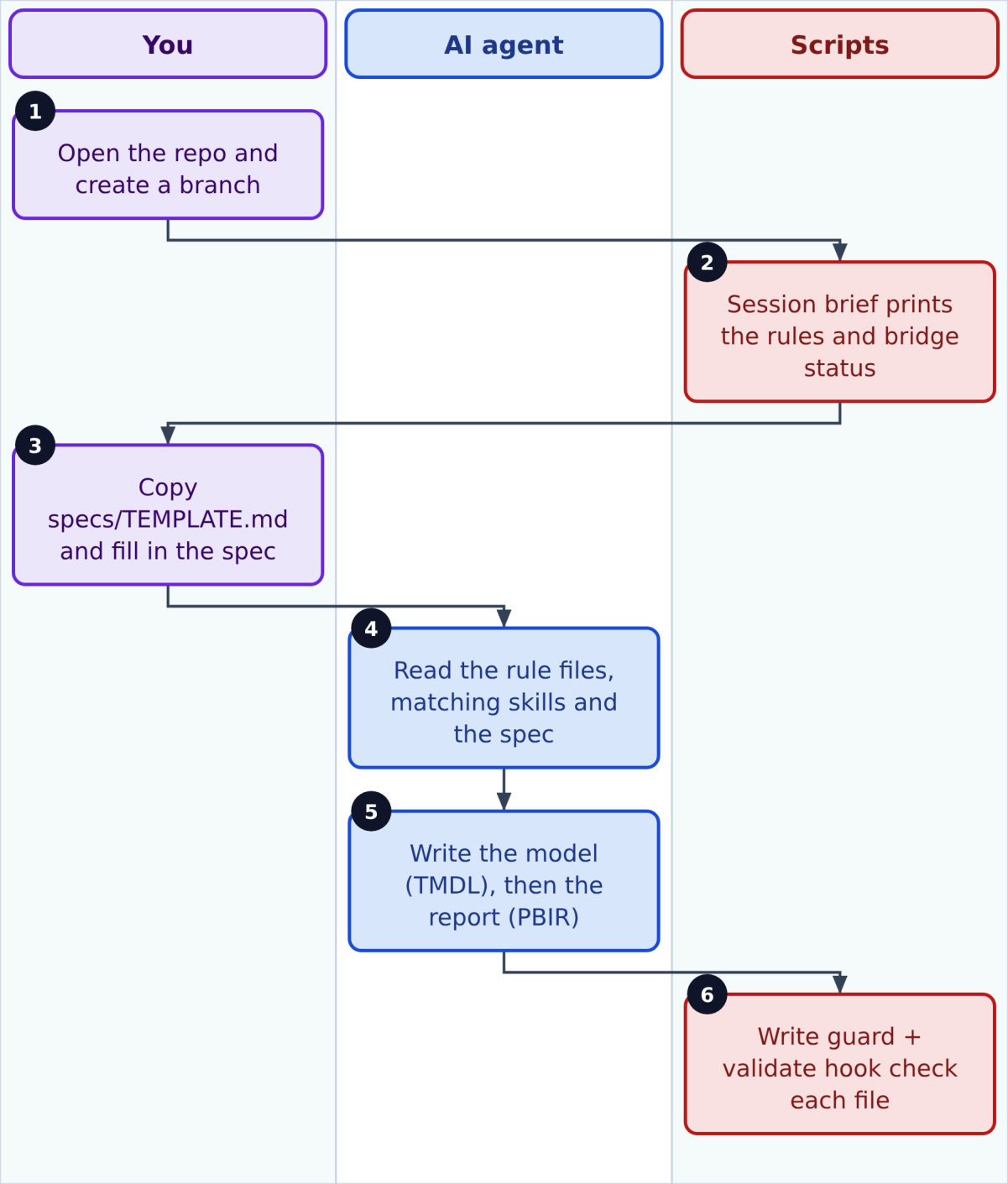


Figure 4.5 — Plan and build (steps 1–6).

1. You open the repo and create a branch.
2. The session brief prints the rules and bridge status.
3. You copy `specs/TEMPLATE.md` and fill it in.
4. The agent reads the rules, matching skills and your spec.
5. The agent writes the model (TMDL), then the report (PBIR).
6. The write guard and validator check each file.

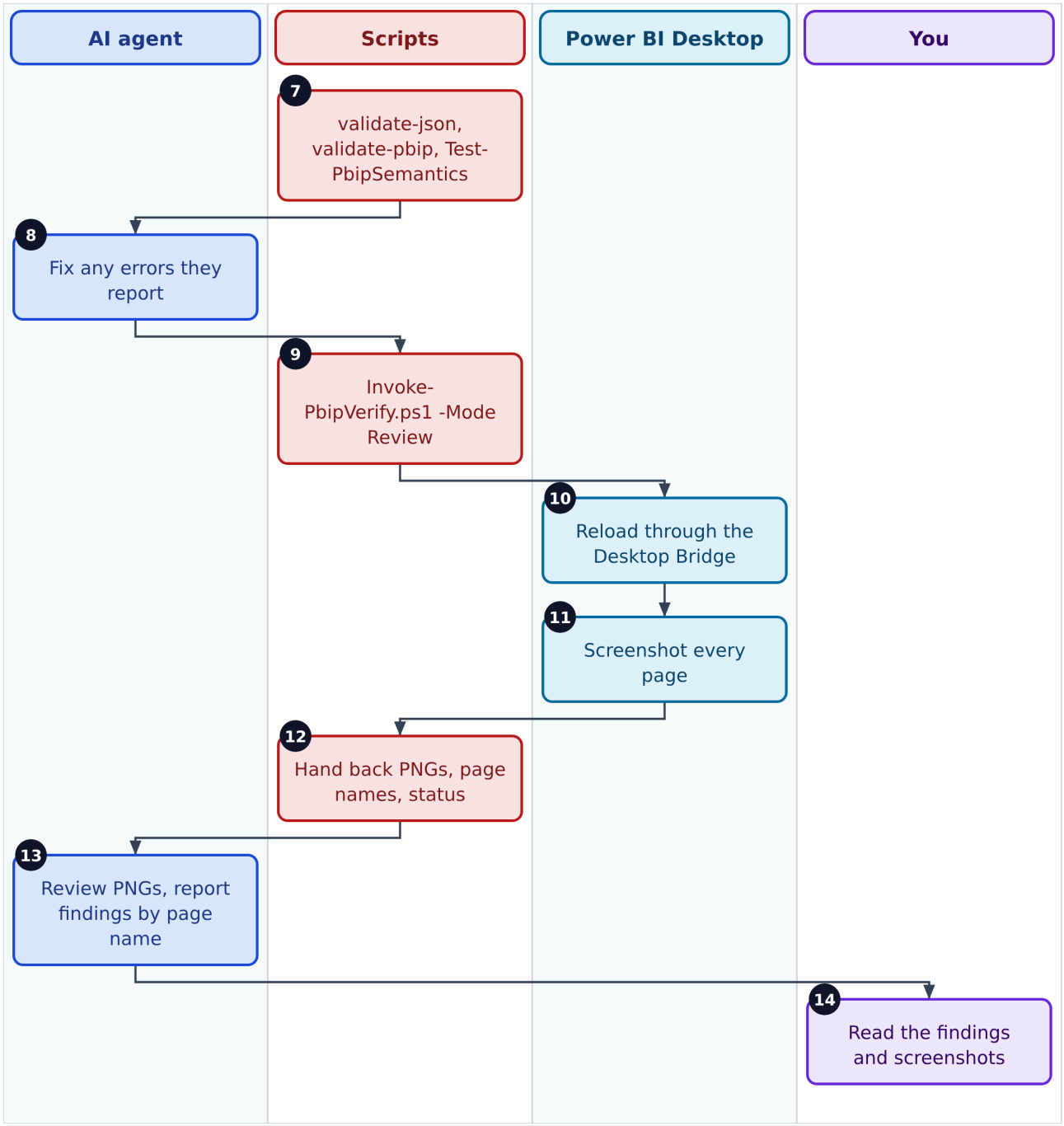


Figure 4.6 — Check and verify (steps 7–14).

7. The three validators run.
8. The agent fixes any errors.
9. The agent runs `Invoke-PbipVerify.ps1 -Mode Review`.
10. Power BI reloads through the bridge.
11. Power BI screenshots every page.
12. The scripts hand back the PNGs, page names and a status.
13. The agent reviews them and reports findings by page name.
14. You check the findings and screenshots yourself.

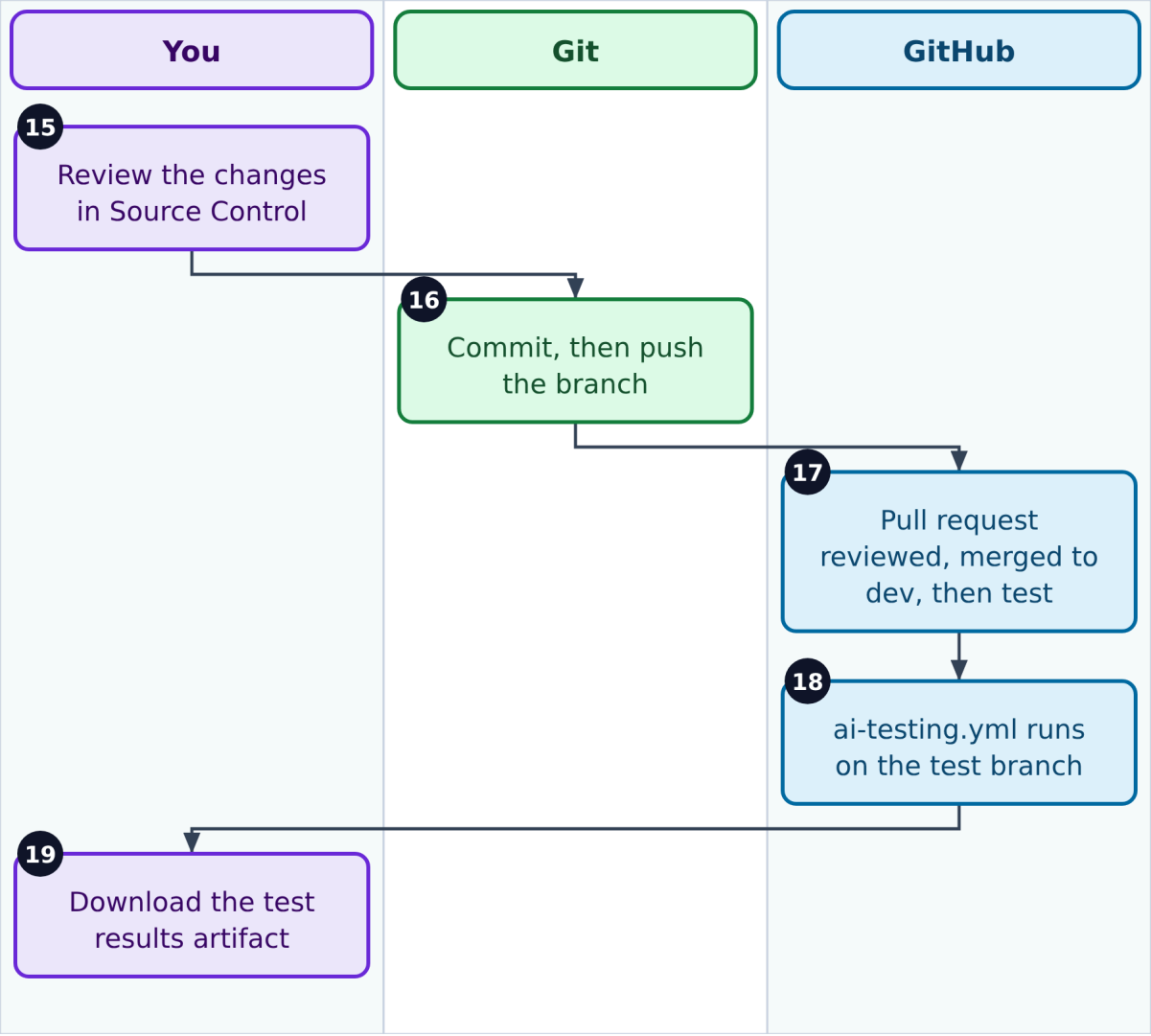


Figure 4.7 — Save and share (steps 15–19).

15. You review the changes in Source Control.
16. Git commits and pushes the branch.
17. The pull request is reviewed and merged to `dev`, later to `test`.
18. On `test`, the AI testing workflow runs.
19. You download the results from the workflow run.

Which script uses which

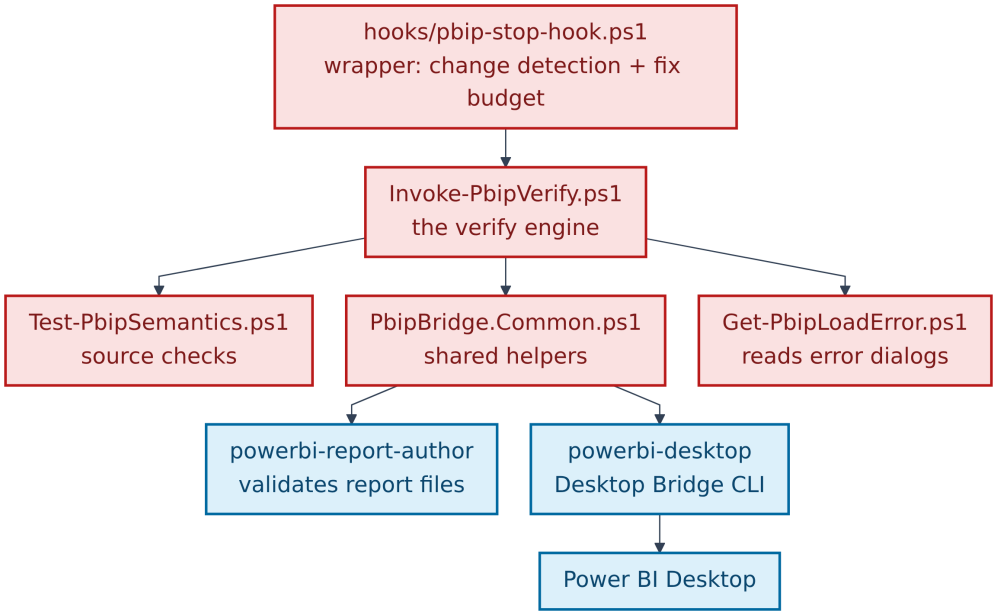


Figure 4.8 — The verify engine. An arrow means “loads or calls”. All scripts are Red: run them, don’t edit them.

1. `hooks/pbip-stop-hook.ps1` wraps the engine, adding change detection and the fix budget.
2. `Invoke-PbipVerify.ps1` is the engine.
3. It loads `Test-PbipSemantics.ps1` and the shared helpers in `PbipBridge.Common.ps1`, and starts `Get-PbipLoadError.ps1` when Power BI shows an error dialog.
4. The helpers call Microsoft’s two command-line tools: `powerbi-report-author` (validates report files) and `powerbi-desktop` (the Desktop Bridge CLI).
5. `powerbi-desktop` talks to Power BI Desktop.

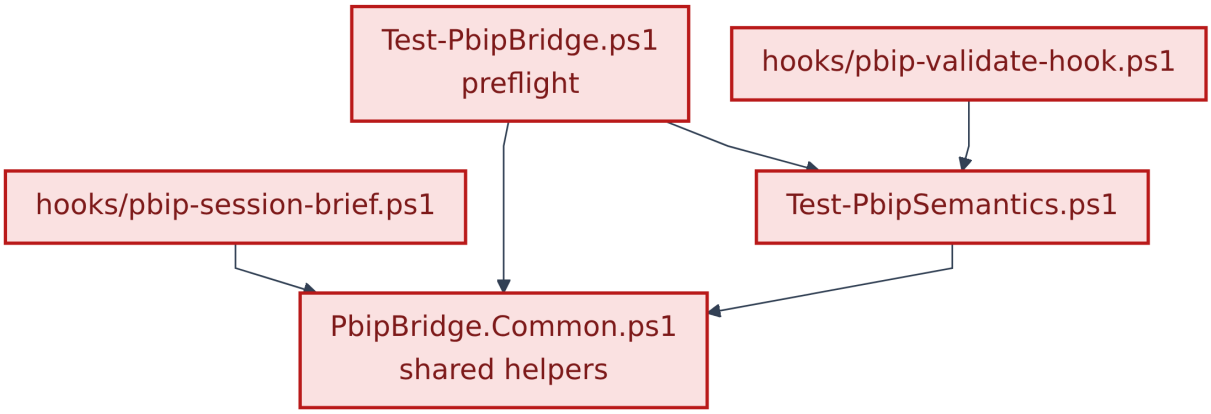


Figure 4.9 — Other scripts that share the helpers.

1. `Test-PbipBridge.ps1` (preflight) uses the helpers and the semantic checks.
2. `hooks/pbip-validate-hook.ps1` uses `Test-PbipSemantics.ps1` on one file at a time.
3. `hooks/pbip-session-brief.ps1` uses the helpers to read the bridge status.

Five scripts stand alone: `validate-json.ps1`, `validate-pbip.ps1`, `run-ai-md-test.ps1`, `hooks/pbip-write-guard.ps1`, and `Invoke-PbipRefresh.ps1`, which clicks Power BI's Refresh button using Windows UI Automation.

5. Rules for the AI: steering, specs and hooks

`docs/AI-CONTEXT.md` explains that the repo keeps three kinds of guidance apart on purpose.

	1. STEERING the rulebook	2. SPECS the plan for one job	3. HOOKS the automatic checks
Answers	**How** do we always build?	**What** are we building, this once?	**When** does a check run?
Lives in	.github/copilot-instructions.md, copilot-instructions.md, AGENTS.md, .github/instructions/, .github/skills/	specs/ (one file per dashboard or page)	scripts/hooks/ + scripts/Invoke-PbipVerify.ps1
Lifetime	Permanent, versioned, maintained	Disposable: written before the build, then left alone	Permanent; code, not prose
Loads	Every session, automatically	On demand, for one task	Run manually in this template
Authority	Highest	Local to its task only	Deterministic: no judgement
How it fails	Grows too long and stops being read	Gets maintained and becomes a second, conflicting rulebook	Encodes judgement and blocks work it should not
Real-world comparison	The building code	The architect's drawing for one house	The smoke alarm

Source: docs/AI-CONTEXT.md (vocabulary borrowed from Kiro). Skills are procedures inside the steering layer.

Figure 5.1 — Steering, specs and hooks.

1. **Steering** is the permanent rulebook: *how* we always build. It loads every session and has the highest authority.
2. **Specs** are plans for one job: *what* we build this time. Written before the build, then left alone.
3. **Hooks** are small scripts that run at fixed moments and make no judgement calls.

Each fails in its own way: a rulebook that keeps growing stops being read; a spec that keeps being updated becomes a second, conflicting rulebook; a hook that tries to judge blocks work it shouldn't. Keeping them apart avoids all three.

Who wins when rules disagree

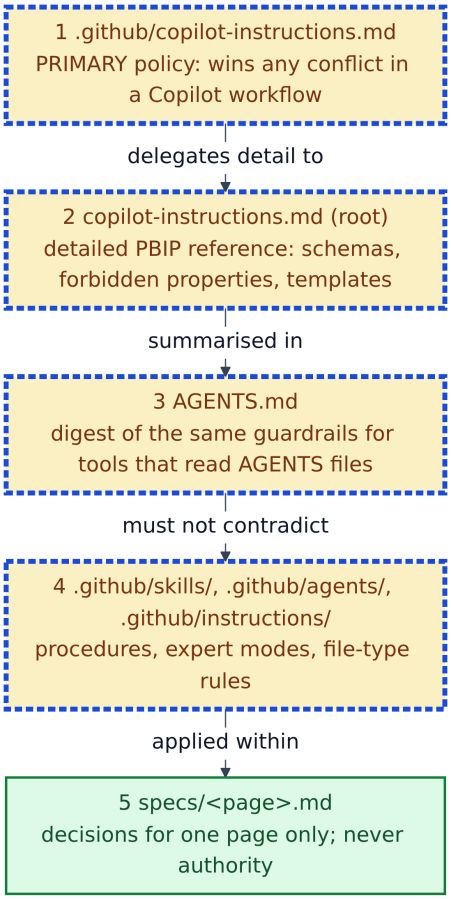


Figure 5.2 — The precedence ladder. The higher box wins.

1. `.github/copilot-instructions.md` is the primary policy. The README: “If there is any conflict … follow `.github/copilot-instructions.md`.”

2. `copilot-instructions.md` (in the root) is the detailed technical reference: file schemas, forbidden properties, JSON templates.

3. `AGENTS.md` repeats the guardrails for tools that read AGENTS files.

4. Skills, expert modes and scoped instruction files must not contradict the files above.

5. A spec covers one page and is never authority.

Watch out: the three files in `.github/agents/` and two skills call the *root* file “the final authority”. Chapter 14 lists that as gap G5.

The guardrails

These “non-negotiable” rules appear in the README, `AGENTS.md` and `.github/copilot-instructions.md`:

1. New business measures go in `_Measures`; keep existing measure homes unless migration is approved.
2. SVG picture-card measures go in `_SVG_Measures`, named by page: `P1_...`, `P2_...`.
3. SVG backgrounds and custom themes live in `StaticResources/RegisteredResources/` and must be registered in `report.json`.
4. Follow the schema rules in `copilot-instructions.md` exactly, including forbidden properties.
5. Save `.tmdl` as UTF-8 without a BOM; no `//` comments in TMDL.
6. Format numbers sensibly and consistently, with thousand separators.
7. SVG cards use the image-visual pattern.
8. Make fake CSV data with Python (numpy and pandas).
9. Never call a page done on valid JSON alone; check the screenshot.
10. Give every measure a description.

`.github/copilot-instructions.md` adds two: keep “SVG-header” page style as the default, and treat `docs/theme/Theme.json` as source only.

Where should a new rule go?

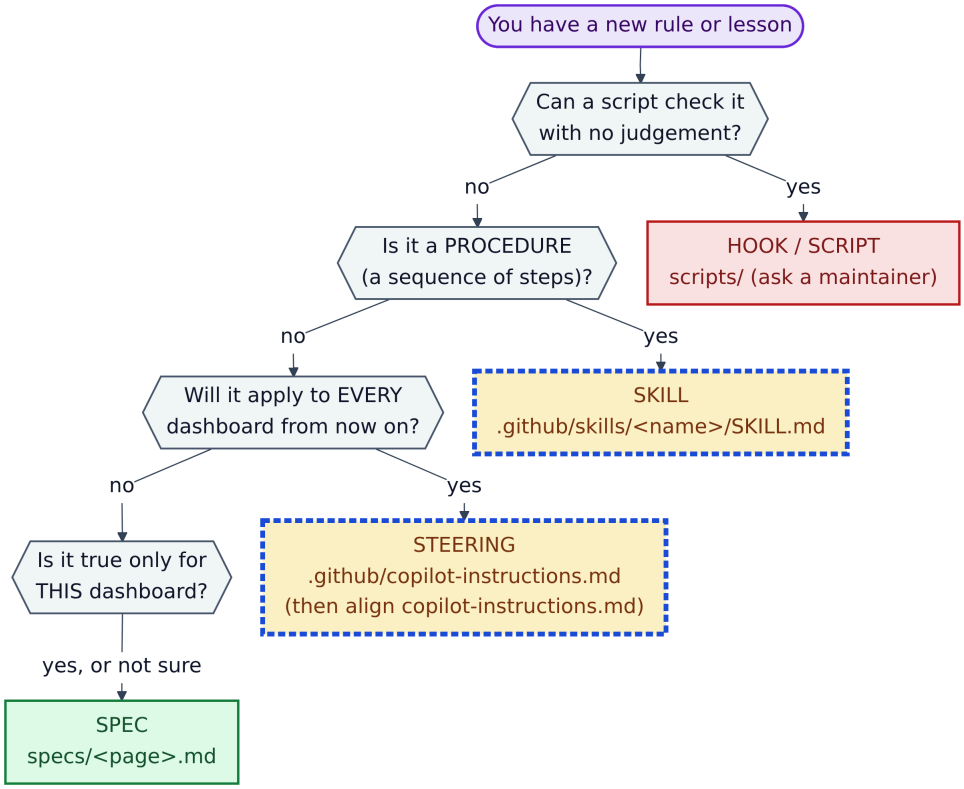


Figure 5.3 — Choosing the layer, following the questions at the end of `docs/AI-CONTEXT.md`.

1. Can a script check it with no judgement? Make it a hook or script: “If a machine can check it, do not spend context asking an agent to remember it.” Ask a maintainer.
2. Is it a procedure? Make it a skill in `.github/skills/<name>/SKILL.md`.
3. Will it apply to every dashboard? Add it to `.github/copilot-instructions.md`, then align the root file.
4. Only for this dashboard, or unsure? Put it in the spec.

The life of a spec

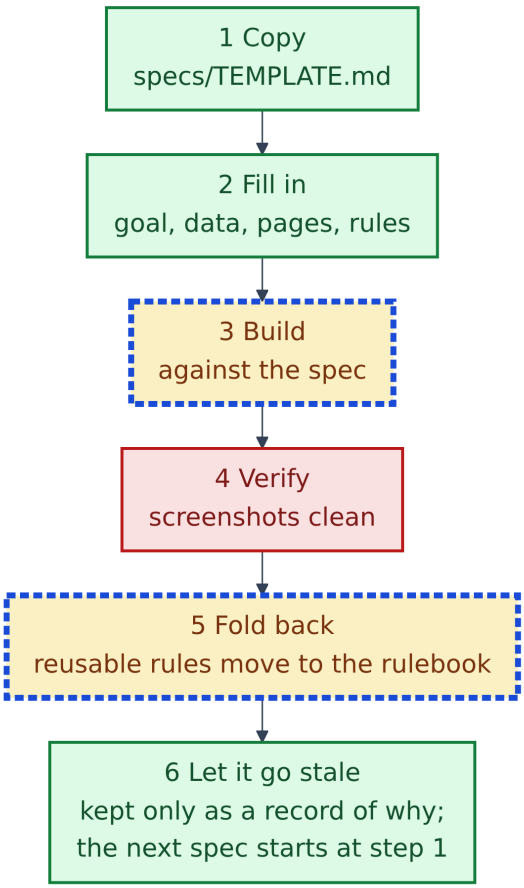


Figure 5.4 — The spec lifecycle (`specs/README.md`).

1. Copy `specs/TEMPLATE.md` to `specs/<name>.md`.

2. Fill in its eight sections: goal, data, pages, rules, build tasks, definition of done, open questions, fold-back.

3. Build against it.

4. Verify every page.

5. Fold back reusable rules into the rulebook. The example spec folded back two: keep visuals 30 px below their card title, and a “Top 10” label must really be filtered to 10.

6. Leave the spec as a record. The next page gets a new one.

You need a spec for a new dashboard or page, a new fact table, or a rebuild that changes a page’s data. You don’t for a colour tweak or label fix.

The four hooks

The README says the hooks are run manually in this template. All of them **fail open**: if something unexpected happens, they let you carry on rather than lock you out.

Hook script	When	What it does
`pbip-session-brief.ps1`	Session start	Prints the guardrails and bridge status
`pbip-write-guard.ps1`	Before a write	Blocks `Proposals/`, `**/.pbi/`, `tests/screenshots/`, `.github/.pbip-state/`
`pbip-validate-hook.ps1`	After a write	Checks that one file: JSON parses, filters are well formed, TMDL has no BOM, `//` or name clash

Hook script	When	What it does
`pbip-stop-hook.ps1`	Before finishing	Runs the verify loop if anything changed and makes the agent review the screenshots

`docs/AI-CONTEXT.md` is honest about the limit: the write guard only sees the agent's file-writing tools, so shell commands can bypass it. "It is a guardrail, not a sandbox."

6. Set up your computer

You need Windows: Microsoft lists only Windows ("Windows 10 or Windows Server 2016 or later") for Power BI Desktop, and the scripts target Windows PowerShell 5.1, which comes with Windows.

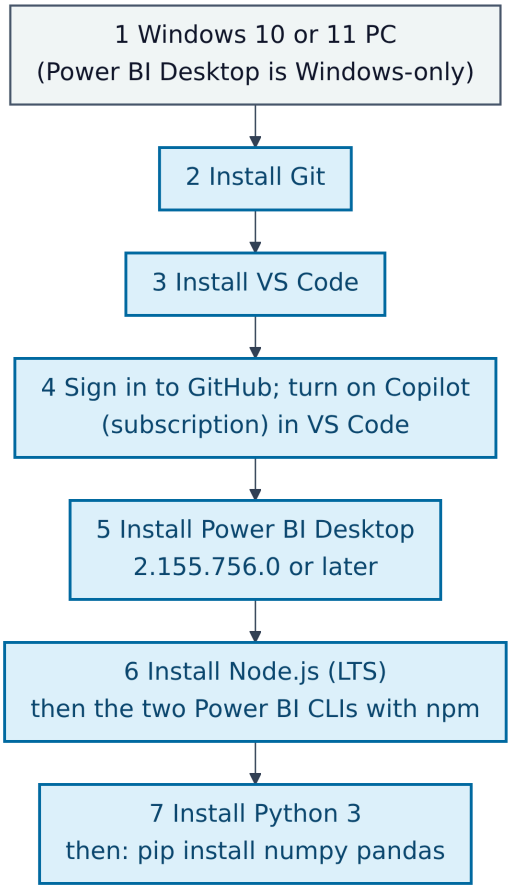


Figure 6.1 — Install the tools, once per machine.

1. A Windows 10 or 11 PC.

2. Git.

3. VS Code (Git must be installed separately; VS Code uses it).

4. Sign in to GitHub in VS Code and turn on Copilot (needs a Copilot subscription).

5. Power BI Desktop **2.155.756.0 or later**. The README says older builds have no Desktop Bridge.

6. Node.js (LTS), then the two Power BI CLIs with npm.

7. Python 3, then `pip install numpy pandas` (guardrail 8; no virtual environment needed).

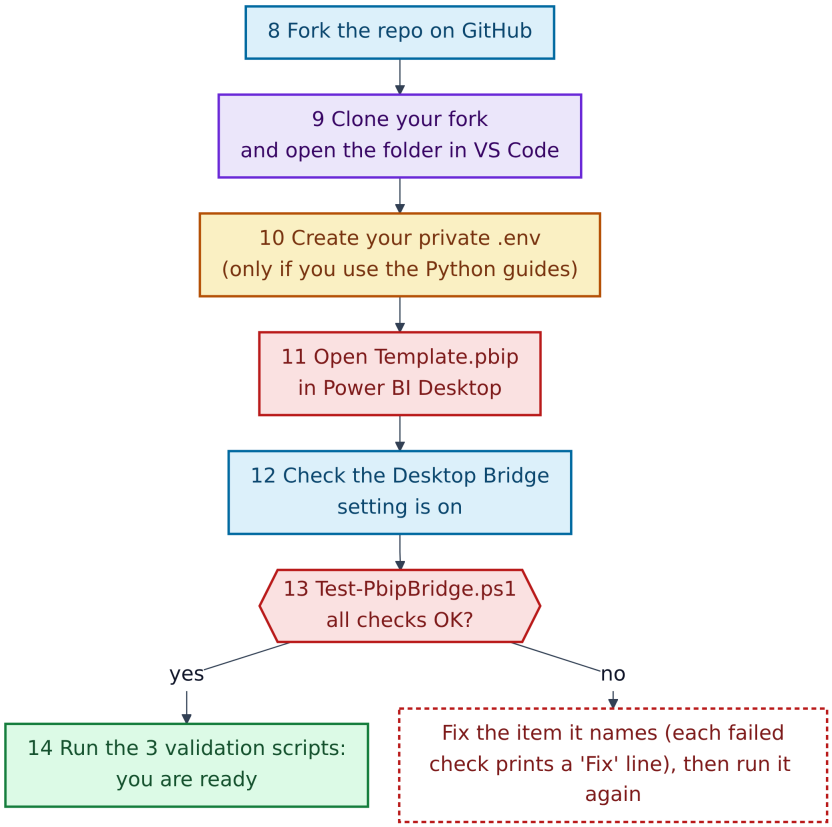


Figure 6.2 — Get the repo and prove the setup works.

8. Fork `GibbonGambits/PGE-PBIP-Template` on GitHub.
9. Clone your fork and open the folder in VS Code.
10. Create your private `.env` file, only if you'll use the Python guides (Figure 6.4).
11. Open `Template.pbip` in Power BI Desktop.
12. Check the Desktop Bridge setting is on (see the note below).
13. Run the preflight `scripts\Test-PbipBridge.ps1`. Each failed check prints a "Fix" line; do it and rerun.
14. Run the three validators. You're ready.

> **Where's the Desktop Bridge setting?** Microsoft's page "What is the Power BI Desktop Bridge?" says *File > Options and settings > Options > **Security** > Desktop Bridge*, "Enable external tool access to Power BI Desktop through secure local APIs", and that "this setting is on by default". The repo's README and `copilot-instructions.md` say **Preview features**. Look under Security first. (Gap G12 in Chapter 14.)

Which tool you need for what

Task	Git	VS Code	Copilot	Power BI Desktop	Node.js + 2 CLIs	Windows PowerShell	Python 3	pbiviz tools
Save and share work (Git, GitHub)	✓	✓						
Ask the AI agent to build a page		✓	✓					
Open and look at the report				✓				
Run validate-json / validate-pbip						✓		
Run Test-PbipSemantics						✓		
Run the verify loop (screenshots)				✓	✓	✓		
Generate fake CSV data (guardrail 8)							✓	
Python email / alert guides							✓	
Build a custom visual (Chapter 11)				✓	✓	✓	✓	✓
AI test on GitHub (Chapter 10)								

A tick means the task needs that tool on your PC. The AI test on GitHub runs on GitHub's own computers, so it needs nothing locally, only a repository secret (Chapter 10). The custom-visual kit needs Node.js 18 or later.

Figure 6.3 — Tools by task. A tick means you need that tool installed.

1. Git and VS Code are enough to save and share work.
2. Copilot is needed to have the agent build pages.
3. The three validators need only Windows PowerShell.
4. The verify loop needs Power BI Desktop, Node.js with the two CLIs, and PowerShell.
5. Python is for fake data and the Python guides.
6. Custom visuals need everything plus the `pbiviz` tools (Node.js 18 or later).
7. The AI test on GitHub runs on GitHub's computers; you only add a secret.

Commands

Run these in a PowerShell terminal in the repo folder (VS Code: *Terminal > New Terminal*).

```
git --version
node --version
npm install -g @microsoft/powerbi-desktop-bridge-cli@latest
npm install -g @microsoft/powerbi-report-authoring-cli@latest
powerbi-desktop --version
powerbi-report-author --version
pip install numpy pandas
```

For the Python guides, also `pip install python-dotenv pyodbc`. For custom visuals, `npm i -g powerbi-visuals-tools`.

Copilot CLI and the Fabric skills (optional). The README asks for a one-time step in GitHub Copilot CLI. GitHub's install page offers `winget install GitHub.Copilot` (it needs PowerShell 6 or later on Windows, or use npm with Node.js 22+). Then, inside Copilot CLI:

```
/plugin marketplace add microsoft/skills-for-fabric
/plugin install powerbi-authoring@fabric-collection
/skills list
```

Prove it works:

```
powershell -NoProfile -File scripts\Test-PbipBridge.ps1
powershell -NoProfile -File scripts\validate-json.ps1
powershell -NoProfile -File scripts\validate-pbip.ps1
powershell -NoProfile -File scripts\Test-PbipSemantics.ps1
```

The preflight checks eight things: the project is found, Power BI is new enough, both CLIs are on your PATH, the bridge pipe exists, a running Power BI has *this* project open, the semantic checks are clean, and the report files validate.

Create your private `.env` safely

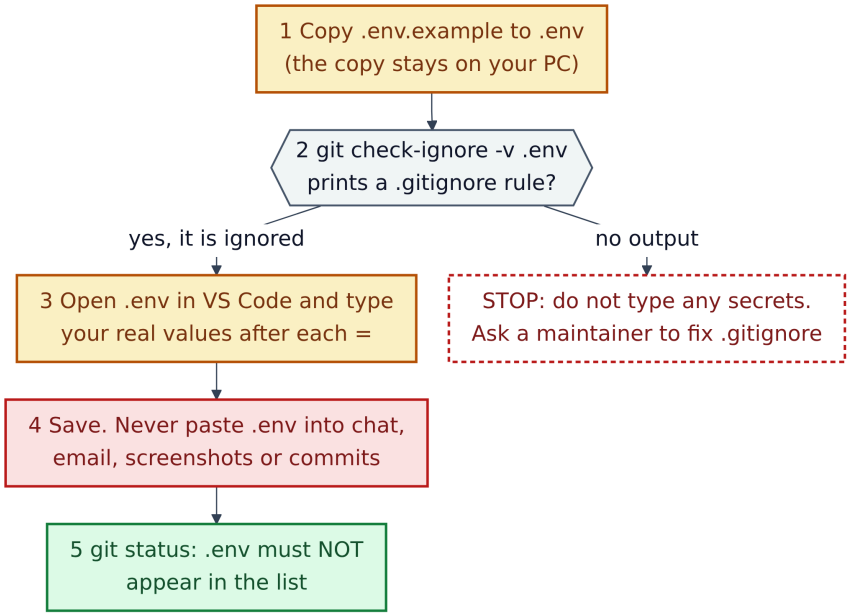


Figure 6.4 — Creating `.env` without leaking secrets. A `.env` file holds `NAME=value` settings such as passwords. It must never leave your machine.

1. Copy the example: `Copy-Item .env.example .env`. The example has only names (`DB_SERVER`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_DRIVER`, `SMTP_EMAIL`, `SMTP_PASSWORD`).
2. Prove Git ignores it: `git check-ignore -v .env` should print a rule such as `.gitignore:4:.env`. If it prints nothing, stop.
3. Type your real values after each `=`. Never put real values in `.env.example`; that file is in Git.
4. Never paste `.env` into a chat, email, screenshot or pull request.
5. Check `git status`: `.env` must not appear.

The Python guides load it with `python-dotenv` (`load_dotenv()`), then read `os.environ['DB_SERVER']` and so on. GitHub automation never sees your `.env`; it uses repository **secrets** instead (Chapter 10).

7. Task: build a dashboard page

This follows the “Dashboard Creation Steps” in `copilot-instructions.md`. Normally the agent does the steps; you write the spec and review.

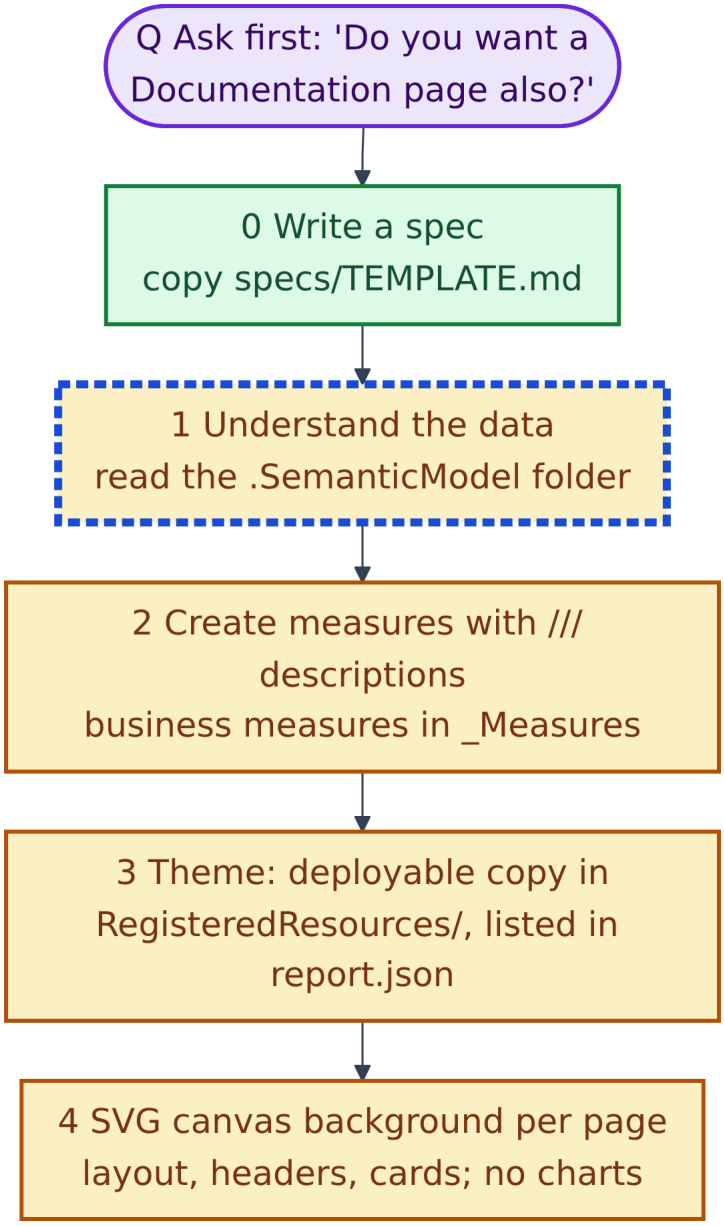


Figure 7.1 — From question to background.

- **Q. Ask first.** On a first request the agent must ask: “Do you want a Documentation page also?” If yes, it adds a `Documentation` page whose background summarises the project.

0. Write a spec. Decisions “are cheap to change in a spec and expensive to change in generated PBIR JSON.”

1. Understand the data: read the model’s tables, columns, relationships and measures.

2. Create measures with a `///` description. Business measures go in `_Measures`.

3. Register the theme: the deployable theme file goes in `RegisteredResources/` and is listed in `report.json` (Chapter 9).

4. Create an SVG background per page. SVG is a text-based image format. The background draws the layout, card boxes and card titles, but no charts.

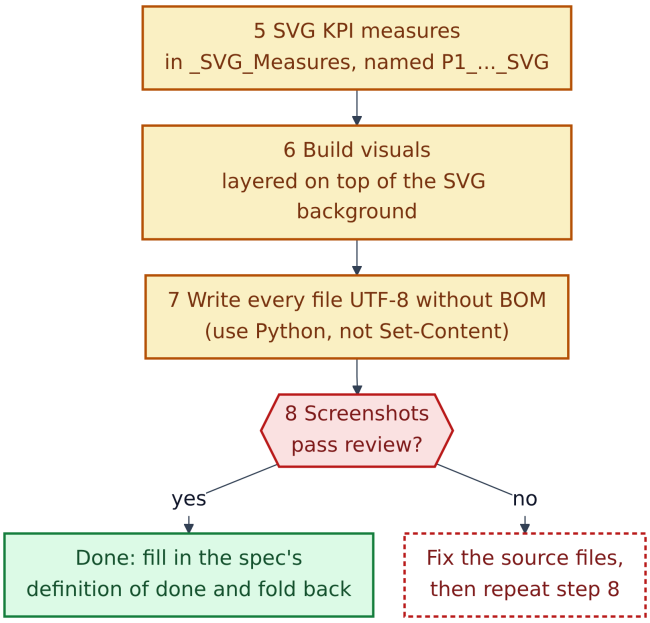


Figure 7.2 — From measures to proof.

5. **SVG KPI measures** go in `_SVG_Measures`, named by page: `P1_Sales_KPI_SVG`. (KPI: a key performance indicator, one headline number.)
6. **Build the visuals** on top of the background.
7. **Write every file as UTF-8 without a BOM.** The rulebook says use Python, `open(f, 'w', encoding='utf-8')`.
8. **Screenshots pass review?** Then tick the spec's definition of done. If not, fix the source and repeat step 8.

The most common layout bug



Rule 16 in copilot-instructions.md: the title is drawn at card_y + 24, so a visual must start at card_y + 30 or lower. Example: card at y=192 means visual at y=222 or later.

Figure 7.3 — Keep visuals below the card title.

1. The SVG background draws each card's title at `card_y + 24` pixels.
2. A Power BI visual sits on top. If its top edge is too high, it covers the title.
3. Rule 16 in `copilot-instructions.md`: put the visual at `card_y + 30` or lower. Card at `y=192` → visual at `y=222` or later.

`specs/TEMPLATE.md` calls this “the single most common rendering defect in this template”, and no JSON check can see it. Because titles come from the background, visuals have no title, background, border or shadow of their own. That default is called **SVG-header mode**; giving visuals their own titles (“visual-title mode”) needs an exception in the spec.

Where does a new measure go?

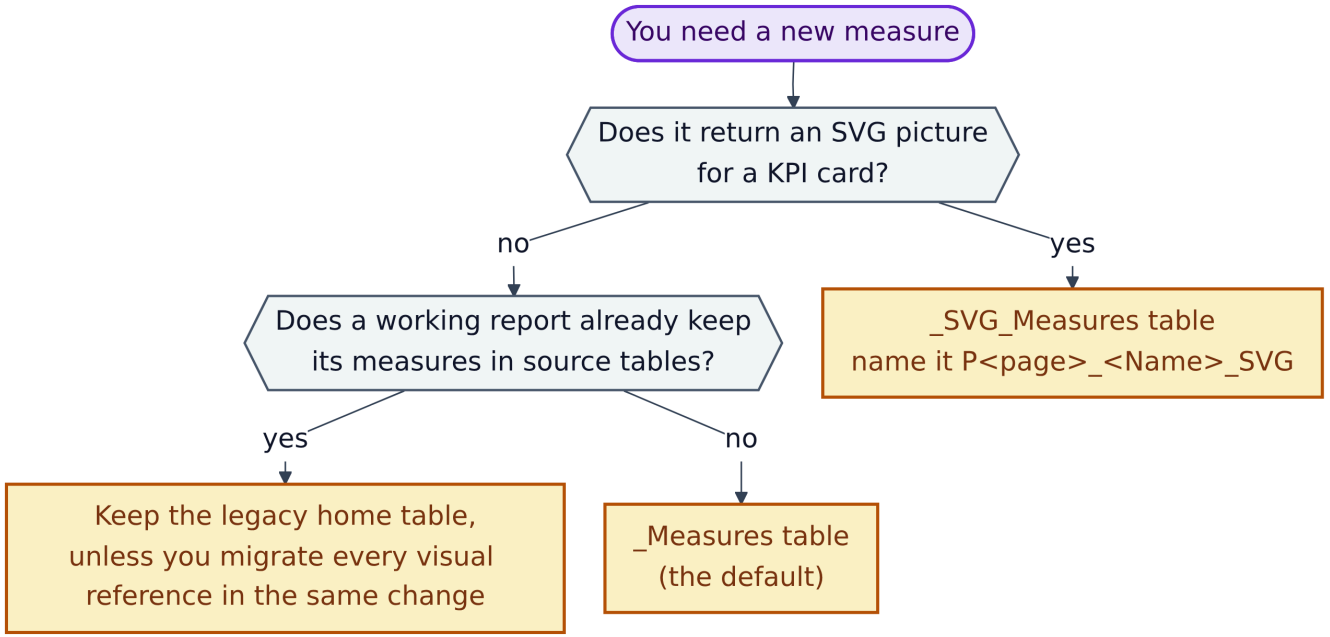


Figure 7.4 — Choosing a measure’s table.

1. Returns an SVG picture for a KPI card? `_SVG_Measures`, named `P<page>_<Name>_SVG`.
2. An existing report already keeps measures in source tables (e.g. `Fact-MaterialMaster`)? Leave them there (“legacy-safe mode”) unless you move every visual reference in the same change.
3. Otherwise: `_Measures`. If it doesn’t exist yet, create it as a one-row placeholder table. The rulebook writes `_Measures = BLANK{}`; the correct DAX, which it uses for `_SVG_Measures`, is `{ BLANK() }` (gap G7).

The Python build-script order

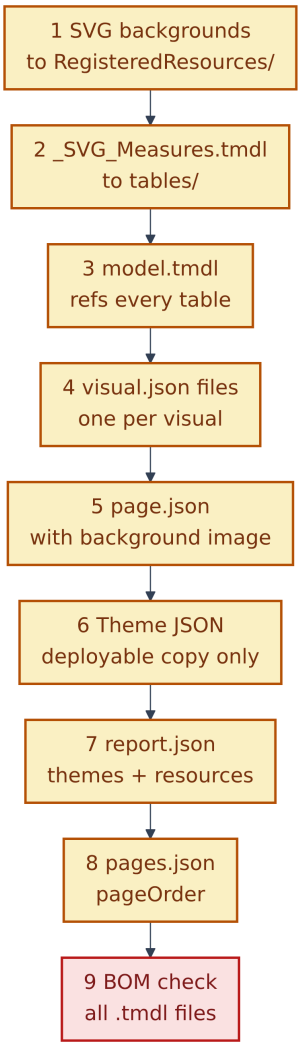


Figure 7.5 — Nine steps for one build script. `copilot-instructions.md` recommends a single Python script that writes everything.

1. SVG backgrounds into `RegisteredResources/`.
2. `_SVG_Measures.tmdl` into `tables/`.
3. `model.tmdl`, referencing every table.
4. Each `visual.json`.
5. Each `page.json`, with its background.
6. The deployable theme JSON.
7. `report.json`: themes and resources.
8. `pages.json`: page order.
9. A final check that no `.tmdl` file has a BOM.

House style for visuals

`copilot-instructions.md` lists 16 formatting rules. The short version:

Rule	Meaning
Number formats	Whole numbers by default; decimals, currency or percent when justified; thousand separators
Names	No “Sum of”, no underscores in headers or tooltips
Data labels	On for every chart; full numbers, not K/M, unless the spec says so
Sort	Descending by value; months Jan→Dec by a month-number column
Chrome	No visual background, border, shadow or title

Rule	Meaning
Axes	No axis titles; value axis hidden; category axis shown
Top N	If the SVG label says “Top 10”, filter the visual to 10

The same file has JSON templates for bar, line, donut and table visuals, and a list of **forbidden properties** that make Power BI reject a file, e.g. `"filters": []` at the top of a visual, `"selector": null`, or `sort` inside `visual` (use `query.sortDefinition`).

SVG KPI cards are image visuals with `sourceType = 'imageData'` and the measure in `sourceField`. The measure returns text starting `data:image/svg+xml;utf8,<svg...` (never base64), uses `rgb(...)` or `#HEX` colours, Segoe UI, `INT()` for positions, and no headings: the heading is on the background card.

Definition of done

From `specs/README.md`, a page is done only when:

- ☐ The three validators pass.
- ☐ The project opens in Power BI Desktop with no error dialog (the window title shows the project, not *Untitled*).
- ☐ `powerbi-desktop status` reports the project path.
- ☐ Every page screenshot has been reviewed with the `powerbi-visual-verify` skill.
- ☐ No visual covers its card title; no cut-off labels; no blank charts.
- ☐ Findings are reported by page display name.

If you said yes to a Documentation page, it follows the pattern in `copilot-instructions.md` (816 × 2740 pixels, actual size, background `#FBF7F2`); the prompt `tests/prompts/PBIP_Documentation_Page.md` builds it.

8. Task: check your work

Checking happens in two stages: fast text checks that never open Power BI, then the verify loop that screenshots what Power BI actually drew.

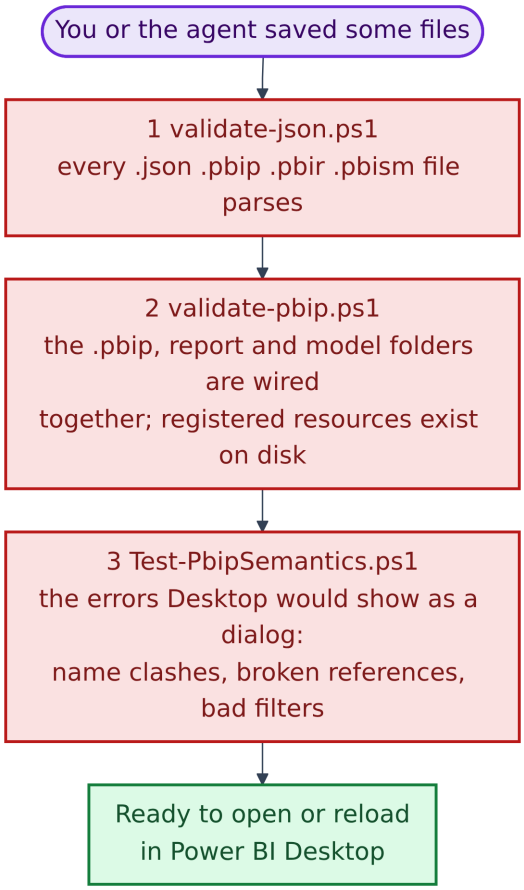


Figure 8.1 — The three validators, cheapest first. Each runs in seconds with only Windows PowerShell.

1. **`validate-json.ps1`** checks every `.json`, `.pbip`, `.pbir` and `.pbism` file parses. It skips `node_modules`, `dist`, `.tmp`, `.vscode`, `.git`, `package-lock.json`, and `tsconfig.json` (which may contain comments).
2. **`validate-pbip.ps1`** follows the pointer chain from Chapter 1: required files exist, every resource in `report.json` exists on disk, and every custom visual a page uses is registered. Add `-Project <name>` if there's more than one project.
3. **`Test-PbipSemantics.ps1`** finds, in the text, the errors Power BI would show as a pop-up: a measure and column with the same name, duplicate names, a misspelled filter key (it suggests the fix, e.g. `Inn` → `In`), and, once the model is complete, relationships to missing columns, unresolved `'Table'[Column]` references, unregistered resources and duplicate page ids. Add `-Path <file>` to check one file.

```
powershell -NoProfile -File scripts\validate-json.ps1
powershell -NoProfile -File scripts\validate-pbip.ps1
powershell -NoProfile -File scripts\Test-PbipSemantics.ps1
```

`-NoProfile` starts PowerShell without personal start-up settings, so scripts behave the same everywhere.

The verify loop

The README sums it up: "edit PBIR/TMDL → validate → reload → screenshot every page → review → fix → repeat". The engine is `scripts\Invoke-PbipVerify.ps1`.

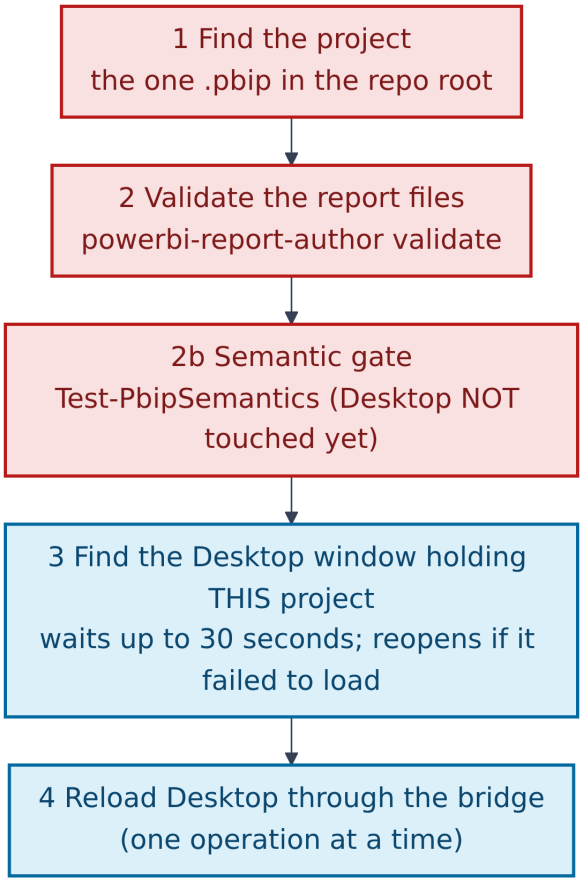


Figure 8.2 — Before anything is drawn.

1. Find the project: the one `.pbip` in the repo root. Scripts never assume the name `Template`.

2. Validate the report files with `powerbi-report-author validate`.

2b. Run the semantic checks before Power BI is touched.

3. Find the Power BI window that has *this* project open. Wait up to 30 seconds; if loading failed, capture the error, close that window, and reopen once. It only ever closes a window it started or one stuck on an error.

4. Reload through the bridge. Bridge calls run one at a time; overlapping calls fail with “Cancelled”.

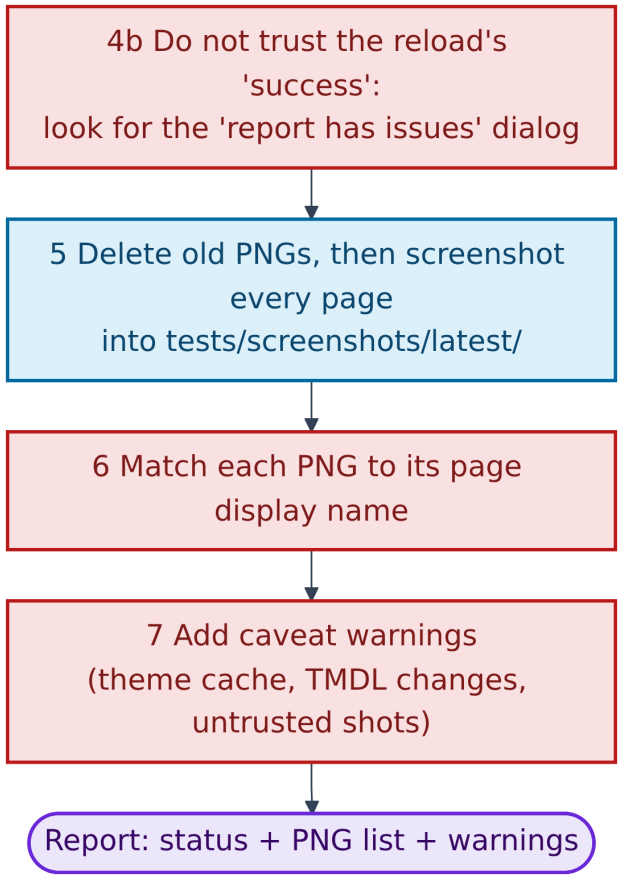


Figure 8.3 — Capture and report.

• **4b.** Don’t trust the reload’s “success”: `powerbi-desktop reload` reports success even when the “Your report has issues” dialog appears, so the script looks for it.

5. Delete old PNGs, then screenshot every page into `tests/screenshots/latest/` (ignored by Git).

6. Match each PNG to its page display name.

7. Add warnings, e.g. stale theme colours or untrusted screenshots.

• **Result:** a status, the PNG list and the warnings.

```
powershell -NoProfile -File scripts\Invoke-PbipVerify.ps1 -Mode Review
```

Options: `-Mode Confirm` (second pass after a fix), `-NoReopen`, `-LoadTimeoutSeconds 30`, `-Scale 2`.

Reading the result

Status	What it means	What you do
loaded	The project loaded and every page was screenshotted.	Open the PNGs and review them against the visual-verify skill.
validation_failed	A report or semantic check failed. Desktop was not touched.	Fix the listed errors in the source files, then run again.
load_failed	Desktop rejected the project and showed an error dialog.	Read the captured dialog text; fix the named file and line.
loaded_with_issues	Desktop loaded only after dropping broken parts. Screenshots are UNTRUSTED.	Fix the reported filter expression; the instance is closed so nothing degraded is saved.
not_running	Power BI Desktop is not running (after an automatic reopen attempt).	Open Template.pbip in Desktop, then run again.
no_bridge	Desktop is running but offers no bridge connection.	Turn on the Desktop Bridge setting (Chapter 6) and restart Desktop.
other_project	Desktop is running, but with a different project open.	Open this project; the message names what is open instead.
bridge_unavailable	The powerbi-report-author CLI is not installed, so the report files cannot be validated.	Install both npm CLIs (Chapter 6).
error	The project could not be found.	Run from inside the repository folder.

Green = success. Amber = skipped: the loop fails open and lets work continue, but nothing was visually verified. Red = a defect that blocks completion.

Figure 8.4 — Statuses and what to do.

1. **Green, `loaded`:** the only success. Now review the PNGs.
2. **Red:** a defect in your files. `loaded_with_issues` is the sneaky one: the page *looks* fine, but broken parts were silently dropped. The script closes that window so the damaged version can't be saved.
3. **Amber:** the loop **failed open**: it couldn't reach Power BI, so it skipped instead of blocking you. Nothing was verified. AGENTS.md says to then "say the report was not visually verified rather than claiming it renders correctly".

When the loop runs by itself

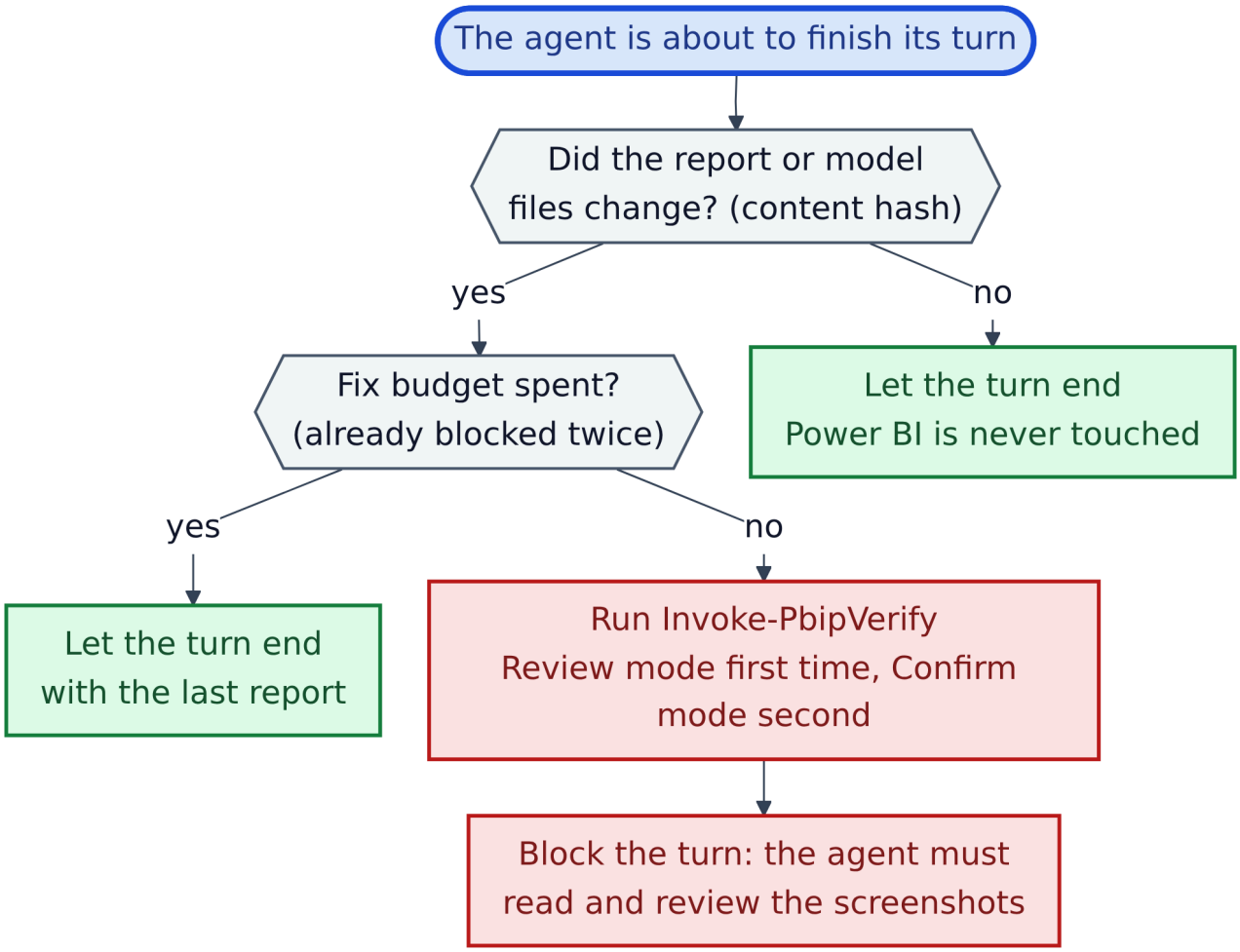


Figure 8.5 — `scripts\hooks\pbip-stop-hook.ps1`, the wrapper that runs when an agent is about to finish.

1. Did anything change? It compares a content hash (a fingerprint of the files). No change means Power BI is never touched.
2. Is the fix budget spent? At most two blocks: “round 1 = review + fix, round 2 = confirmation + report”.
3. Otherwise it runs the loop: Review mode first, Confirm mode second.
4. It blocks the turn so the agent must review the screenshots. Its bookkeeping lives in `.github/.pbip-state/` (ignored by Git; don’t edit it).

When Power BI shows an error window

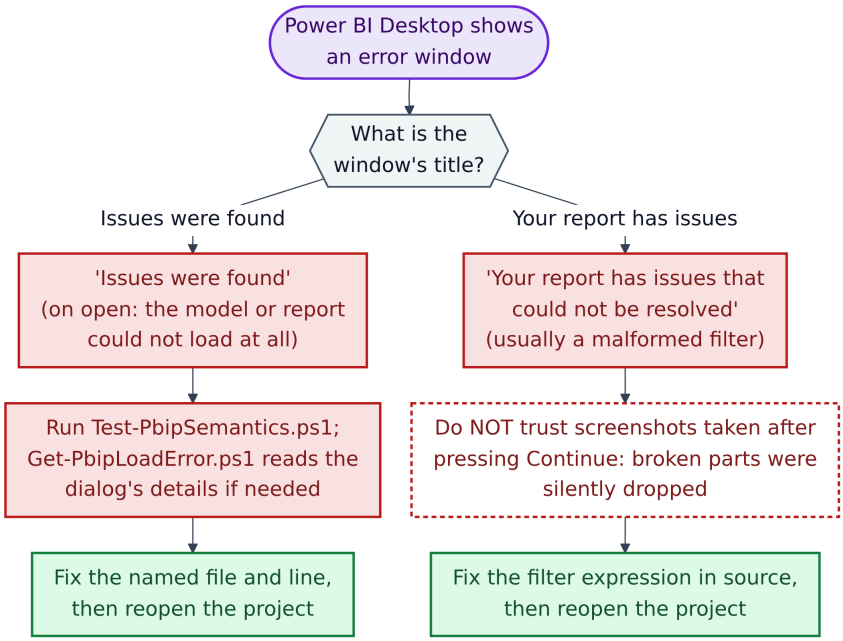


Figure 8.6 — Two error windows, two fixes.

1. **“Issues were found”** appears when the project can’t load; the window often stays *Untitled*. Run `Test-PbipSemantics.ps1`. If that finds nothing, `Get-PbipLoadError.ps1` reads the dialog through Windows UI Automation, including the “Copy details to clipboard” text that names the file and line.
2. **“Your report has issues that could not be resolved”** usually means one malformed filter. Its list of about 49 issues usually comes from a single bad key. Don’t press *Continue* and trust the result: it silently drops the broken parts.
3. Either way, fix the source file, then reopen. Usual causes: a measure missing from TMDL, a resource not registered in `report.json`, a property the schema version doesn’t allow, or a BOM.

Gotchas that make screenshots lie

Gotcha	What you see	What to do
Theme cache	Old colours after a theme edit	Rename the theme with a new suffix and update `report.json`, or reopen Power BI
Model changes	Stale or missing measure values	Reopen: `powerbi-desktop open "<project>.pbip"`
First open	Power BI refuses to load, stays on *Untitled*	Fix the cause; never use *continue with errors*
Wrong window	An idle *Untitled* window fails with `REPORT_DIR_REQUIRED`	Open the project there; scripts match by file path

Gotcha	What you see	What to do
Empty visuals on a new project	“Some of the tables have incomplete or no data”	Run `scripts\Invoke-PbipRefresh.ps1`, then save in Power BI

Reviewing the PNGs

Use `.github/skills/powerbi-visual-verify/SKILL.md`: score the relevant checklist rows Pass / Needs Work / N/A, cite the visible evidence, fix the **source files, never the image**, and report by **page display name**.

9. Task: change the theme

A Power BI **theme** is a JSON file of colours and fonts. Microsoft’s docs: report colours “come from data colors in the theme”. This repo keeps an editable source file and a cleaned, loadable copy.

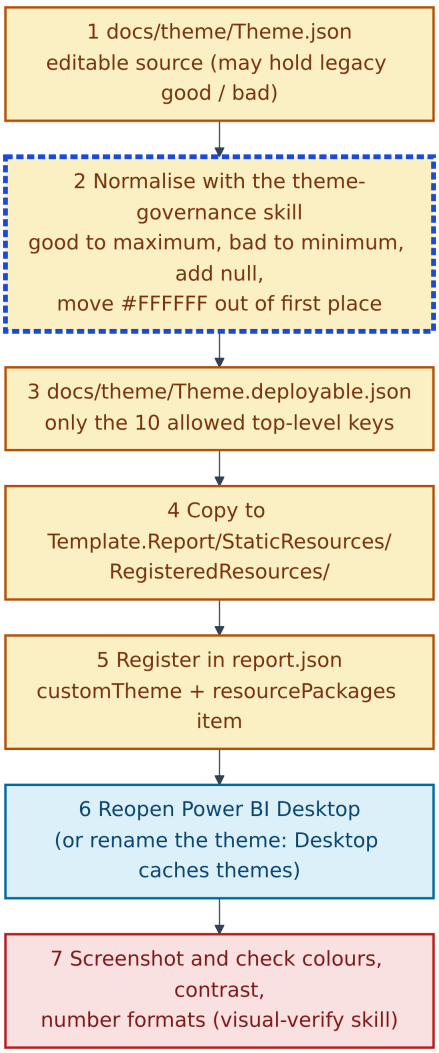


Figure 9.1 — The seven-step theme pipeline.

1. Edit `docs/theme/Theme.json`, the source theme `PGEColors`. It may contain older keys (`good`, `bad`) that Power BI won’t accept.

2. Normalise it with the `powerbi-theme-governance` skill: `good` → `maximum`, `bad` → `minimum`, add `null`, and move white (`#FFFFFF`) out of first place in `dataColors`.

3. The result, `docs/theme/Theme.deployable.json`, may only use ten top-level keys: `name`, `dataColors`, `background`, `foreground`, `tableAccent`, `maximum`, `minimum`, `null`, `textClasses`, `visualStyles`. “Any other key causes import failure.”

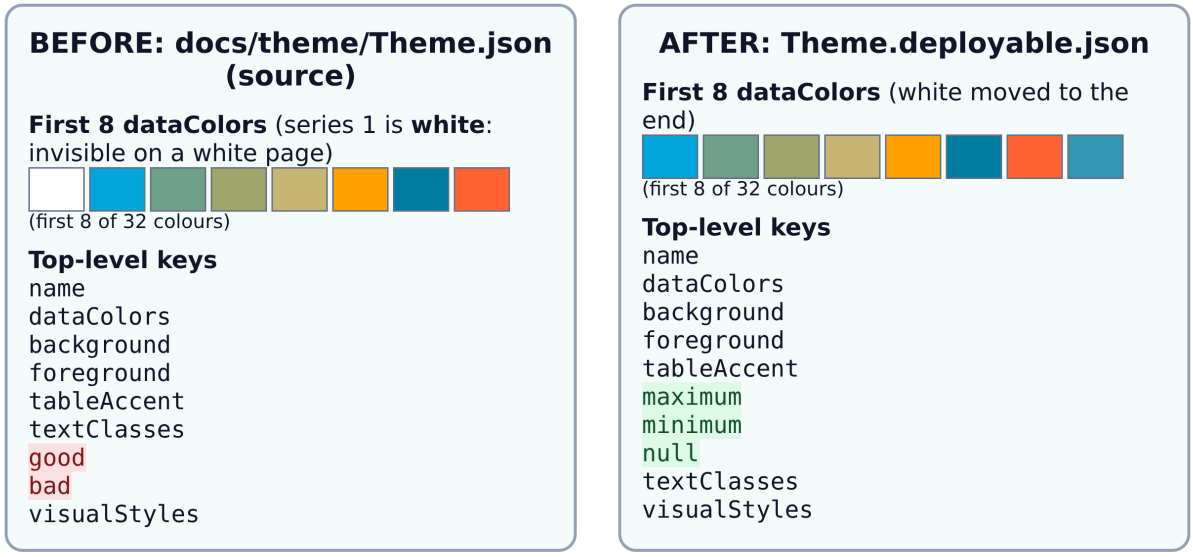
4. Copy it into `Template.Report/StaticResources/RegisteredResources/`. Today the two copies are identical.

5. Register it in `report.json` under `themeCollection > customTheme` (keep `reportVersionAtImport`) and as a `CustomTheme` item in the `RegisteredResources` package.

6. Reopen Power BI: it caches themes, so a plain reload can show old colours.

7. Screenshot and check: theme applied (checklist row 51), no white-on-white or low contrast (rows 67, 71, 75), consistent number formats (row 72).

What promotion changed here



good (#008001) became maximum; bad (#D2222D) became minimum; null (#94A3B8) was added. Red = not allowed in a deployable theme; green = added during promotion.

Figure 9.2 — The real before and after.

1. In the source, the first series colour is white, invisible on a white page. The deployable copy moves it to the end of the 32 colours.
2. `good` (`#008001`) became `maximum`; `bad` (`#D2222D`) became `minimum`; `null` (`#94A3B8`) was added.
3. Everything else is unchanged.

Two themes at once

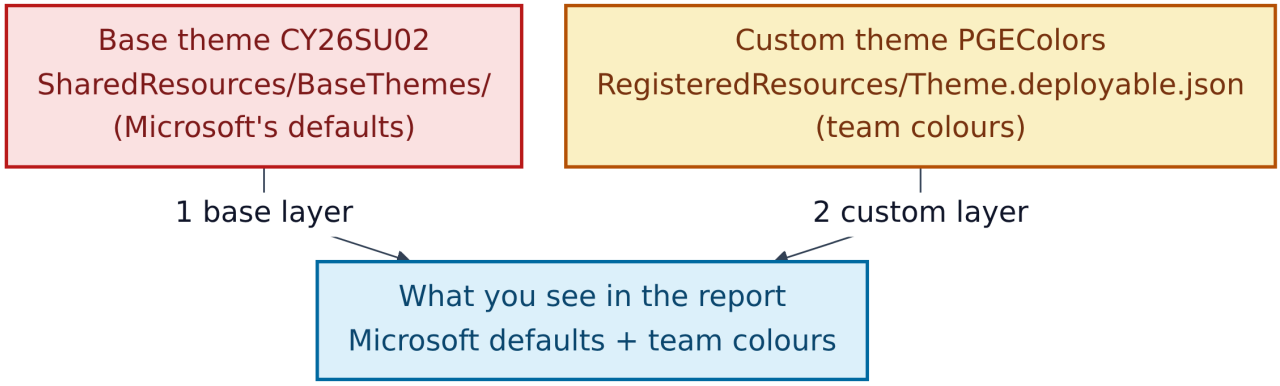


Figure 9.3 — Two layers, both registered in `report.json`.

1. The base theme `CY26SU02` is Microsoft's file in `SharedResources/BaseThemes/`. Red.
2. The custom theme `PGEColors` is the team's file in `RegisteredResources/`. Amber.
3. The report combines Microsoft's defaults with the team's colours and fonts.

Do and don’t

Do	Don’t
Edit colours in `docs/theme/Theme.json`	Register `Theme.json` directly
Re-run the promotion rules after every edit	Leave white as the first data colour
Keep `reportVersionAtImport` (without it: “Required properties are missing”)	Add keys like `visualContainerHeader`, `axisLine` or `gridlines`
Reopen Power BI, or rename the theme, before judging colours	Trust colours after a plain reload

One detail: `docs/theme/Theme.json` starts with a BOM, so Python’s `json.load` rejects it unless you open it with `encoding='utf-8-sig'` (gap G14).

10. Tests and GitHub automation

GitHub Actions runs automation on GitHub’s computers. A **workflow** is a YAML file in `.github/workflows/`; GitHub defines it as “a configurable automated process that will run one or more jobs”. A **runner** is the cloud machine that runs it.

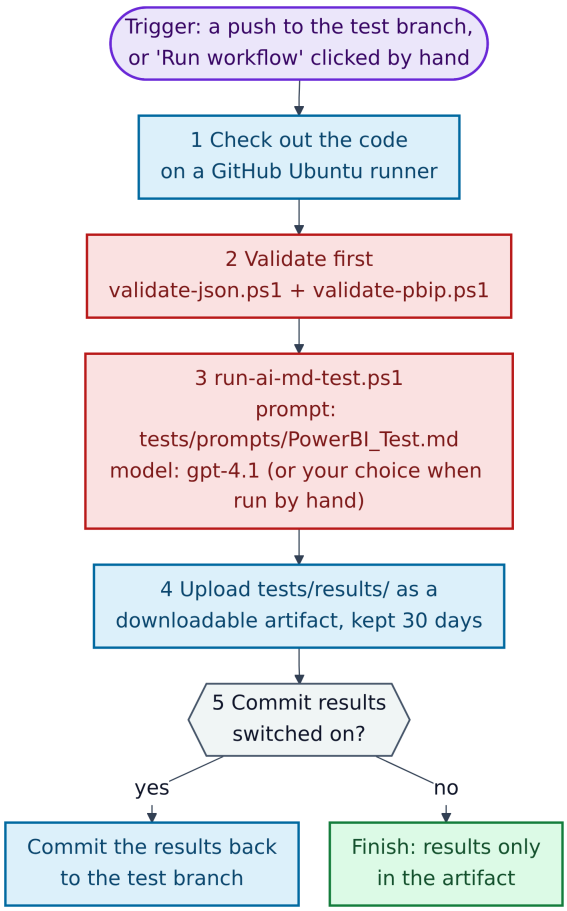


Figure 10.1 — `.github/workflows/ai-testing.yml`.

• **Trigger:** a push to the `test` branch, or *Run workflow* by hand (`workflow_dispatch`). A manual run can pick the model (default `gpt-4.1`) and whether to include Best Practice Analyzer rules.

1. Check out the code on an Ubuntu runner, in the `test` environment.

2. Run `validate-json.ps1` and `validate-pbip.ps1` with PowerShell 7 (`pwsh`). A failure stops the run.

3. Run `scripts/run-ai-md-test.ps1` with the prompt `tests/prompts/PowerBI_Test.md`.

4. Upload `tests/results/` as an artifact (a downloadable file bundle), kept for 30 days.

5. Only if the run is on `test` **and** the variable `AI_TEST_COMMIT_RESULTS` is `true`, commit the results back (with `[skip ci]` so it doesn’t trigger itself).

Inside the test runner

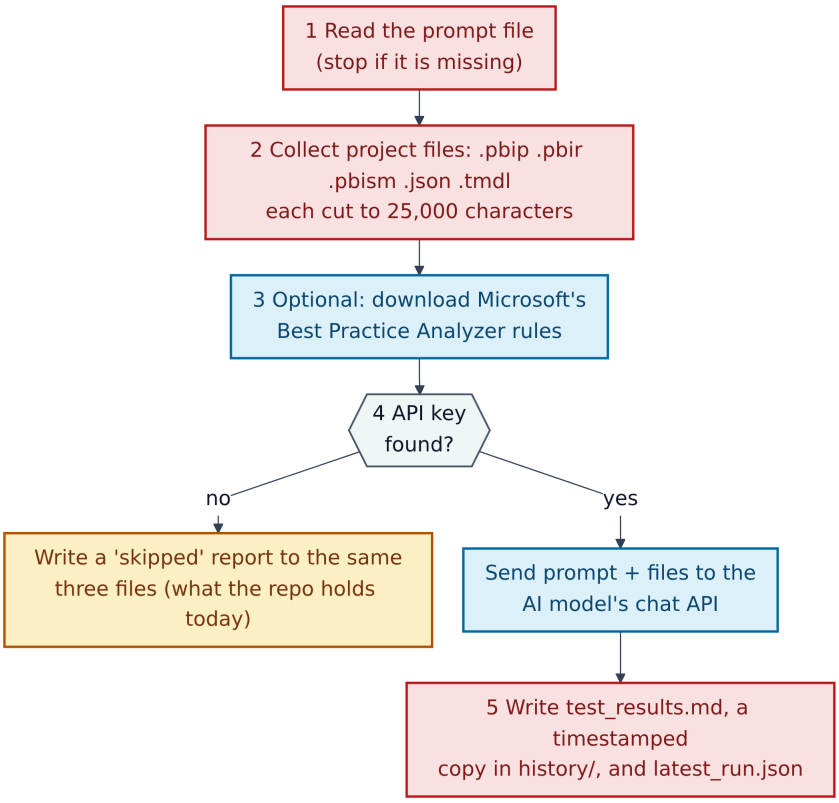


Figure 10.2 — `scripts/run-ai-md-test.ps1`.

1. Read the prompt file; stop if it's missing.
2. Collect the project files (`.pbip`, `.pbir`, `.pbism`, `.json`, `.tmdl`), each cut to 25,000 characters.
3. With `-IncludeBpaRules`, download Microsoft's Best Practice Analyzer rules from the `microsoft/Analysis-Services` repo.
4. Look for an API key in `GITHUB_MODELS_API_KEY`, then `AZURE_OPENAI_API_KEY`. No key: write a "skipped" report. That's what `tests/results/` holds today (two runs on 17 April 2026). With a key: send everything to the chat API, by default `https://models.inference.ai.azure.com/chat/completions`.
5. Write `test_results.md`, a timestamped copy in `history/`, and `latest_run.json`.

```
./scripts/run-ai-md-test.ps1 -PromptFile tests/prompts/PowerBI_Test.md -OutputDir
tests/results -OutputFileName test_results.md -IncludeBpaRules
```

One automated run, step by step

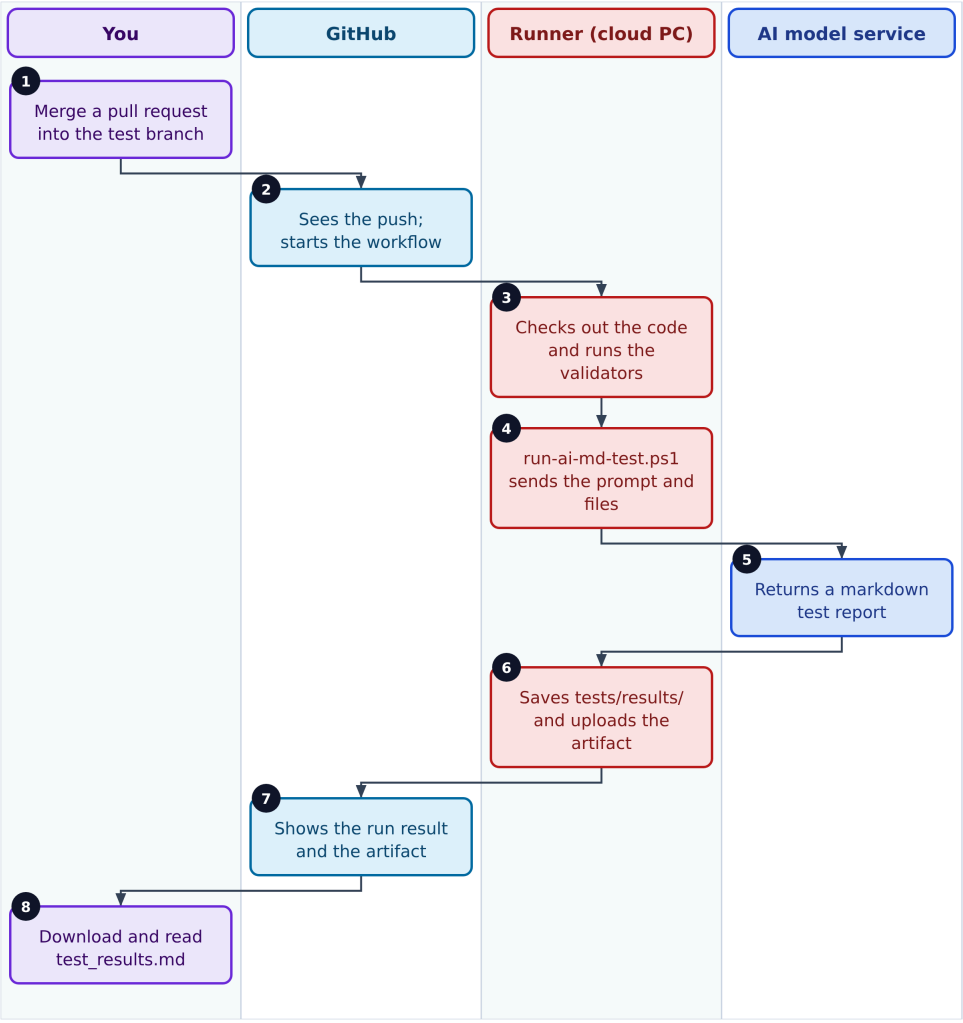


Figure 10.3 — Who does what.

1. You merge a pull request into `test` (a merge is a push).
2. GitHub starts the workflow.
3. The runner checks out the code and runs the validators.
4. The script sends the prompt and files to the AI model.
5. The model returns a markdown report.
6. The runner saves and uploads `tests/results/`.
7. GitHub shows the result and the artifact.
8. You download and read `test_results.md`.

What the AI test checks

`tests/prompts/PowerBI_Test.md` is a “Measure Killer-style” audit: it hunts for unused and broken parts of the model by reading files only (running scripts is forbidden). In eight phases it inventories the model and report, finds references to missing measures or columns and things nothing uses, writes a report with severity markers, adds a Best Practice Analyzer check, and gives one verdict.

`tests/prompts/Python_Data_Value_Test.md` is different: it doesn’t write code. It asks Copilot for a blueprint of your own Python checker, e.g. “sales of branch `ABV` in March 2026”, with options `--branch`, `--month`, `--year`, `--dry-run`.

Settings to add on GitHub

Setting	Kind	Where	Why
`GITHUB_MODELS_API_KEY` or `AZURE_OPENAI_API_KEY`	Secret (at least one)	*Settings > Secrets and variables > Actions*	Lets the test call a model
`AI_TEST_COMMIT_RESULTS` = `true`	Variable	Same page, *Variables*	Commits results back to `test`
`test`	Environment	*Settings > Environments*	The job runs in it

GitHub doesn't pass secrets to workflows triggered from forks, and an unset secret reads as empty, which is why a fork produces a "skipped" report instead of an error.

`docs/CICD-SETUP.md` also describes Fabric deployment secrets (`FABRIC_TENANT_ID`, `FABRIC_CLIENT_ID`, `FABRIC_CLIENT_SECRET`) and workspace variables (`PBI_WORKSPACE_ID_DEV`, `_TEST`, `_PROD`) for a `cd.yml` deployment workflow that isn't in the repo (gap G1). It also recommends GitHub environments `dev`, `test` and `prod`, with required reviewers for `test` and `prod`.

11. Custom visuals (the .pbiviz kit)

A **custom visual** is a chart type you write yourself in TypeScript and package as a `.pbiviz` bundle. The skill `.github/skills/powerbi-pbiviz-custom-visuals/SKILL.md` warns: "A custom visual is code to maintain. Say so if the request does not need one."

Before building, the agent must ask you three questions in one go:

- What should the visual show, and what decision does it support? Is there a reference screenshot or a visual to replace?
- How interactive should it be? The default is fully interactive: tooltips, click to cross-filter, highlight.
- What author name and email go in `pbiviz.json`? Packaging refuses without them.

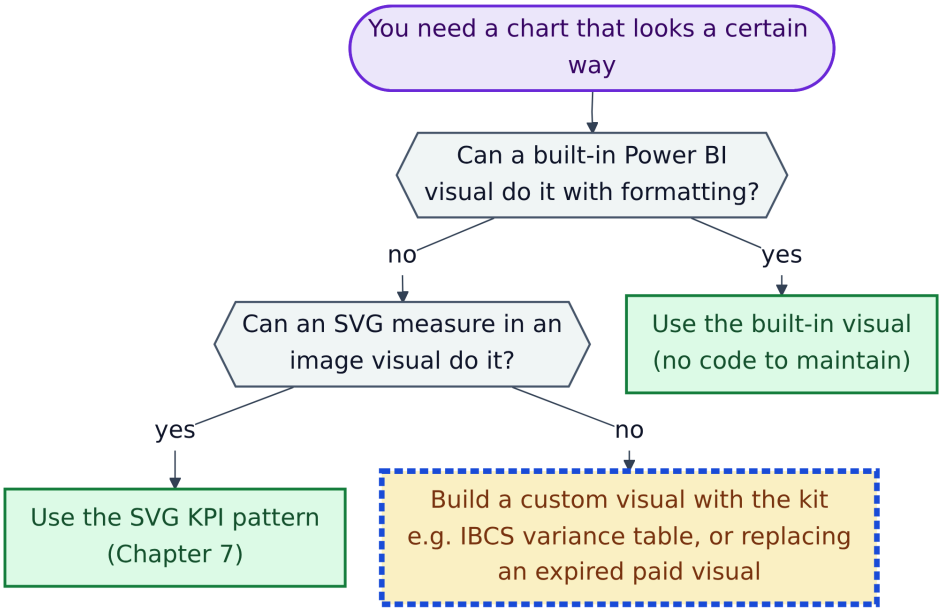


Figure 11.1 — Do you really need one?

1. Can a built-in visual do it with formatting? Use that.
2. Can an SVG measure in an image visual do it? Use the SVG KPI pattern (Chapter 7).
3. Neither? Use the kit, e.g. for an **IBCS** finance layout (a business-chart standard: actuals solid dark, variances green and red) or to replace a paid marketplace visual whose licence expired.

The workflow

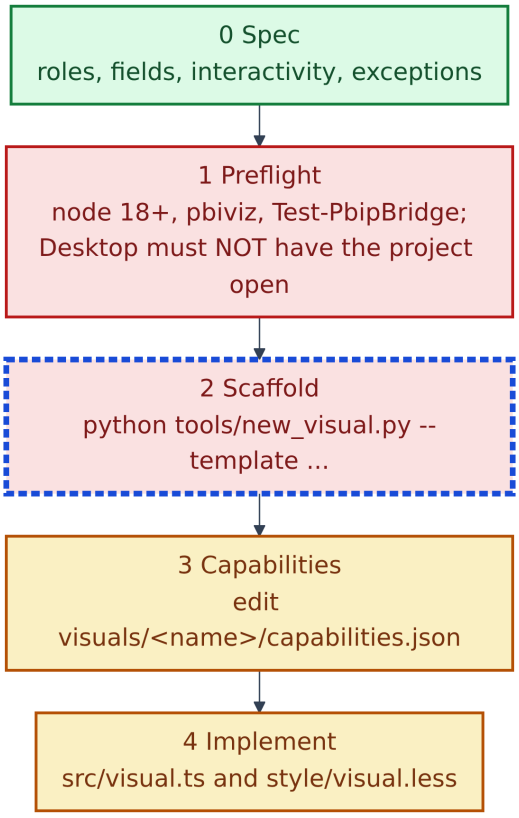


Figure 11.2 — Plan and write.

0. Spec: data roles and fields, Format-pane options, logic, interactivity, size, exceptions.

1. Preflight: Node.js 18+, `pbiviz --version` (`npm i -g powerbi-visuals-tools`), and `Test-PbipBridge.ps1`. Power BI must *not* have the project open while syncing.

2. Scaffold: `python .github\skills\powerbi-pbiviz-custom-visuals\assets\tools\new_visual.py --name finVarianceTable --template varianceTable ...`. It creates `visuals/<name>/` in your dashboard repo, gives it a permanent GUID, and runs `npm install`.

3. Capabilities: edit `capabilities.json`. Microsoft: it tells Power BI "what kind of data the visual accepts".

4. Implement: `src/visual.ts` and `style/visual.less`. Shared code is edited only in `visuals/shared/`.

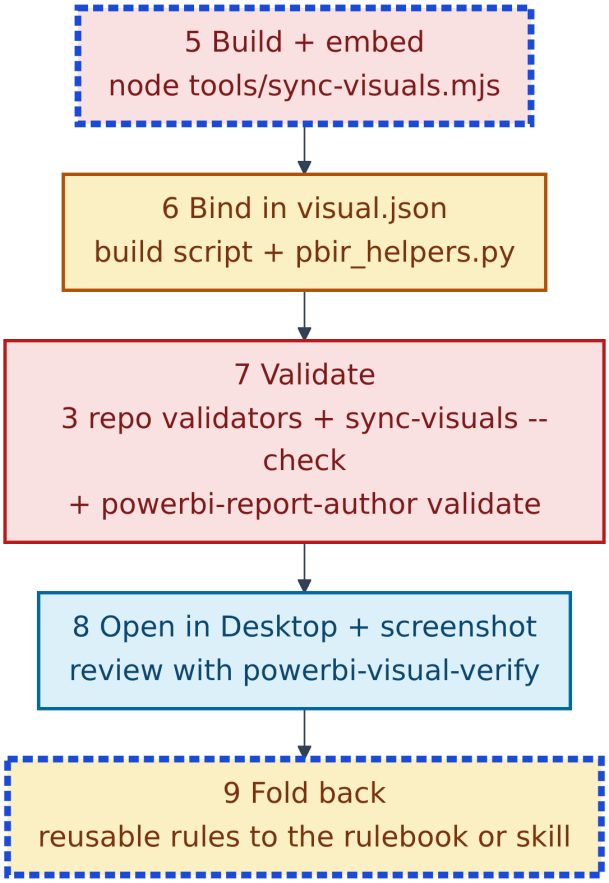


Figure 11.3 — Build, bind and prove.

5. Build and embed: `node tools\sync-visuals.mjs` (or `--only <name>`). TypeScript or lint errors stop here.

6. Bind: write the page's `visual.json`, ideally from a Python build script using `tools/pbir_helpers.py`, and check it with `check_against_capabilities(...)`.

7. Validate: the three validators, `sync-visuals.mjs --check`, and `powerbi-report-author validate`.

8. Open and screenshot: no empty boxes, licence overlays or clipped labels; totals match; clicks and expand work.

9. Fold back reusable rules.

Three things must line up

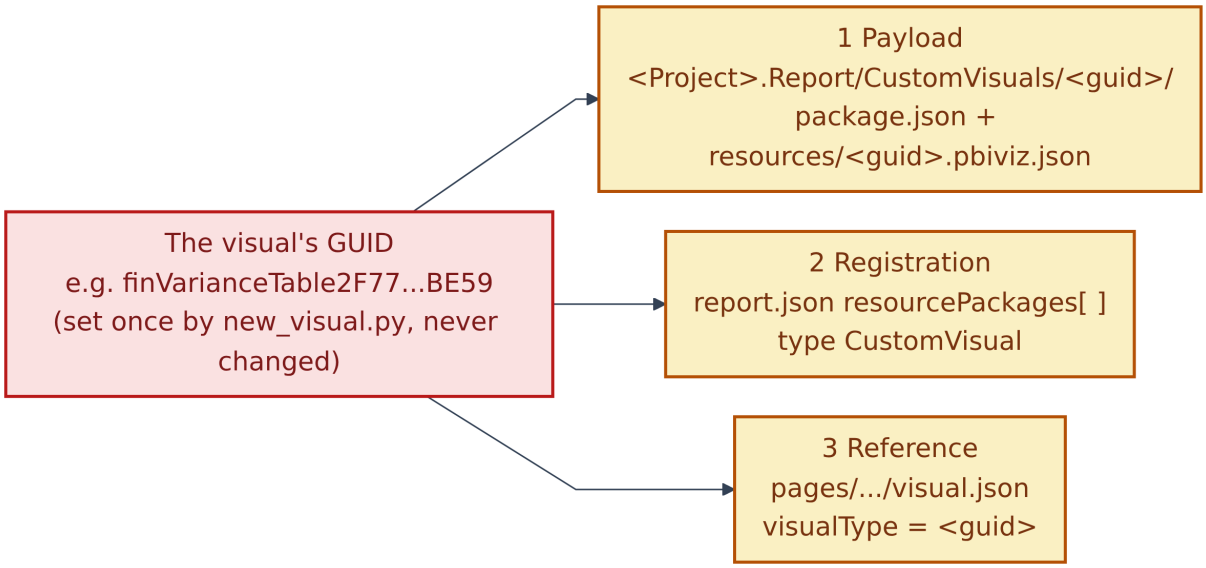


Figure 11.4 — The GUID triad (`references/PBIP-EMBEDDING.md`). A GUID is a long unique id.

1. **Payload:** the built bundle in `<Project>.Report/CustomVisuals/<guid>/`.
2. **Registration:** the GUID in `report.json` `resourcePackages`, type `CustomVisual`.
3. **Reference:** each `visual.json` names the GUID as `visualType`.

“Get one wrong and the visual renders as an empty box with **no error dialog**.” The GUID never changes once used; rename a visual by its `displayName` only.

What sync-visuals.mjs does

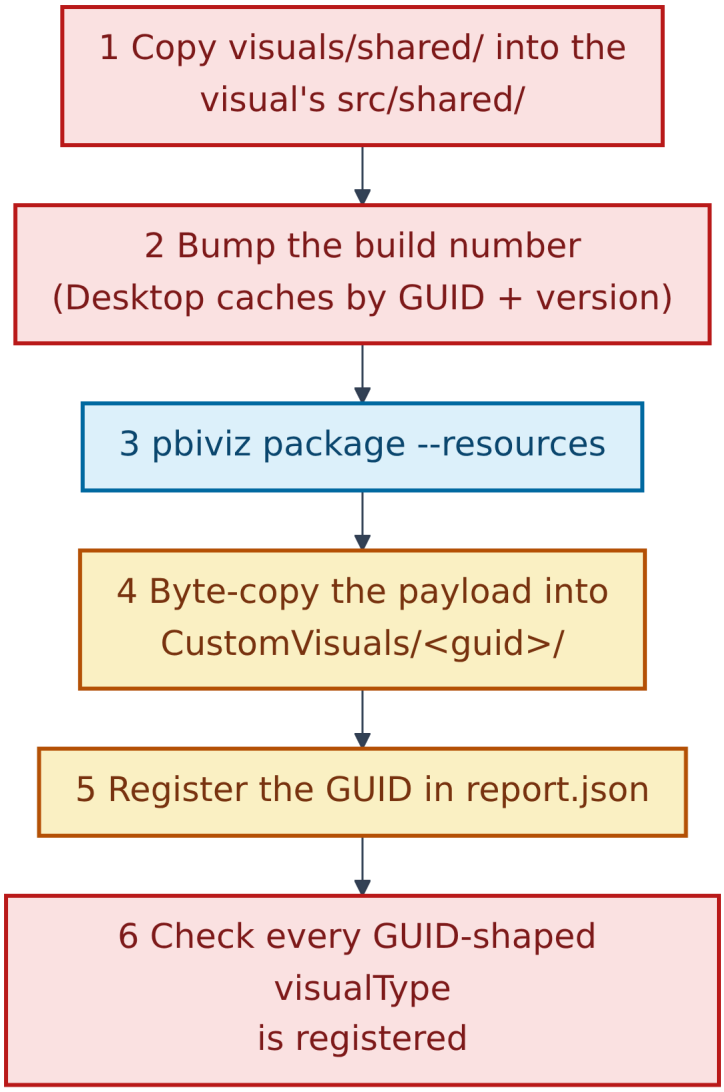


Figure 11.5 — Six steps per visual.

1. Copy the shared library into the visual’s `src/shared/`.

2. Bump the build number (Power BI caches bundles by GUID and version).

3. Run `pbiviz package --resources`.

4. Copy the payload byte for byte into `CustomVisuals/<guid>/`.

5. Register the GUID in `report.json`.

6. Check every GUID-shaped `visualType` is registered.

After syncing, close and reopen Power BI; a reload serves the cached bundle.

What’s in the kit

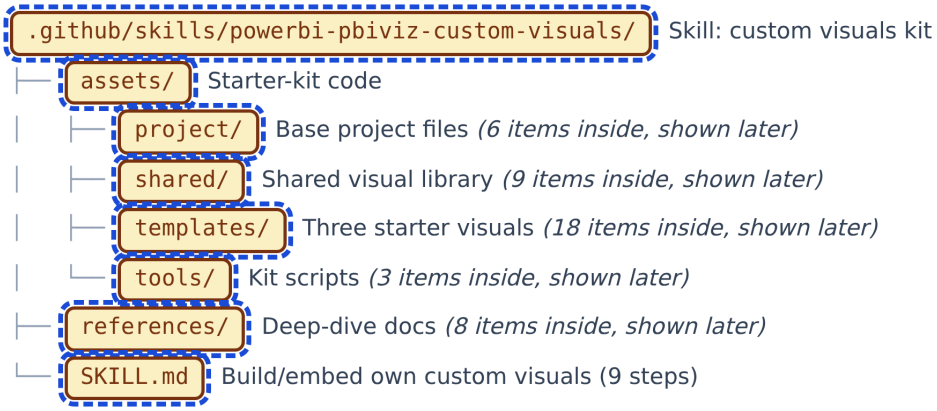


Figure 11.6 — The kit, two levels deep. All kit files are Red + AI: the agent copies them into your dashboard repo, and you change the copies.

1. `assets/project/`: starting project files with pinned versions (`powerbi-visuals-api` 5.11.1, `powerbi-visuals-utils-formattingmodel` 6.0.4, TypeScript 5.5.4).
2. `assets/shared/`: a shared TypeScript library: colour tokens, safe DOM builders (no `innerHTML`), number formats, clicks and tooltips, shapes, Format-pane settings, scales, text fitting.
3. `assets/templates/`: three starter visuals: `varianceTable`, `varianceChart`, `blank`.
4. `assets/tools/`: `new_visual.py`, `sync-visuals.mjs`, `pbir_helpers.py`.
5. `references/`: eight deep-dive documents.
6. `SKILL.md`: the procedure.

The hard rules, short version

1. A GUID never changes once bound.
2. `visual.json` keys must match the role names and options in `capabilities.json`.
3. Close Power BI while syncing; reopen after.
4. Copy built payloads byte for byte.
5. No `innerHTML`, no internet access (`privileges: []`), no vendor branding.
6. Visuals hide Power BI’s container chrome; the SVG background draws cards and titles.
7. Visuals that add up numbers in code only use additive measures; ratios are computed from their parts.
8. Never copy a paid visual’s pop-ups, logo or patent strip.
9. Interactive by default; always signal rendering start and finish (needed for PDF/PowerPoint export).
10. A successful build is not done; only a reviewed screenshot counts.

`.github/instructions/power-bi-custom-visuals-development.instructions.md` loads automatically for `.ts`, `.tsx`, `.js`, `.jsx`, `.less` and `.css` files. It’s a general React and D3.js reference; use the skill for anything that touches the PBIP project.

12. The Python email and alert guides

The `python/` folder holds **instructions only**; no Python script ships with the template. You (or the agent) follow these guides to write scripts in your own project.

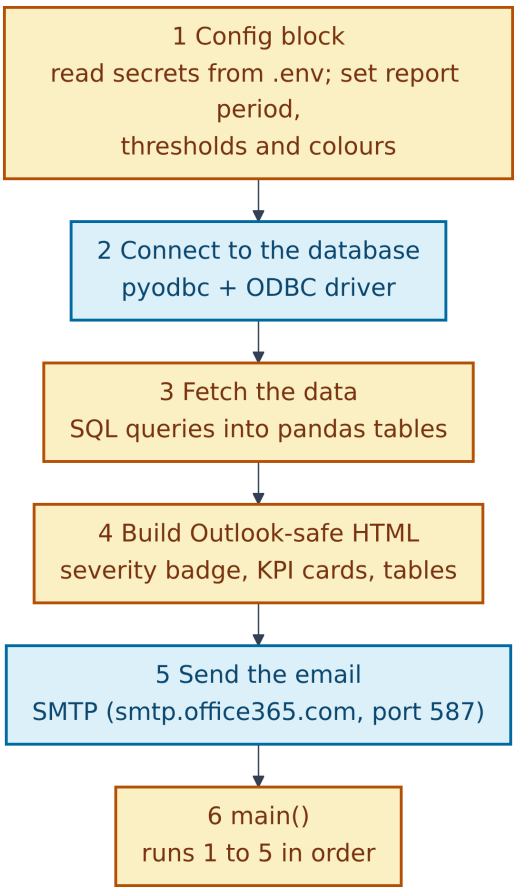


Figure 12.1 — How the alert-email script is organised (`python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md`).

1. Config: load secrets from `.env` and set the period, thresholds and colours.

2. Connect to the database with `pyodbc` and an ODBC driver.

3. Fetch data with SQL into `pandas` tables.

4. Build HTML that Outlook displays correctly: severity badge, KPI cards, tables.

5. Send with SMTP; the example uses `smtp.office365.com`, port 587.

6. `main()` runs the steps in order.

Its `requirements.txt` lists `pyodbc`, `pandas` and `python-dotenv`.

How the alert picks its colour

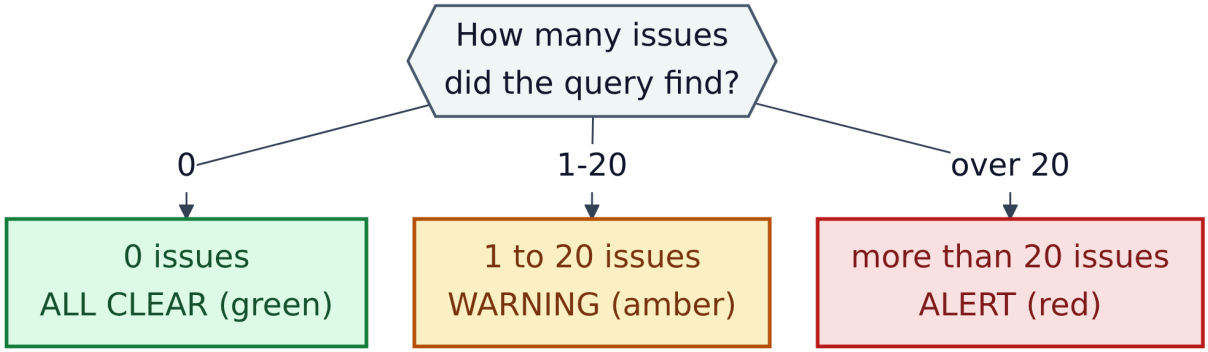


Figure 12.2 — The severity rule (section 7.1). These colours are about urgency, not the file-safety legend.

1. 0 issues: “ALL CLEAR”, green.
2. 1 to 20 issues: “WARNING”, amber.
3. More than 20: “ALERT”, red.

Classify first, because the level sets the badge colour and the subject line.

Outlook-safe HTML

Do NOT use (Outlook breaks it)	Use instead
`background: shorthand`	`background-color: inline`
`linear-gradient(...)`	`a flat background-color`
`border-radius:`	`skip it (Outlook ignores it)`
`box-shadow:`	`skip it (not supported)`
`display:flex / display:grid`	`<table> layout only`
`<div> for coloured backgrounds`	`<td bgcolor="...">`
`max-width: on <table>`	`width="720" attribute`

Source: python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md, section 7.2 (Outlook-Safe HTML Rules).

Figure 12.3 — What Outlook breaks, and what to use instead.

1. Desktop Outlook ignores rounded corners, shadows, gradients and flex or grid layouts.
2. So the guide builds layout from `<table>` elements, inline `background-color`, and a fixed 720-pixel width.

Making Python numbers match Power BI

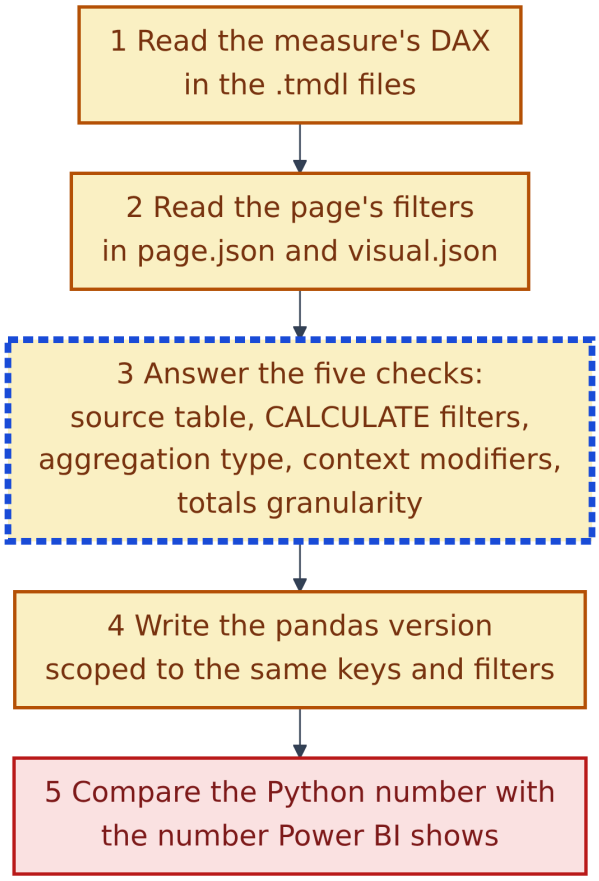


Figure 12.4 — The DAX-to-Python check (`python/email_reporting/emal_style_instructions.md`, which calls it “MANDATORY”).

1. Read the measure’s DAX in the `.tmdl` files; the model and page JSON are “the source of truth”.

2. Read the page’s filters in `page.json` and `visual.json`.

3. Answer five checks for each measure: which source table; every `CALCULATE` filter; the aggregation type (`SUM`, `DISTINCTCOUNT`, `SUMX`, `AVERAGEX` differ in totals); context modifiers like `ALL(...)`; and whether totals double-count at a finer grain.

4. Write the pandas version with the same keys and filters, using helpers like `safe_div` and `fmt`/`fmt_pct`.

5. Compare with the number Power BI shows. They must match before any email goes out.

The three files

File	What it gives you
`python/README.md`	How to build your own value checker: read the PBIP, find a “Sales by Branch” visual, generate SQL, support `--dry-run`. Example: `python python/<your_script_name>.py --branch ABV --month March --year 2026`
`python/db_alerts/ALERT_EMAIL_INSTRUCTIONS.md`	A template for any database-triggered alert email: `.env` credentials, six phases, Outlook HTML parts, a pre-send checklist
`python/email_reporting/emal_style_instructions.md`	Rules for emails that copy a Power BI page. It is three documents joined: a boilerplate skill, Parts A–E (validation, HTML, Python structure, charts, project reference), and an HTML style guide

Safety: never hard-code passwords; both guides read them from `.env`. Use the email guide’s preview mode, which writes the HTML to a file instead of sending it, before emailing real people.

13. Which file do I touch?

Start with the quick look-up table. If your task isn’t in it, the three decision trees after it walk you to the right file.

Quick look-up

I want to…	Touch	Safety	Then run
Plan a page	copy `specs/TEMPLATE.md`	Green	nothing yet
Add a measure	`tables/_Measures.tmdl`	Amber	`Test-PbipSemantics.ps1`
Add an SVG KPI card	`tables/_SVG_Measures.tmdl` + image `visual.json`	Amber	validators + verify loop
Add a table	`tables/<Table>.tmdl` + `model.tmdl`	Amber	all three validators
Add a page	new `pages/<id>/page.json` + `pages.json`	Amber	`validate-pbip.ps1`, verify loop

I want to…	Touch	Safety	Then run
Add a background or theme	file in `RegisteredResources/` + `report.json` entry	Amber	`validate-pbip.ps1`
Change team colours	`docs/theme/Theme.json`, then promote	Amber	reopen Power BI, verify loop
Fix a typo in a doc	the `.md` under `docs/` or `specs/`	Green	nothing
Change what the AI always does	`.github/copilot-instructions.md`	Amber + AI	get a review
Store a secret for Python	your `.env`	Amber	`git check-ignore -v .env`
Store a secret for Actions	*Settings > Secrets and variables > Actions*	–	rerun the workflow
Change the test workflow	`.github/workflows/ai-testing.yml`	Red	ask a maintainer
Edit `version.json`, `.platform`, `definition.pbir`, `.pbism`, `database.tmdl`	**don't**	Red	–
Edit `tests/results/`, `tests/screenshots/`, `.github/.pbip-state/`, `**/.pbi/`	**don't**: generated	Red	–

Decision trees

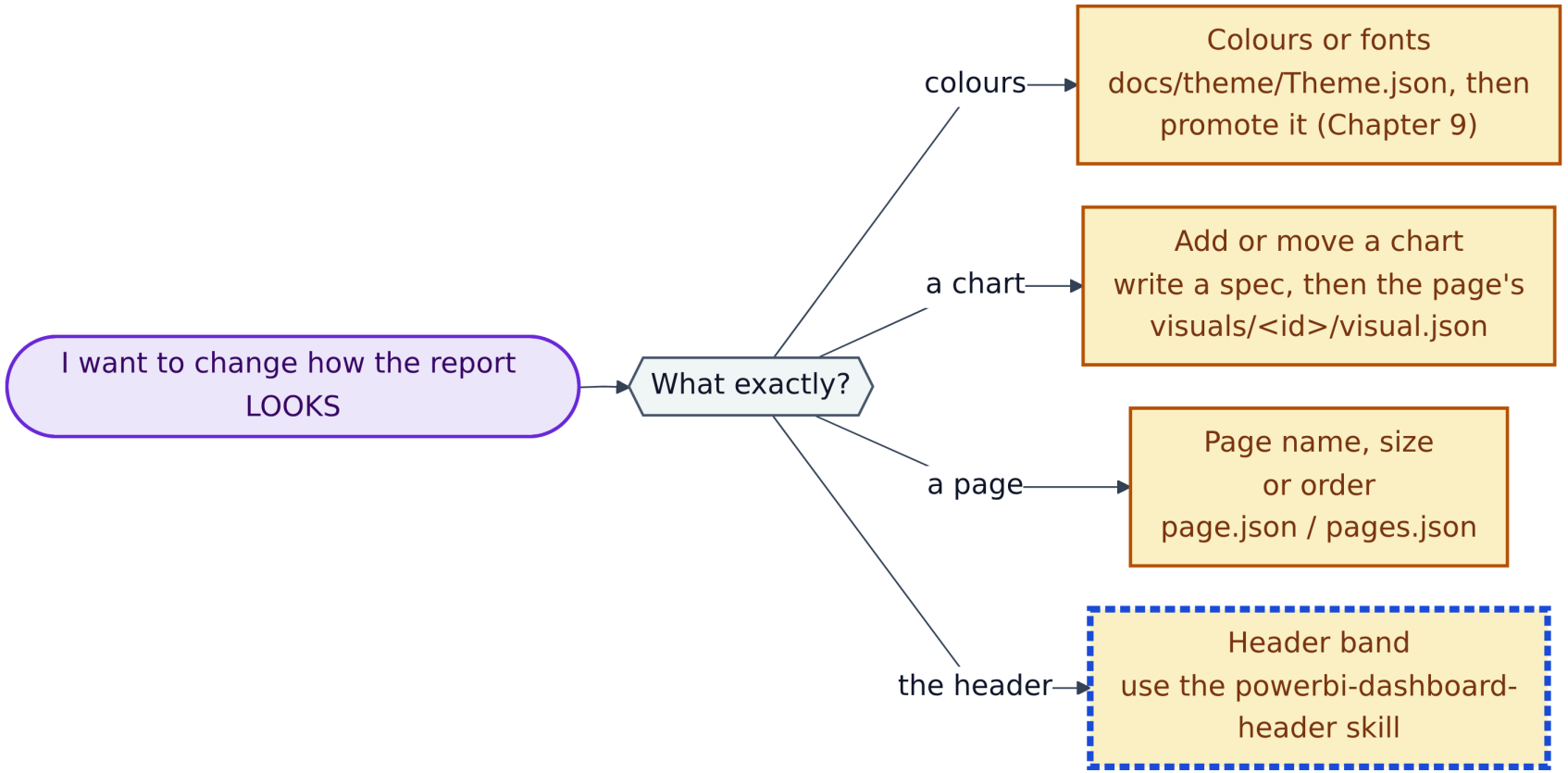


Figure 13.1 — “I want to change how the report looks.”

1. **Colours or fonts:** edit `docs/theme/Theme.json`, then promote it (Chapter 9). Don’t edit only the copy in `RegisteredResources/`; the next promotion overwrites it.
2. **Add or move a chart:** update the spec, then the page’s `visuals/<id>/visual.json`. Mind the `card_y + 30` rule.
3. **Page name, size or order:** the page’s `page.json`, or `pages.json`.
4. **Header band:** follow the `powerbi-dashboard-header` skill and note any exception in the spec.

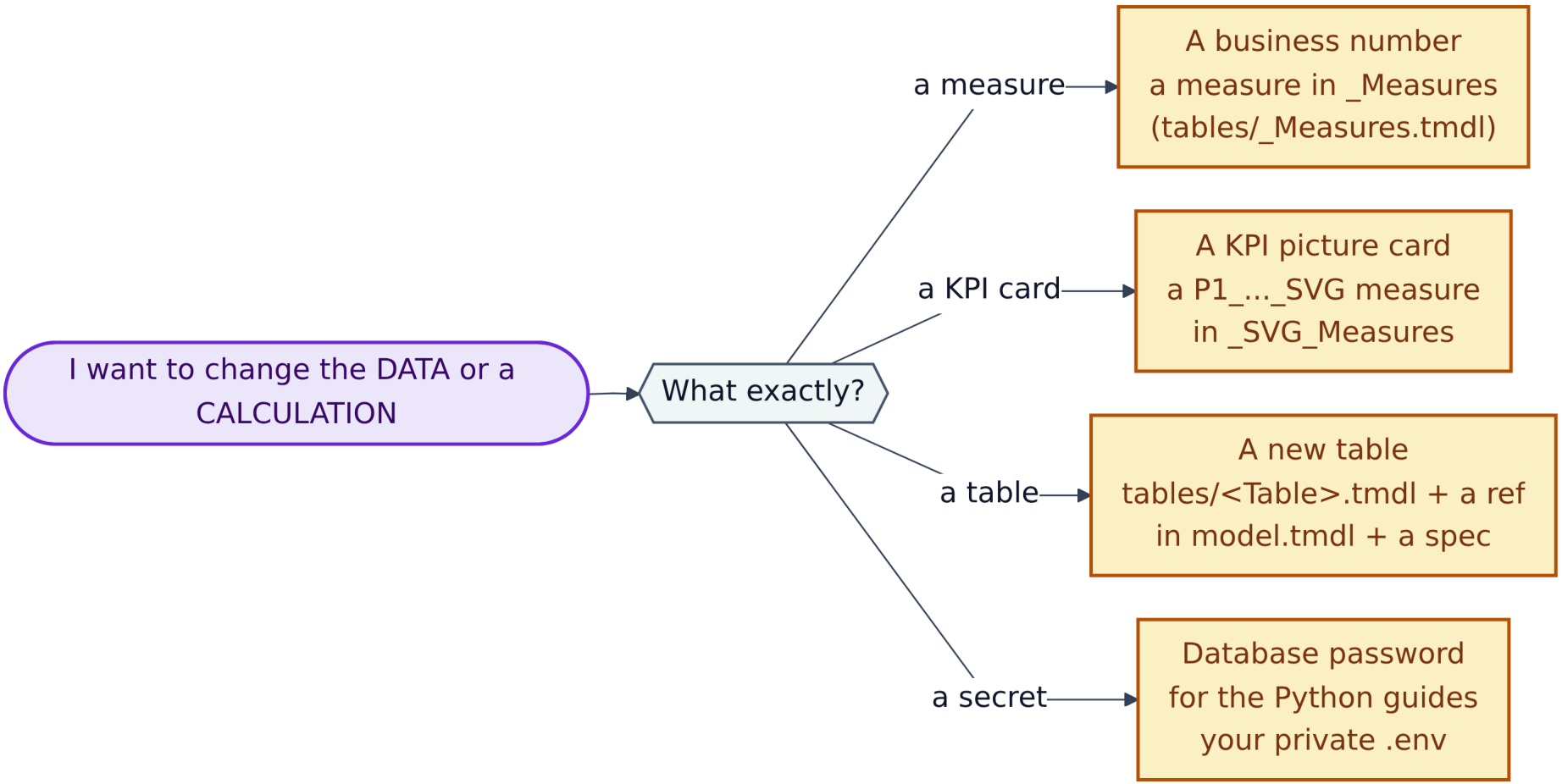


Figure 13.2 — "I want to change the data or a calculation."

1. **A business number:** a measure in `_Measures`, with a `///` description.
2. **A KPI picture card:** an SVG measure in `_SVG_Measures`, shown with an image visual.
3. **A new table:** `definition/tables/<Table>.tmdl`, a `ref table` line in `model.tmdl`, and a spec.
4. **A database password for the Python guides:** your private `.env` only.

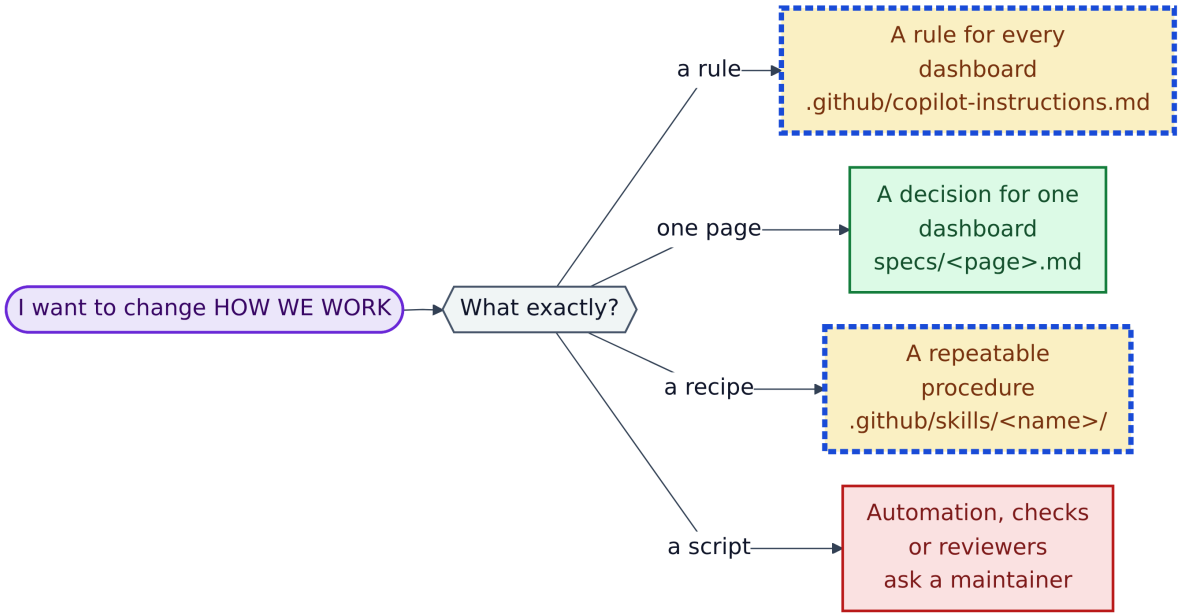


Figure 13.3 — “I want to change how we work.”

1. **A rule for every dashboard:** `.github/copilot-instructions.md`, then align `copilot-instructions.md` and `AGENTS.md`. Get it reviewed.
2. **A decision for one dashboard:** the spec.
3. **A repeatable procedure:** a skill, `.github/skills/<name>/SKILL.md`; the folder name must match its `name:` and the `description:` must say when to use it.
4. **Automation, checks or reviewers** (`scripts/`, `.github/workflows/`, `.github/CODEOWNERS`): ask a maintainer.

14. Known gaps and how to fix them

While reading every file for this guide, we found 15 places where the repo’s docs disagree with each other, point to things that don’t exist, or promise behaviour the scripts don’t have. None of them stops the template working, but each can trip you up.

This guide changes no repo files. Each fix below is for you or a maintainer to make as a separate change.

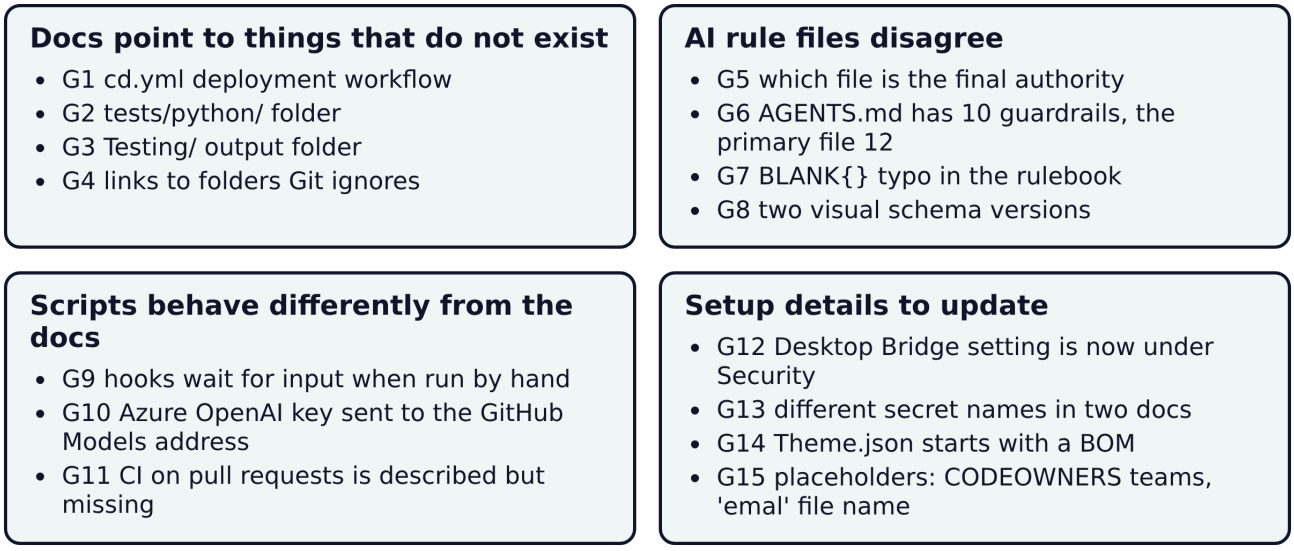


Figure 14.1 — The 15 gaps in four groups.

1. **G1–G4:** docs point to things that don't exist.
2. **G5–G8:** the AI rule files disagree.
3. **G9–G11:** scripts behave differently from the docs.
4. **G12–G15:** setup details to update.

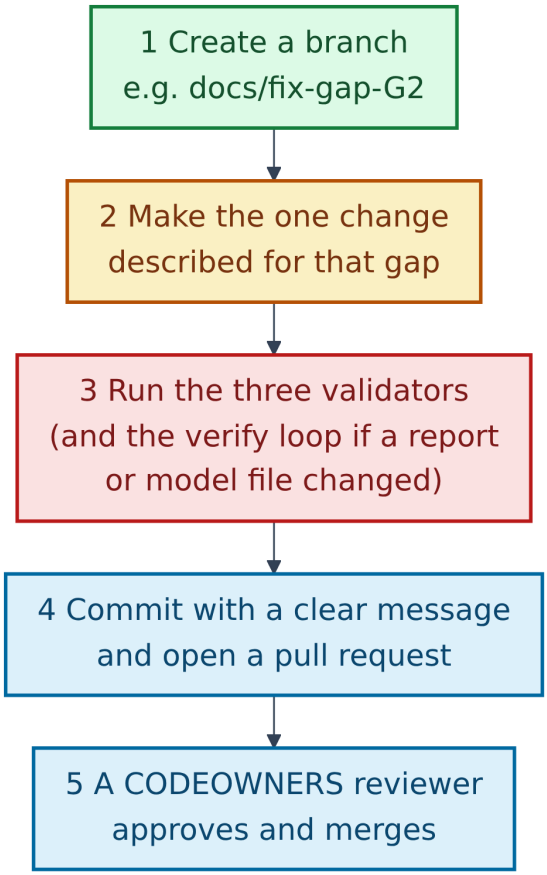


Figure 14.2 — Apply any fix in five steps.

1. Create a branch named after the gap, e.g. `docs/fix-gap-G2`.

2. Make only that change.

3. Run the three validators (and the verify loop if you touched report or model files).

4. Commit and open a pull request naming the gap.

5. A `CODEOWNERS` reviewer approves and merges. Amber + AI files always need a review.

Docs point to things that don't exist

G1. No `cd.yml`. `docs/CICD-SETUP.md` describes deployments "using `workflow_dispatch` in `.github/workflows/cd.yml`", but only `ai-testing.yml` exists. *Fix:* if you don't need automated deployment yet, mark sections 2–6 of that doc as "planned". If you do, have a

platform engineer write `cd.yml` with the inputs the doc names (`target_environment`, `git_ref`) and the listed secrets, and test it on `dev` first.

G2. No `tests/python/` folder. `docs/CICD-SETUP.md`, `tests/README.md` and `tests/prompts/Python_Data_Value_Test.md` point to `tests/python/`, but the guidance lives in `python/README.md`. *Fix:* replace `tests/python/` with `python/` in those three files, then search the repo for `tests/python` to confirm none are left.

G3. Wrong output folder in the audit prompt. `tests/prompts/PowerBI_Test.md` says to write `Testing/test_results.md`; everything else uses `tests/results/test_results.md`. *Fix:* change both mentions (hard requirement 4 and Phase 5), then run the prompt once and confirm the file lands in `tests/results/`.

G4. Links to ignored folders. `docs/REPOSITORY-GUIDE.md` links to `tests/screenshots/`, `.github/.pbip-state/` and `Proposals/`. Git ignores them, so the links 404 on GitHub. *Fix:* keep the names in code formatting but remove the links; optionally add "(created locally, never committed)".

The AI rule files disagree

G5. Which file is "the final authority"? The README, `AGENTS.md` and `.github/copilot-instructions.md` say `.github/copilot-instructions.md` wins. The three `.github/agents/*.agent.md` files and the `powerbi-dashboard-header` and `powerbi-pbiviz-custom-visuals` skills say the root `copilot-instructions.md` does. *Fix:* in those five files, change the sentence to the README's wording ("`.github/copilot-instructions.md` is the primary policy authority; `copilot-instructions.md` is the detailed PBIP reference") and have a reviewer compare them.

G6. 10 vs 12 guardrails. `.github/copilot-instructions.md` has 12; `AGENTS.md` and the README have 10. An agent that reads only `AGENTS.md` (like Claude Code) misses rules 11 and 12. *Fix:* copy rules 11 and 12 into `AGENTS.md` and the README's guardrail list; check the wording matches.

G7. `BLANK{}` typo. Step 2 of the creation steps in `copilot-instructions.md` says `_Measures = BLANK{}`; the same file correctly writes `_SVG_Measures = { BLANK() }`. *Fix:* replace `BLANK{}` with `{ BLANK() }`.

G8. Two visual schema versions. The rulebook's default is `visualContainer/2.7.0`; the kit's `pbir_helpers.py` writes `2.12.0`. *Fix:* first check which version your Power BI Desktop writes for a new visual, agree one default, then align the rulebook (its "Schema (repo default)" line and five templates) or `SCHEMA_VIZ` in `pbir_helpers.py`, and note why.

Scripts behave differently from the docs

G9. Hooks wait for input when run by hand. The README says the hooks are run manually, but `pbip-write-guard.ps1`, `pbip-validate-hook.ps1` and `pbip-stop-hook.ps1` start by reading hook data from standard input (`[Console]::In.ReadToEnd()`), as an agent tool like Claude Code sends it. In a plain terminal they just wait. *Fix:* pipe in an empty object: `'{}' | powershell -NoProfile -File scripts\hooks\pbip-stop-hook.ps1`. For everyday checks use `Test-PbipSemantics.ps1 -Path <file>` and `Invoke-PbipVerify.ps1 -Mode Review` instead. Then document the `'{}' |` pattern in the README.

G10. Azure key sent to the GitHub Models address. `run-ai-md-test.ps1` uses `GITHUB_MODELS_API_KEY` or else `AZURE_OPENAI_API_KEY`, always as a `Bearer` token to the default GitHub Models endpoint, and the workflow never changes the endpoint. So a repo with only the Azure secret will very likely fail to authenticate (from reading the code; not tested live). *Fix:* use `GITHUB_MODELS_API_KEY` and mark Azure OpenAI "not yet supported" in the docs, or have a maintainer pass an endpoint to `-ApiEndpoint` from the workflow and send the key the way your Azure resource expects.

G11. Pull-request CI is described but missing. `docs/CICD-SETUP.md` says pull requests into `dev`, `test` and `main` run CI; no workflow runs on `pull_request`. *Fix:* add a small workflow on `pull_request` that runs `validate-json.ps1` and `validate-pbip.ps1` with `pwsh`, then make it a required check in branch protection.

Setup details to update

G12. The Desktop Bridge setting moved. The repo says *Options > Preview features*; Microsoft's current page says *Options > Security > Desktop Bridge*, on by default, and states no minimum version (2.155.756.0 is the repo's own finding). *Fix:* update the path in the README and `copilot-instructions.md` to "Security (older builds: Preview features)", have a maintainer update the "Fix" messages in `Test-PbipBridge.ps1` and `PbipBridge.Common.ps1`, and link Microsoft's page.

G13. Two sets of secret names. `.env.example` and the alert guide use `DB_SERVER`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_DRIVER`; section 9 of `docs/CICD-SETUP.md` uses `SERVER`, `DATABASE`, `USERNAME`, `PASSWORD`, `DRIVER`. *Fix:* switch the CI/CD doc to the `DB_` names.

G14. `Theme.json` has a BOM. Python's `json.load` rejects it ("Unexpected UTF-8 BOM"). *Fix:* in VS Code click the encoding label in the status bar, choose *Save with Encoding* → *UTF-8*, then rerun `validate-json.ps1`. (The files in `tests/results/` also have one; they're generated, so leave them.)

G15. Placeholders. `CODEOWNERS` uses example handles (`@your-org/...`); `docs/REPOSITORY-GUIDE.md` uses `<upstream-owner>/<upstream-repo>`; and the email guide is spelled `emal_style_instructions.md`. *Fix:* replace the handles with real teams in your fork; use `GibbonGambits/PGE-PBIP-Template` for upstream; optionally rename the guide and update the README reference in the same change.

Appendix A. File-by-file directory

Every one of the repository's 176 files and folders, grouped by folder. Each group has a folder tree (coloured with the legend from Start here) and a table. Folders get one line. Paths in each table are relative to the folder named in its heading.

A.1 Root files

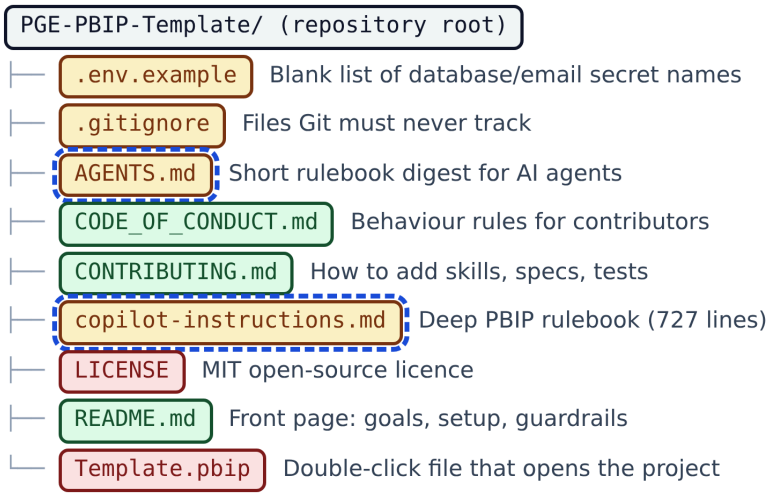


Figure A.1 — Root files. The nine files that sit directly in the repository's top folder.

Item	Type	What it is	Safety
`.env.example`	Env template	A blank form listing the names of the secrets the Python guides need, with no real values.	Amber
`.gitignore`	Git config	The list of files Git must never save, such as your private .env and Power BI's local cache.	Amber
`AGENTS.md`	AI steering	A one-page summary of the house rules for any AI agent.	Amber · AI
`CODE_OF_CONDUCT.md`	Community doc	How people in this project are expected to treat each other.	Green
`CONTRIBUTING.md`	Community doc	Short rules for adding to the project: where skills, specs and tests go.	Green
`copilot-instructions.md`	AI reference	The full technical rulebook for building PBIP dashboards: file layouts, forbidden properties, JSON templates, SVG measure patterns.	Amber · AI
`LICENSE`	Legal	The legal permission to use, copy and change this code.	Red

Item	Type	What it is	Safety
`README.md`	Doc	The front page of the repository: goals, guardrails, quick start, and how the verify loop and custom-visual kit work.	Green
`Template.pbip`	PBIP entry	The file you double-click to open the project in Power BI Desktop.	Red

A.2 Root folders

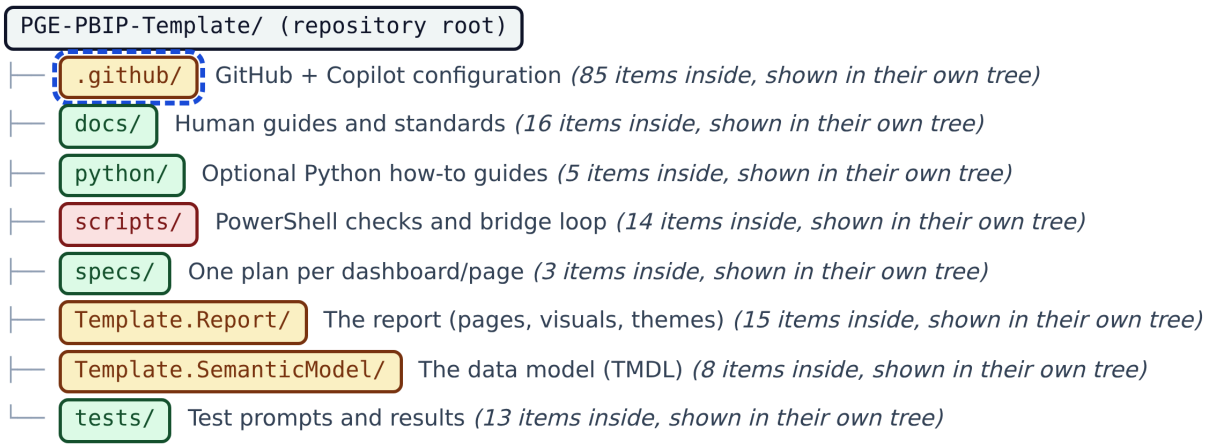


Figure A.2 — Root folders. The nine top-level folders, each opened in its own tree below.

Item	Type	What it is	Safety
`.github/`	Folder	GitHub + Copilot configuration	Amber · AI
`docs/`	Folder	Human guides and standards	Green
`python/`	Folder	Optional Python how-to guides	Green
`scripts/`	Folder	PowerShell checks and bridge loop	Red
`specs/`	Folder	One plan per dashboard/page	Green
`Template.Report/`	Folder	The report (pages, visuals, themes)	Amber
`Template.SemanticModel/`	Folder	The data model (TMDL)	Amber
`tests/`	Folder	Test prompts and results	Green

A.3 .github: top level

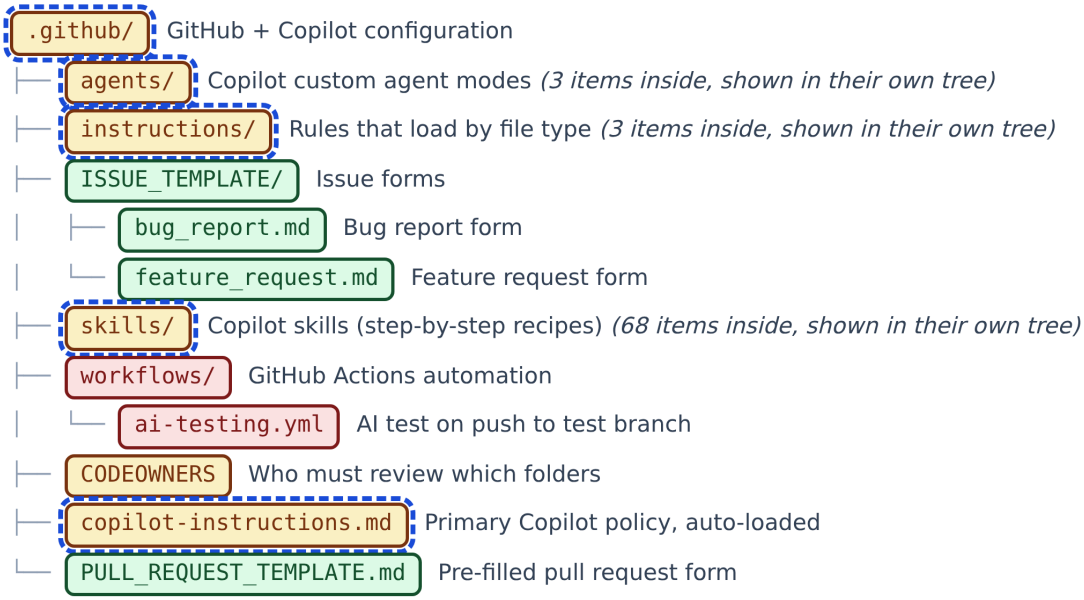


Figure A.3 — .github: top level. GitHub's own configuration, plus the folders the AI agent reads. Folders shown closed are opened in their own tree.

Item	Type	What it is	Safety
`ISSUE_TEMPLATE/`	Folder	Issue forms	Green
`ISSUE_TEMPLATE/bug_report.md`	GitHub template	The form for reporting something broken.	Green
`ISSUE_TEMPLATE/feature_request.md`	GitHub template	The form for asking for something new.	Green
`workflows/`	Folder	GitHub Actions automation	Red
`workflows/ai-testing.yml`	CI workflow	Runs the AI test automatically when code reaches the test branch.	Red
`CODEOWNERS`	GitHub config	Says which team must review changes to which folders.	Amber
`copilot-instructions.md`	AI steering	GitHub Copilot's main rulebook for this repository, loaded automatically every time.	Amber · AI
`PULL_REQUEST_TEMPLATE.md`	GitHub template	The form every pull request starts with.	Green

A.4 .github/agents and .github/instructions

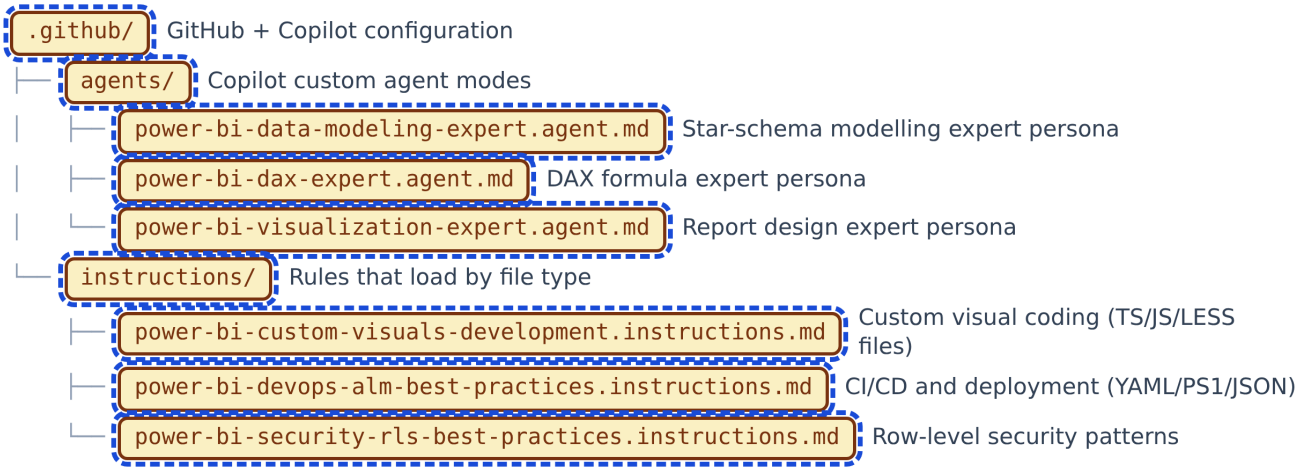


Figure A.4 — .github/agents and .github/instructions. Expert modes and file-type rules for GitHub Copilot.

Item	Type	What it is	Safety
`agents/`	Folder	Copilot custom agent modes	Amber · AI
`agents/power-bi-data-modeling-expert.agent.md`	Agent mode	Turns Copilot into a data-model expert (star schemas, relationships).	Amber · AI
`agents/power-bi-dax-expert.agent.md`	Agent mode	Turns Copilot into a DAX formula expert.	Amber · AI
`agents/power-bi-visualization-expert.agent.md`	Agent mode	Turns Copilot into a report-design expert.	Amber · AI
`instructions/`	Folder	Rules that load by file type	Amber · AI
`instructions/power-bi-custom-visuals-development.instructions.md`	Scoped rules	Coding advice for writing custom charts.	Amber · AI
`instructions/power-bi-devops-alm-best-practices.instructions.md`	Scoped rules	Advice on releasing reports safely through test and production.	Amber · AI
`instructions/power-bi-security-rls-best-practices.instructions.md`	Scoped rules	Advice on making sure each person sees only the data they should.	Amber · AI

A.5 .github/skills, part 1

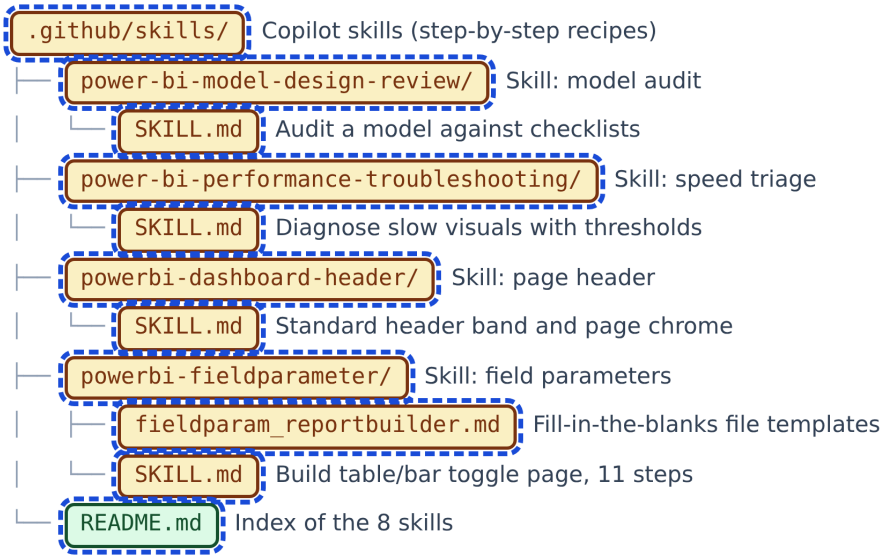


Figure A.5 — .github/skills, part 1. The skills index and the first four skills.

Item	Type	What it is	Safety
`skills/`	Folder	Copilot skills (step-by-step recipes)	Amber · AI
`power-bi-model-design-review/`	Folder	Skill: model audit	Amber · AI
`power-bi-model-design-review/SKILL.md`	Skill	A recipe for auditing an existing data model.	Amber · AI
`power-bi-performance-troubleshooting/`	Folder	Skill: speed triage	Amber · AI
`power-bi-performance-troubleshooting/SKILL.md`	Skill	A recipe for finding why a report is slow.	Amber · AI
`powerbi-dashboard-header/`	Folder	Skill: page header	Amber · AI
`powerbi-dashboard-header/SKILL.md`	Skill	A recipe for a consistent header band across pages: logo, title, filters, buttons.	Amber · AI
`powerbi-fieldparameter/`	Folder	Skill: field parameters	Amber · AI

Item	Type	What it is	Safety
`powerbi-fieldparameter/fieldparam_reportbuilder.md`	Skill boilerplate	Fill-in-the-blanks file templates for the same page.	Amber · AI
`powerbi-fieldparameter/SKILL.md`	Skill	A recipe for a page where users switch what a table and bar chart show.	Amber · AI
`README.md`	Doc	The index card listing the eight skills.	Green · AI

A.6 .github/skills, part 2

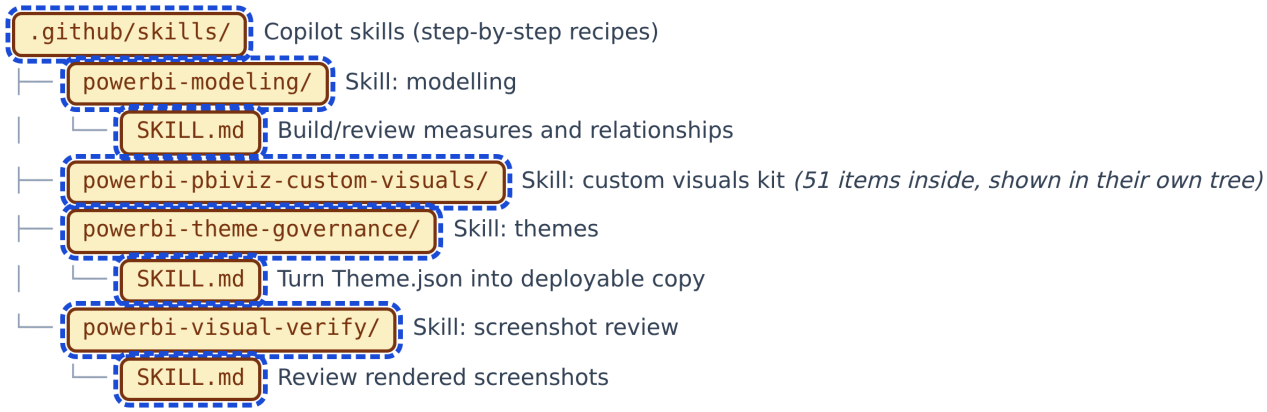


Figure A.6 — .github/skills, part 2. Three more skills, and the custom-visual kit, which gets its own trees.

Item	Type	What it is	Safety
`powerbi-modeling/`	Folder	Skill: modelling	Amber · AI
`powerbi-modeling/SKILL.md`	Skill	A recipe for building and reviewing measures and relationships.	Amber · AI
`powerbi-theme-governance/`	Folder	Skill: themes	Amber · AI
`powerbi-theme-governance/SKILL.md`	Skill	A recipe for turning the editable theme into a safe, loadable one.	Amber · AI
`powerbi-visual-verify/`	Folder	Skill: screenshot review	Amber · AI
`powerbi-visual-verify/SKILL.md`	Skill	A recipe for checking screenshots of the finished pages.	Amber · AI

A.7 Custom-visual kit: project files and tools

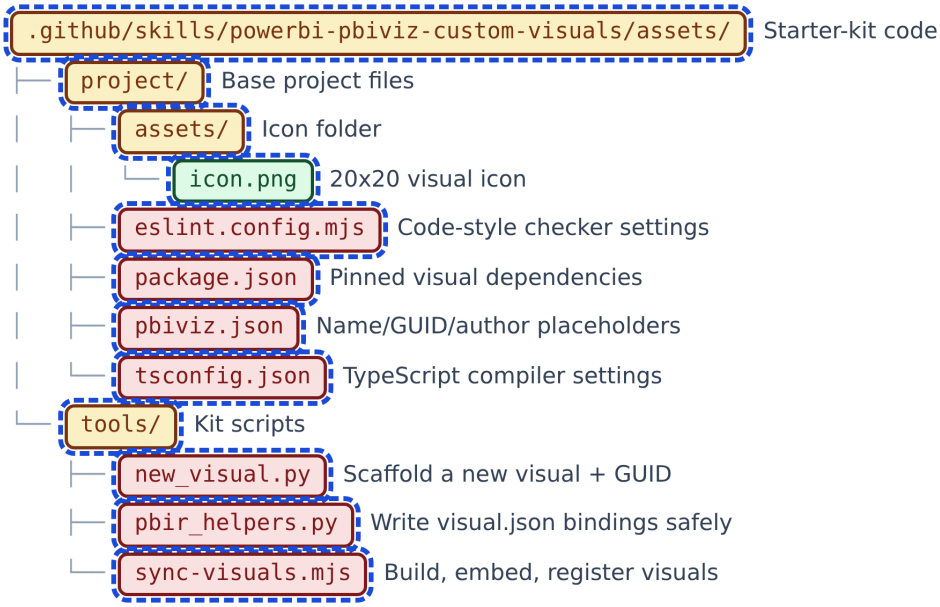


Figure A.7 — Custom-visual kit: project files and tools. The base project every visual starts from, and the three kit scripts.

Item	Type	What it is	Safety
`assets/`	Folder	Starter-kit code	Amber · AI
`project/`	Folder	Base project files	Amber · AI
`project/assets/`	Folder	Icon folder	Amber · AI
`project/assets/icon.png`	Image	The tiny 20 x 20 picture shown for the visual in Power BI.	Green · AI
`project/eslint.config.mjs`	Lint config	Settings for the code-style checker.	Red · AI
`project/package.json`	npm manifest	The shopping list of code libraries the visual needs, with exact versions.	Red · AI
`project/pbiviz.json`	Visual manifest	The visual’s identity card with blanks for name, GUID and author.	Red · AI
`project/tsconfig.json`	TS config	Settings for turning TypeScript into JavaScript.	Red · AI
`tools/`	Folder	Kit scripts	Amber · AI
`tools/new_visual.py`	Python	Creates a new visual project from a template and gives it a permanent ID.	Red · AI

Item	Type	What it is	Safety
`tools/pbir_helpers.py`	Python	Helpers for writing a page's visual files correctly.	Red · AI
`tools/sync-visuals.mjs`	Node script	Builds every visual and plugs it into the report.	Red · AI

A.8 Custom-visual kit: shared library

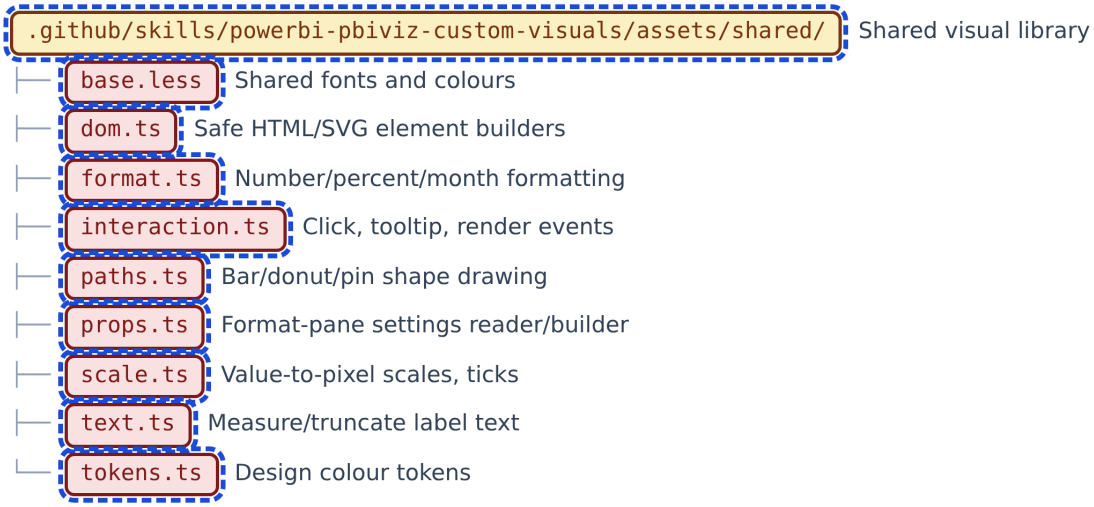


Figure A.8 — Custom-visual kit: shared library. Code shared by every custom visual.

Item	Type	What it is	Safety
`shared/`	Folder	Shared visual library	Amber · AI
`base.less`	Stylesheet	Shared fonts and colours for every visual's styling.	Red · AI
`dom.ts`	TypeScript	Safe helpers for creating page elements.	Red · AI
`format.ts`	TypeScript	Turns raw numbers into readable labels such as thousands and percentages.	Red · AI
`interaction.ts`	TypeScript	Makes visuals respond to clicks and hovering.	Red · AI
`paths.ts`	TypeScript	Draws shapes such as rounded bars and donut rings.	Red · AI
`props.ts`	TypeScript	Reads and builds the visual's settings panel.	Red · AI
`scale.ts`	TypeScript	Converts numbers into positions on the screen.	Red · AI

Item	Type	What it is	Safety
`text.ts`	TypeScript	Measures and shortens labels so they fit.	Red · AI
`tokens.ts`	TypeScript	The master list of colours and fonts for the visuals.	Red · AI

A.9 Custom-visual kit: templates, blank

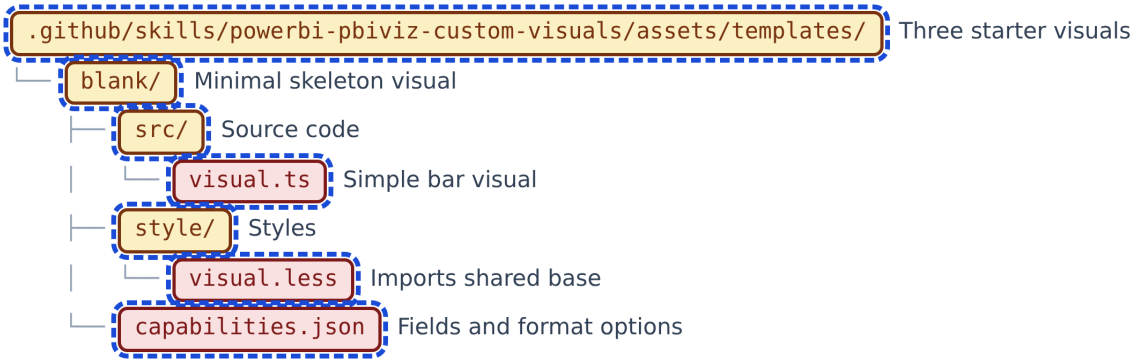


Figure A.9 — Custom-visual kit: templates, blank. The templates folder and the minimal ‘blank’ starter visual.

Item	Type	What it is	Safety
`templates/`	Folder	Three starter visuals	Amber · AI
`blank/`	Folder	Minimal skeleton visual	Amber · AI
`blank/src/`	Folder	Source code	Amber · AI
`blank/src/visual.ts`	TypeScript	The drawing code for the simple bar visual.	Red · AI
`blank/style/`	Folder	Styles	Amber · AI
`blank/style/visual.less`	Stylesheet	The visual’s style sheet.	Red · AI
`blank/capabilities.json`	Visual contract	Tells Power BI which fields this visual accepts and which settings it has.	Red · AI

A.10 Custom-visual kit: templates, varianceChart

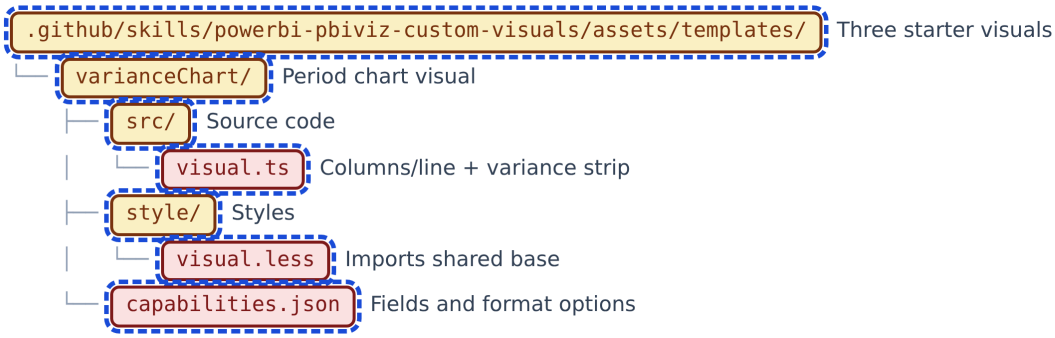


Figure A.10 — Custom-visual kit: templates, varianceChart. The IBCS period-chart starter visual (same six-item layout).

Item	Type	What it is	Safety
`varianceChart/`	Folder	Period chart visual	Amber · AI
`varianceChart/src/`	Folder	Source code	Amber · AI
`varianceChart/src/visual.ts`	TypeScript	The drawing code for the period chart and its variance strip.	Red · AI
`varianceChart/style/`	Folder	Styles	Amber · AI
`varianceChart/style/visual.less`	Stylesheet	The period chart’s style sheet.	Red · AI
`varianceChart/capabilities.json`	Visual contract	Fields and settings for the period chart.	Red · AI

A.11 Custom-visual kit: templates, varianceTable

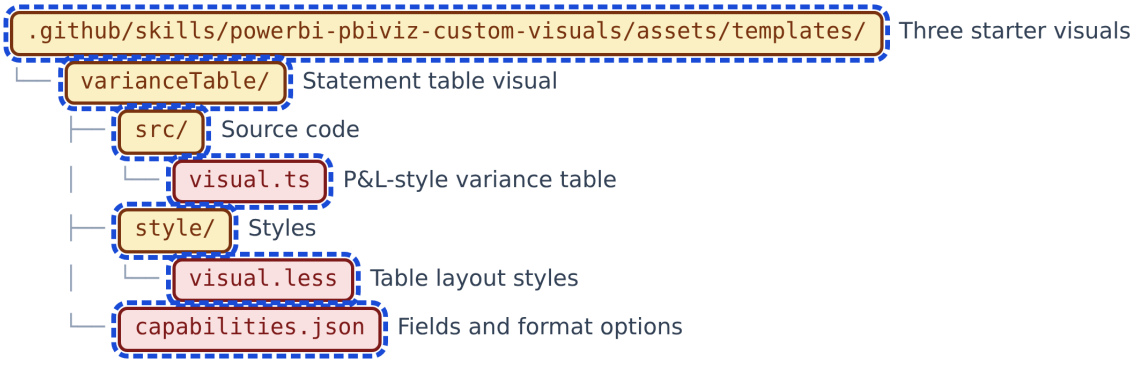


Figure A.11 — Custom-visual kit: templates, varianceTable. The IBCS statement-table starter visual (same six-item layout).

Item	Type	What it is	Safety
`varianceTable/`	Folder	Statement table visual	Amber · AI
`varianceTable/src/`	Folder	Source code	Amber · AI

Item	Type	What it is	Safety
`varianceTable/src/visual.ts`	TypeScript	The drawing code for the statement table.	Red · AI
`varianceTable/style/`	Folder	Styles	Amber · AI
`varianceTable/style/visual.less`	Stylesheet	The statement table’s style sheet.	Red · AI
`varianceTable/capabilities.json`	Visual contract	Fields and settings for the statement table.	Red · AI

A.12 Custom-visual kit: SKILL.md and references

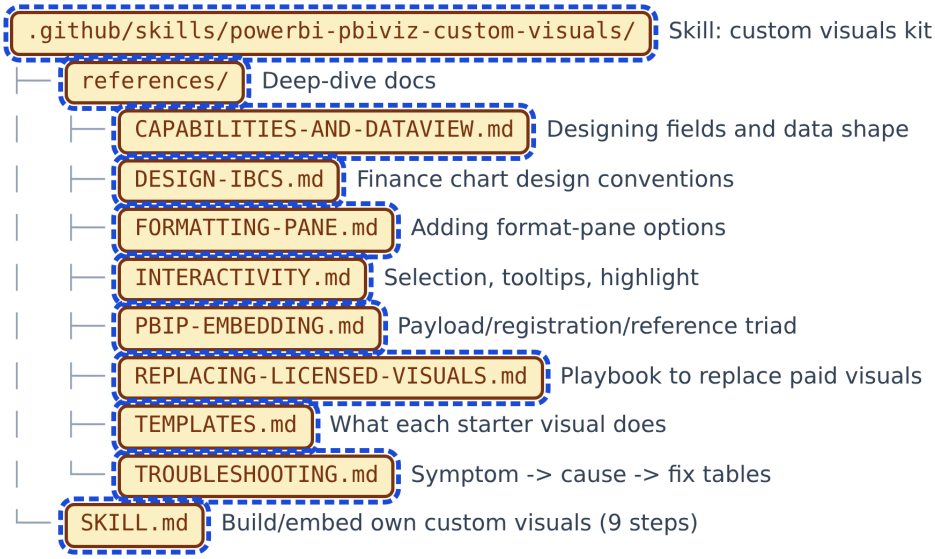


Figure A.12 — Custom-visual kit: SKILL.md and references. The skill procedure and its eight reference documents.

Item	Type	What it is	Safety
`powerbi-pbiviz-custom-visuals/`	Folder	Skill: custom visuals kit	Amber · AI
`references/`	Folder	Deep-dive docs	Amber · AI
`references/CAPABILITIES-AND-DATAVIEW.md`	Doc	How to decide which fields a visual takes and how the data arrives.	Amber · AI
`references/DESIGN-IBCS.md`	Doc	Design rules so custom visuals match the rest of the report.	Amber · AI
`references/FORMATTING-PANE.md`	Doc	How to add settings users can change in the Format pane.	Amber · AI

Item	Type	What it is	Safety
`references/INTERACTIVITY.md`	Doc	How to make clicks, tooltips and highlighting work.	Amber · AI
`references/PBIP-EMBEDDING.md`	Doc	How a built visual is stored inside the report so it opens anywhere.	Amber · AI
`references/REPLACING-LICENSED-VISUALS.md`	Doc	Step-by-step plan for replacing a paid chart whose licence expired.	Amber · AI
`references/TEMPLATES.md`	Doc	What each starter visual does and how to bind it.	Amber · AI
`references/TROUBLESHOOTING.md`	Doc	A symptom-to-fix table for when things go wrong.	Amber · AI
`SKILL.md`	Skill	The recipe for building, embedding and checking your own chart types.	Amber · AI

A.13 Template.Report: definition

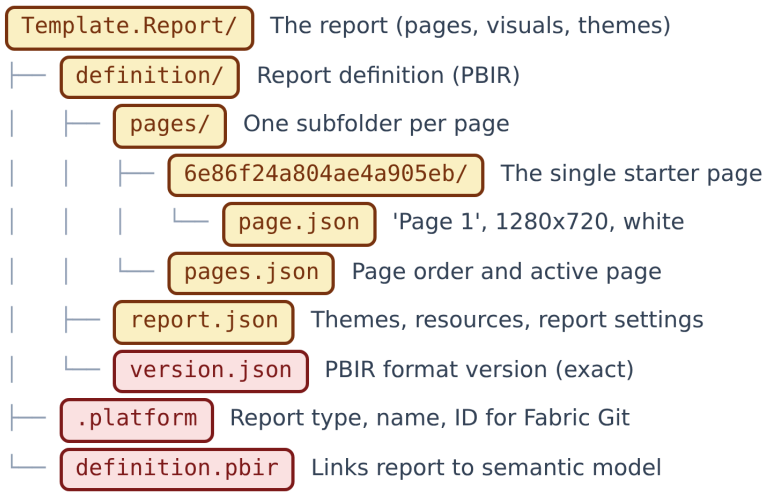


Figure A.13 — Template.Report: definition. The report's pointer file, ID card and definition files.

Item	Type	What it is	Safety
`definition/`	Folder	Report definition (PBIR)	Amber
`definition/pages/`	Folder	One subfolder per page	Amber
`definition/pages/6e86f24a804ae4a905eb/`	Folder	The single starter page	Amber
`definition/pages/6e86f24a804ae4a905eb/page.json`	Page config	Describes the blank starter page: name 'Page 1', size, white surround.	Amber

Item	Type	What it is	Safety
`definition/pages/pages.json`	Report config	The order of pages and which one opens first.	Amber
`definition/report.json`	Report config	Report-wide settings and the list of themes and resources.	Amber
`definition/version.json`	Report config	A tiny file stating the report format version. Must be exactly right.	Red
`.platform`	Fabric metadata	The report's ID card for Microsoft Fabric's Git sync.	Red
`definition.pbir`	Report pointer	Tells the report where its data model is.	Red

A.14 Template.Report: StaticResources

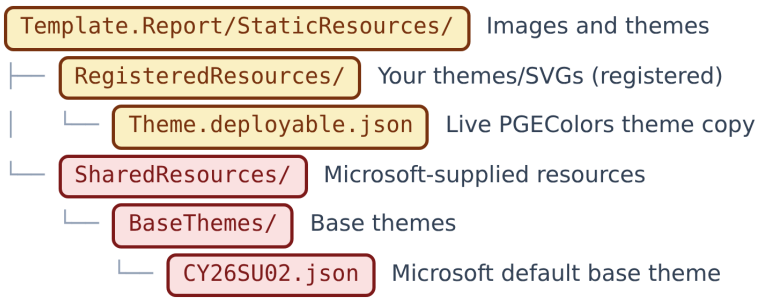


Figure A.14 — Template.Report: StaticResources. Themes and images used by the report.

Item	Type	What it is	Safety
`StaticResources/`	Folder	Images and themes	Amber
`RegisteredResources/`	Folder	Your themes/SVGs (registered)	Amber
`RegisteredResources/Theme.deployable.json`	Theme	The team colour theme the report actually uses.	Amber
`SharedResources/`	Folder	Microsoft-supplied resources	Red
`SharedResources/BaseThemes/`	Folder	Base themes	Red
`SharedResources/BaseThemes/CY26SU02.json`	Theme	Microsoft's default theme underneath the team theme.	Red

A.15 Template.SemanticModel

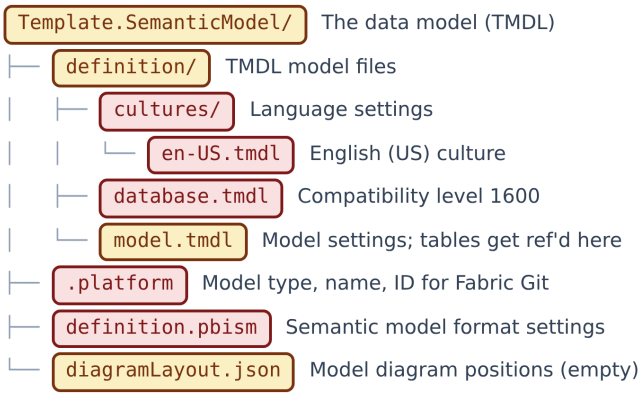


Figure A.15 — Template.SemanticModel. The data model, as TMDL text files.

Item	Type	What it is	Safety
`definition/`	Folder	TMDL model files	Amber
`definition/cultures/`	Folder	Language settings	Red
`definition/cultures/en-US.tmdl`	TMDL	Declares English (US) as the model language.	Red
`definition/database.tmdl`	TMDL	Sets the model engine’s compatibility level.	Red
`definition/model.tmdl`	TMDL	Model-wide settings; tables get listed here as they are added.	Amber
`.platform`	Fabric metadata	The model’s ID card for Microsoft Fabric’s Git sync.	Red
`definition.pbism`	Model pointer	Says which model format version this is.	Red
`diagramLayout.json`	Layout	Remembers where tables sit in Power BI’s model diagram. Empty for now.	Amber

A.16 docs: guides

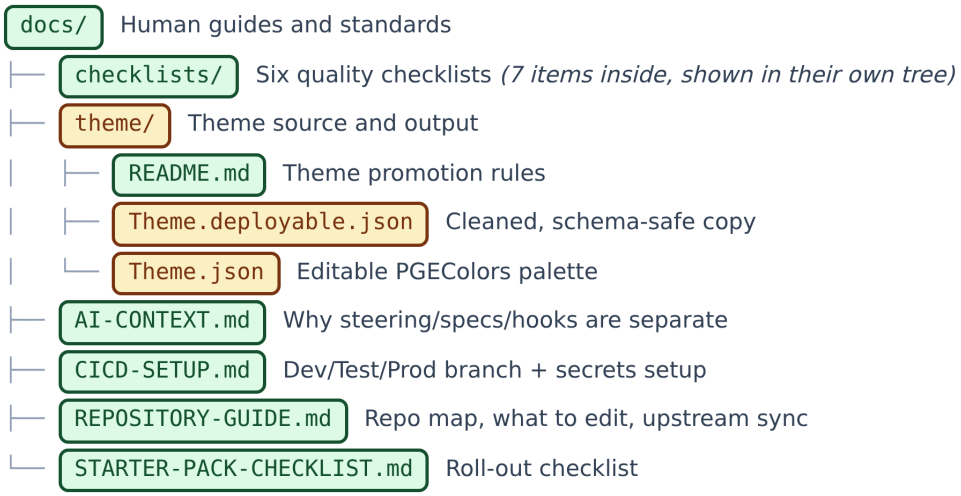


Figure A.16 — docs: guides. Human guides, the theme source, and the checklists folder. Folders shown closed are opened in their own tree.

Item	Type	What it is	Safety
`theme/`	Folder	Theme source and output	Amber
`theme/README.md`	Doc	Explains how the editable theme becomes the deployable one.	Green
`theme/Theme.deployable.json`	Theme output	The cleaned theme, safe to load into a report.	Amber
`theme/Theme.json`	Theme source	The editable team colour theme (PGEColors).	Amber
`AI-CONTEXT.md`	Doc	Explains why rules, plans and automatic checks are kept separate.	Green
`CICD-SETUP.md`	Doc	How to set up dev, test and prod branches, environments and secrets.	Green
`REPOSITORY-GUIDE.md`	Doc	A map of what to reuse, what to customise and how to get template updates.	Green
`STARTER-PACK-CHECKLIST.md`	Doc	A to-do list for turning the template into a complete team platform.	Green

A.17 docs/checklists

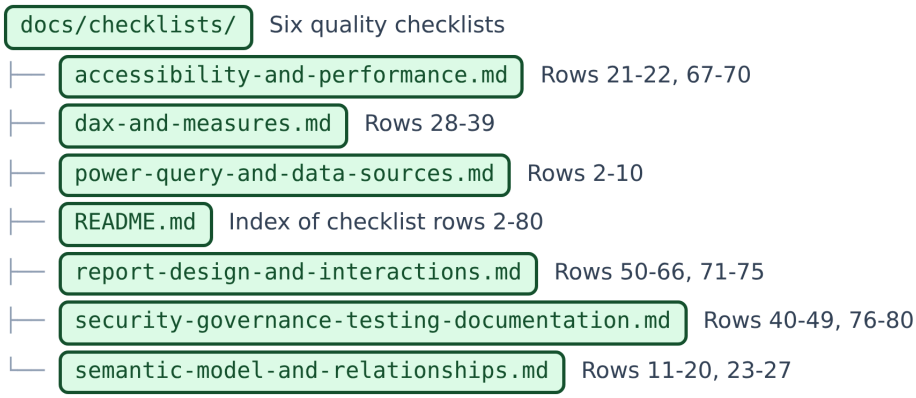


Figure A.17 — docs/checklists. Six focused quality checklists and their index.

Item	Type	What it is	Safety
`checklists/`	Folder	Six quality checklists	Green
`accessibility-and-performance.md`	Checklist	Checks for speed and for readability by everyone.	Green
`dax-and-measures.md`	Checklist	Checks for well-written calculations.	Green
`power-query-and-data-sources.md`	Checklist	Checks for how data is loaded and cleaned.	Green
`README.md`	Doc	The index of the six checklists and how to use them.	Green
`report-design-and-interactions.md`	Checklist	Checks for layout, visuals and clicks.	Green
`security-governance-testing-documentation.md`	Checklist	Checks for security, sign-off, testing and documentation.	Green
`semantic-model-and-relationships.md`	Checklist	Checks for a well-shaped data model.	Green

A.18 python

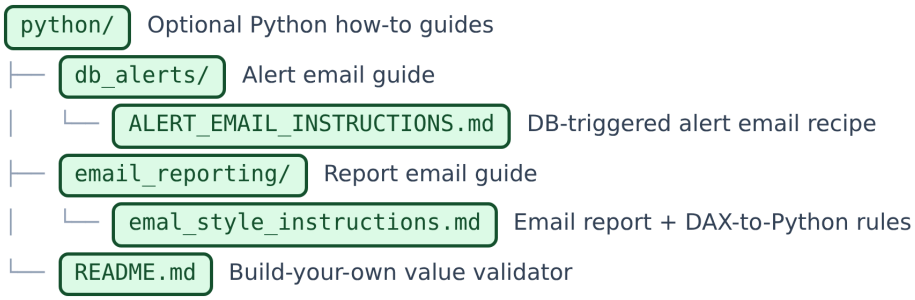


Figure A.18 — python. Optional Python guidance; no scripts ship here.

Item	Type	What it is	Safety
`db_alerts/`	Folder	Alert email guide	Green
`db_alerts/ALERT_EMAIL_INSTRUCTIONS.md`	Doc	A complete recipe for a database-triggered alert email.	Green
`email_reporting/`	Folder	Report email guide	Green
`email_reporting/emal_style_instructions.md`	Doc	Rules for emails that copy a Power BI page, including how to make Python numbers match DAX.	Green
`README.md`	Doc	How to write your own Python checker that compares dashboard numbers with the database.	Green

A.19 scripts

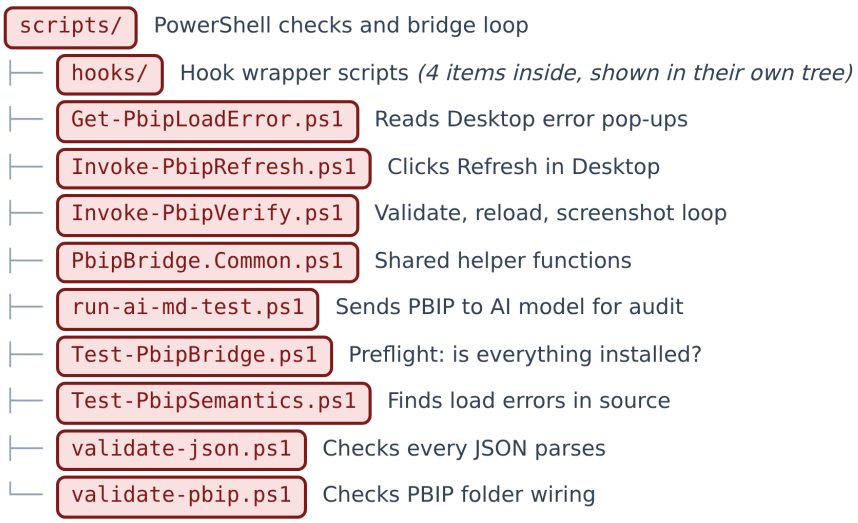


Figure A.19 — scripts. PowerShell validation and verification scripts. Folders shown closed are opened in their own tree.

Item	Type	What it is	Safety
`Get-PbipLoadError.ps1`	PowerShell	Reads Power BI's error pop-ups so they can be fixed.	Red
`Invoke-PbipRefresh.ps1`	PowerShell	Clicks Power BI's Refresh button so a new project shows data.	Red
`Invoke-PbipVerify.ps1`	PowerShell	Reloads Power BI and takes a screenshot of every page.	Red
`PbipBridge.Common.ps1`	PowerShell	Shared helper functions used by the other bridge scripts.	Red
`run-ai-md-test.ps1`	PowerShell	Sends the project files and a test prompt to an AI model and saves its report.	Red
`Test-PbipBridge.ps1`	PowerShell	A preflight check that everything needed for the screenshot loop is installed and open.	Red
`Test-PbipSemantics.ps1`	PowerShell	Finds, in the text, the errors Power BI would otherwise show as a pop-up.	Red
`validate-json.ps1`	PowerShell	Checks that every JSON-style file can be read.	Red
`validate-pbip.ps1`	PowerShell	Checks the project folders are wired together properly.	Red

A.20 scripts/hooks

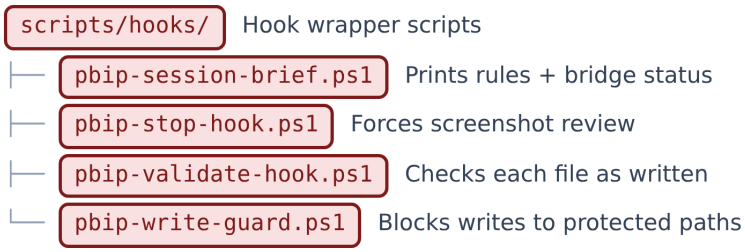


Figure A.20 — scripts/hooks. The four hook wrapper scripts.

Item	Type	What it is	Safety
`hooks/`	Folder	Hook wrapper scripts	Red
`pbip-session-brief.ps1`	Hook	Prints the house rules and whether Power BI's bridge is ready.	Red

Item	Type	What it is	Safety
`pbip-stop-hook.ps1`	Hook	Makes sure screenshots are reviewed before work is called finished.	Red
`pbip-validate-hook.ps1`	Hook	Checks each file the moment it is written.	Red
`pbip-write-guard.ps1`	Hook	Stops the agent writing into protected folders.	Red

A.21 specs

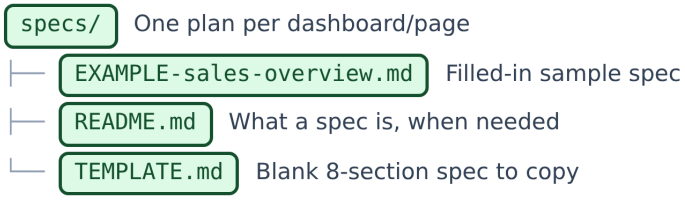


Figure A.21 — specs. Plans for one dashboard or page at a time.

Item	Type	What it is	Safety
`EXAMPLE-sales-overview.md`	Example	A filled-in example plan for a Sales Overview page.	Green
`README.md`	Doc	Explains what a spec is, when one is required, and the definition of done.	Green
`TEMPLATE.md`	Template	The blank eight-part plan to copy for every new page.	Green

A.22 tests: prompts

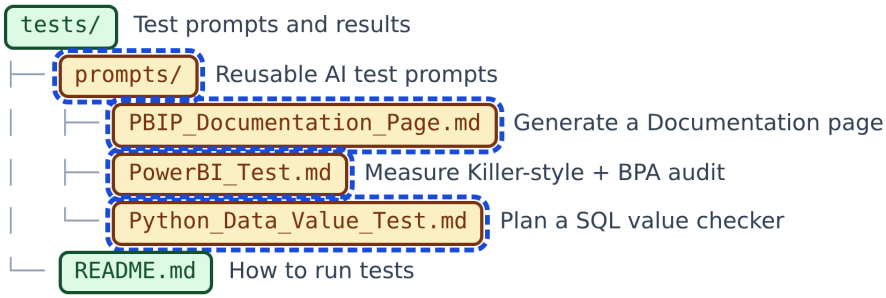


Figure A.22 — tests: prompts. The tests folder, its guide and the three AI prompts.

Item	Type	What it is	Safety
`prompts/`	Folder	Reusable AI test prompts	Amber · AI

Item	Type	What it is	Safety
`prompts/PBIP_Documentation_Page.md`	AI prompt	The AI prompt that builds a Documentation page summarising the project.	Amber · AI
`prompts/PowerBI_Test.md`	AI prompt	The AI audit prompt that looks for unused or broken model parts.	Amber · AI
`prompts/Python_Data_Value_Test.md`	AI prompt	The AI prompt that plans (but does not write) a Python number checker.	Amber · AI
`README.md`	Doc	How to run the tests locally and on GitHub.	Green

A.23 tests/results

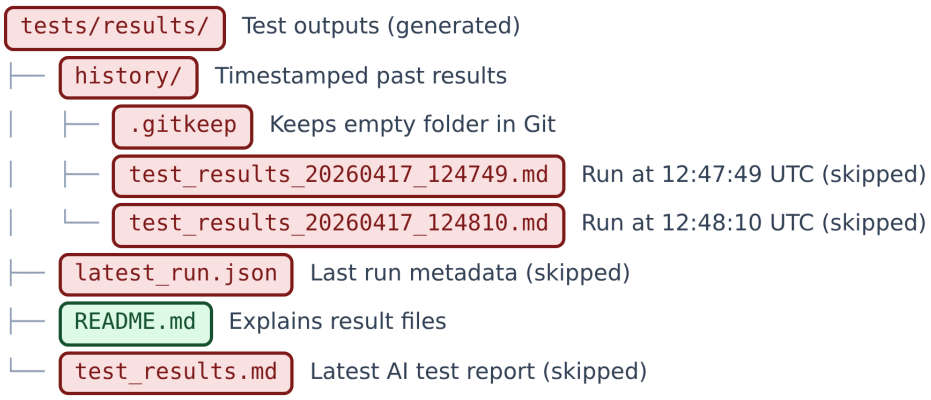


Figure A.23 — tests/results. Generated test reports (Red: written by scripts).

Item	Type	What it is	Safety
`results/`	Folder	Test outputs (generated)	Red
`history/`	Folder	Timestamped past results	Red
`history/.gitkeep`	Placeholder	An empty file that keeps the empty folder in Git.	Red
`history/test_results_20260417_124749.md`	Generated	A past test report from 17 April 2026, 12:47:49 UTC (skipped).	Red
`history/test_results_20260417_124810.md`	Generated	A past test report from 17 April 2026, 12:48:10 UTC (skipped).	Red
`latest_run.json`	Generated	Details of the latest test run.	Red
`README.md`	Doc	Explains the result files.	Green

Item	Type	What it is	Safety
`test_results.md`	Generated	The latest AI test report; today it says the test was skipped for lack of a key.	Red

Coverage: 176 of 176 items listed.

Appendix B. Verified official references

Group	Official site	Links
Power BI and Microsoft	`learn.microsoft.com`	17
Git and GitHub	`git-scm.com, docs.github.com`	12
VS Code and Copilot in VS Code	`code.visualstudio.com`	6
Claude and Anthropic	`code.claude.com, platform.claude.com`	4
Other tools	`nodejs.org, python.org, github.com/microsoft`	3

Every link was opened on 10 November 2026 and its content checked. Links that failed to open were left out.

Figure B.1 — Where the references come from. Every link below was opened on 10 November 2026 and the fact this guide uses was checked against the page. Links that wouldn’t open (for example the npm pages for the two Power BI CLIs) are left out.

Power BI and Microsoft (Microsoft Learn)

Page	Use it for
Power BI Desktop projects (PBIP)	What a `.pbip` project is and what saving one creates
Power BI Desktop project report folder	The PBIR files: `definition.pbir`, `report.json`, `pages.json`, `page.json`, `visual.json`
Power BI Desktop project semantic model folder	`definition.pbism`, the TMDL folder, and which `.pbi` files Git ignores
Git integration with Power BI Desktop projects	Putting a PBIP folder under Git in VS Code
Tabular Model Definition Language (TMDL)	How TMDL files are structured
Use report themes in Power BI	Theme JSON files and `dataColors`
What is the Power BI Desktop Bridge?	The bridge, its Security setting, and the `powerbi-desktop` CLI commands
Download Power BI Desktop	Installing Power BI Desktop and its system requirements

Page	Use it for
Set up an environment for developing a Power BI visual	Node.js and the `pbiviz` tools for custom visuals
Capabilities and properties of Power BI visuals	What `capabilities.json` declares
Use field parameters in Power BI reports	The feature behind the field-parameter skill
Row-level security (RLS) with Power BI	Restricting which rows each person sees
Understand star schema and the importance for Power BI	Fact and dimension tables
Use Performance Analyzer to examine report performance	Timing visuals for the performance skill
DAX overview	The DAX formula language
Overview of Fabric Git integration	Syncing Fabric workspaces with Git
What is PowerShell?	The shell the repository's scripts use

The six files in `docs/checklists/` also have a Microsoft Learn link for every checklist row.

Git and GitHub

Page	Use it for
Git book: About Version Control	What version control is
What is GitHub?	How Git and GitHub differ
Fork a repository	Forking, and adding the `upstream` remote
About pull requests	Proposing and reviewing changes
Ignoring files	How `.gitignore` works
About code owners	How `CODEOWNERS` requests reviewers
Understanding GitHub Actions	Workflows, jobs and runners
Events that trigger workflows	`push` and `workflow_dispatch`
Using secrets in GitHub Actions	Adding the API-key secrets
Adding repository custom instructions for GitHub Copilot	`.github/copilot-instructions.md`, `.instructions.md` files and `AGENTS.md`
About agent skills	Where Copilot looks for skills
Installing GitHub Copilot CLI	Installing Copilot in the terminal

VS Code and Copilot in VS Code

Page	Use it for
Download Visual Studio Code	Installing VS Code

Page	Use it for
Source control in VS Code	Committing and pushing from the editor
AI in VS Code overview	What an agent does in VS Code
Use custom instructions in VS Code	Instruction files and `applyTo` patterns
Custom agents in VS Code	`.agent.md` expert modes
Use Agent Skills in VS Code	`SKILL.md` format: `name` and `description`

Claude and Anthropic

Page	Use it for
Claude Code overview	What Claude Code is
Claude Code hooks reference	Hook events and exit code 2, which this repository's hook scripts follow
How Claude remembers your project	`CLAUDE.md`, and when Claude reads `AGENTS.md`
Agent Skills overview	How skills work for Claude

Other tools

Page	Use it for
Download Node.js	Installing Node.js and npm
Download Python	Installing Python 3
microsoft/skills-for-fabric	The Copilot CLI plugin collection named in the README

For Git itself, use the download link on the Git website's home page (`git-scm.com`); the direct downloads address only showed a redirect notice when checked.

Appendix C. Glossary

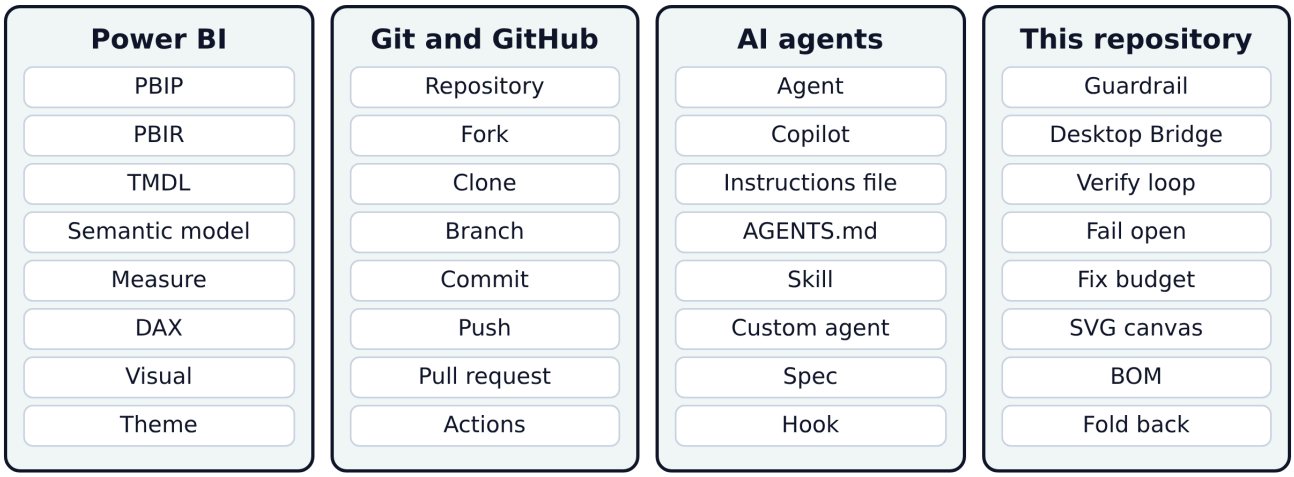
Every tool-specific term used in this guide, A to Z, with the chapter where it is explained. A summary by family follows the table.

Term	Meaning	Chapter
Actions (GitHub Actions)	GitHub's automation service; runs workflows on cloud machines	10
Agent (AI coding agent)	An AI assistant that reads files, edits them and runs commands	3
AGENTS.md	A cross-tool instruction file for AI agents	3
applyTo	The file pattern that decides when an `.instructions.md` file loads	3

Term	Meaning	Chapter
Artifact	A file bundle a workflow saves for download	10
BOM	Byte-order mark: an invisible marker at a file's start; TMDL files must not have one	1
BPA	Best Practice Analyzer: Microsoft's rule set for checking data models	10
Branch	A separate line of work in Git	2
`card_y + 30` rule	A visual must start 30 px below its card's top so it doesn't cover the title	7
Clone	Download a full copy of a repository with its history	2
CODEOWNERS	A GitHub file naming who reviews which folders	2
Commit	A saved snapshot of changes, with a message	2
Copilot instructions	A Markdown file of standing rules Copilot adds to every request (`.github/copilot-instructions.md`)	3
Custom agent	A saved expert role for Copilot (`.agent.md`)	3
Custom visual	A chart type you write in TypeScript and package as `.pbiviz`	11
DAX	Data Analysis Expressions, Power BI's formula language	1
Desktop Bridge	A local connection that lets programs control a running Power BI Desktop	4
Dimension / fact table	Tables of things (products, dates) / tables of events (sales)	1
Fabric	Microsoft's online analytics platform; `.platform` files are for its Git sync	1
Fail open	When a check breaks, it lets you continue instead of blocking	5
Field parameter	A Power BI feature that lets readers switch which field a visual shows	A
Fix budget	The verify loop allows one fix round, then lets the turn end	8
Fold back	Move a rule from a spec into the permanent rulebook	5

Term	Meaning	Chapter
Fork	Your own copy of someone else’s repository on GitHub	2
Guardrail	One of the repo’s non-negotiable rules	5
GUID	A long unique identifier; a custom visual’s permanent id	11
Hook	A small script that runs at a fixed moment, such as before a file is written	5
IBCS	A design standard for business charts	11
Image visual	A visual that shows a picture; here, an SVG returned by a measure	7
KPI	Key performance indicator: one headline number	7
Measure	A named calculation, such as “Total Sales”	1
PBIP	Power BI Project: a report saved as a folder of text files	1
PBIR	Power BI’s enhanced report format: the report as JSON files	1
`.pbix`	Power BI’s classic single-file format	1
Power Query	Power BI’s tool for loading and shaping data	1
Preflight	`Test-PbipBridge.ps1`: checks everything is ready	6
Pull request	A request to review and merge a branch	2
Runner	The cloud machine that runs a GitHub workflow	10
Secret	A hidden setting a workflow can use but nobody can read	10
Semantic model	The tables, relationships and measures behind a report	1
Skill	A folder with a `SKILL.md` procedure the agent loads when the task matches	3
Spec	Your plan for one dashboard or page, written before building	5
Star schema	Fact tables in the middle, dimension tables around them	1
Steering	The permanent rule files that guide every session	5

Term	Meaning	Chapter
SVG background	A page background image that draws cards and titles	7
Theme	A JSON file of colours and fonts for a report	9
TMDL	Tabular Model Definition Language: the model as indented text	1
Upstream / origin	The original repository / your fork	2
Verify loop	Validate, reload Power BI, screenshot every page, review, fix	8
Visual	One chart, table, card or image on a page	1
Workflow	An automated process defined in a YAML file	10



The core terms in four families.

Figure C.1 — The same terms in four families. Power BI words describe the report and its data; Git and GitHub words describe saving and sharing; AI-agent words describe how the assistant is guided; the last column is this repo’s own vocabulary.